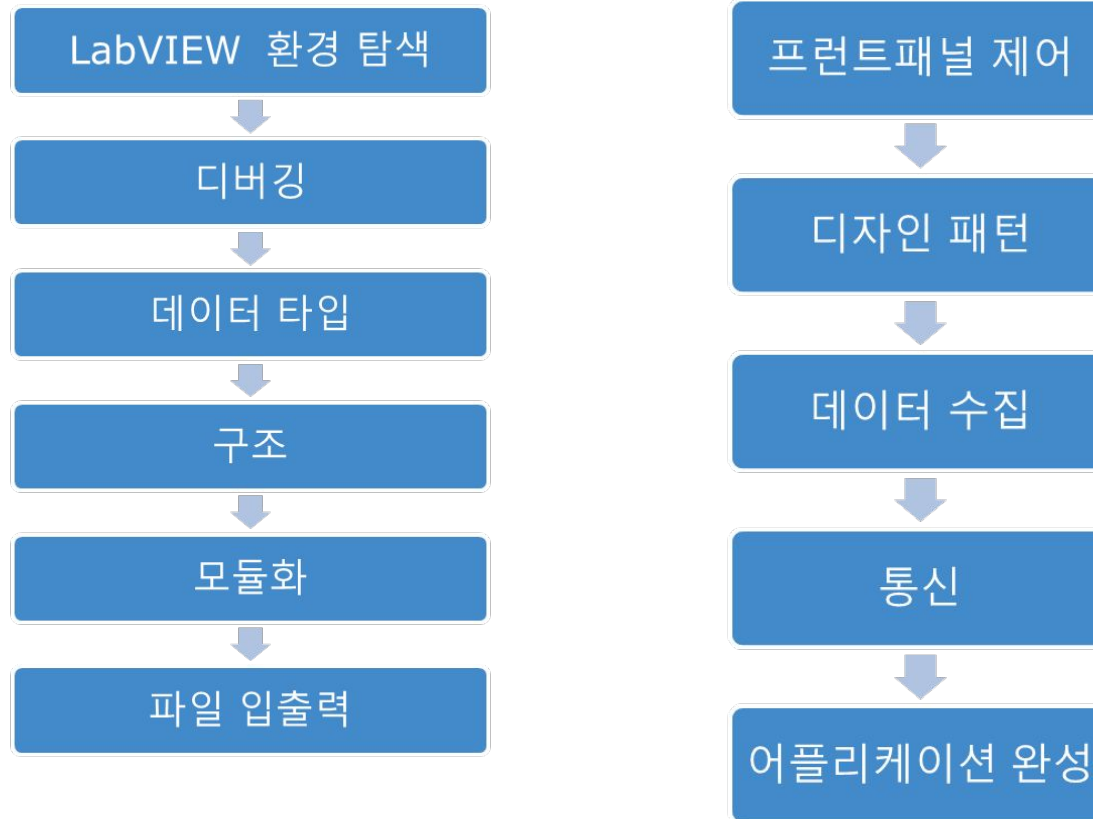


LabVIEW 강의 자료

신안산대학 강의자료

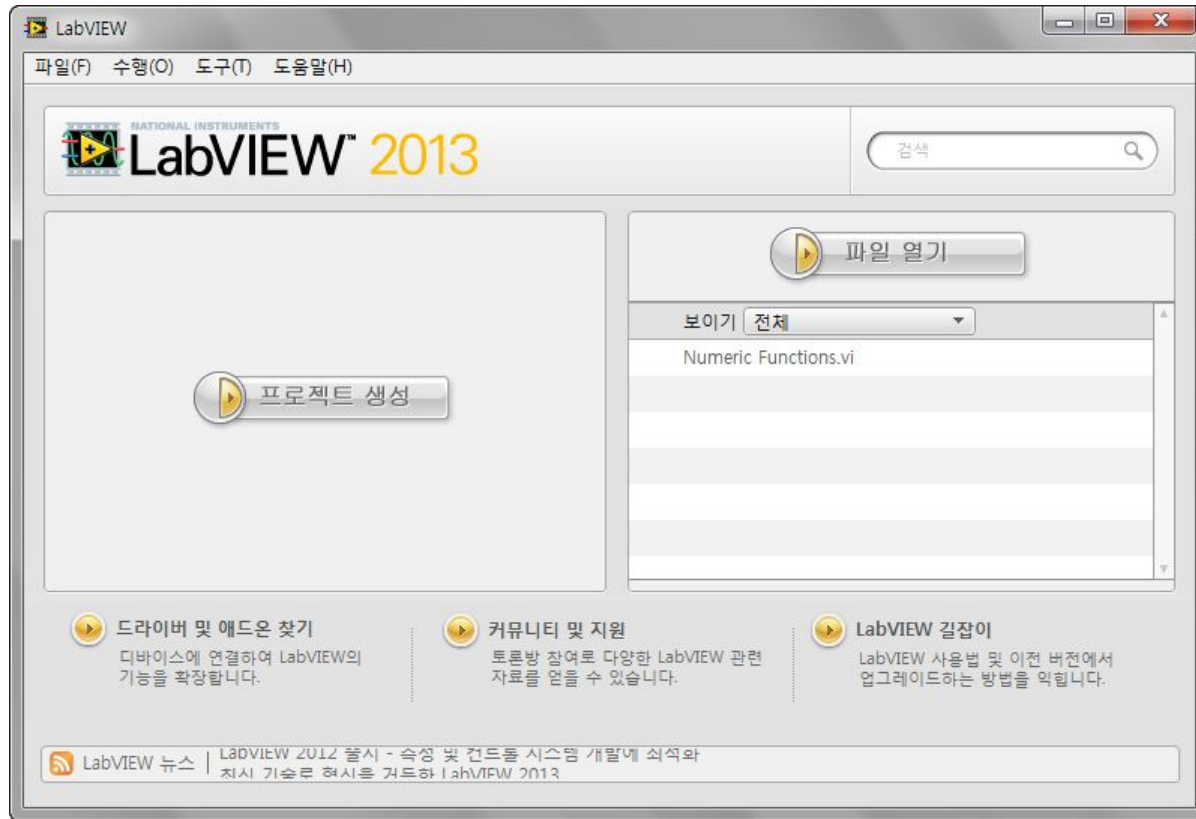
임성국

목차

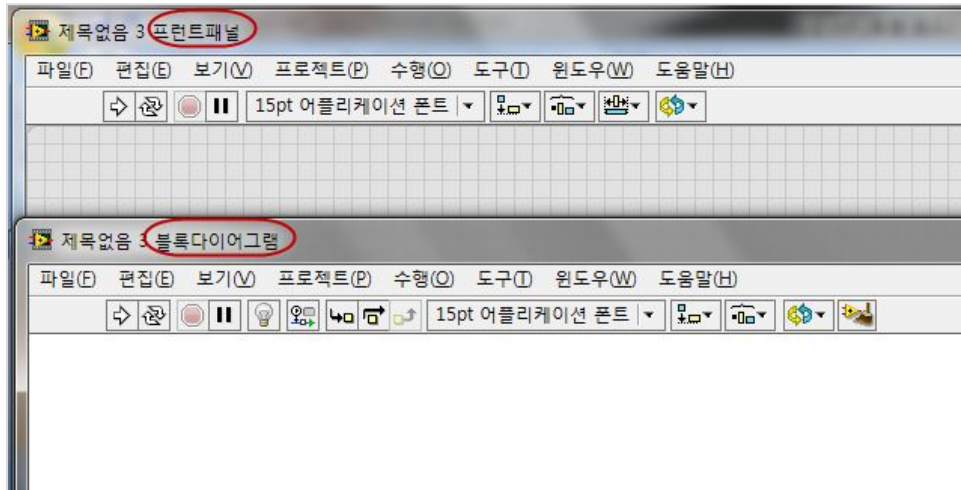


1. LabVIEW 환경 탐색

LabVIEW 시작 화면



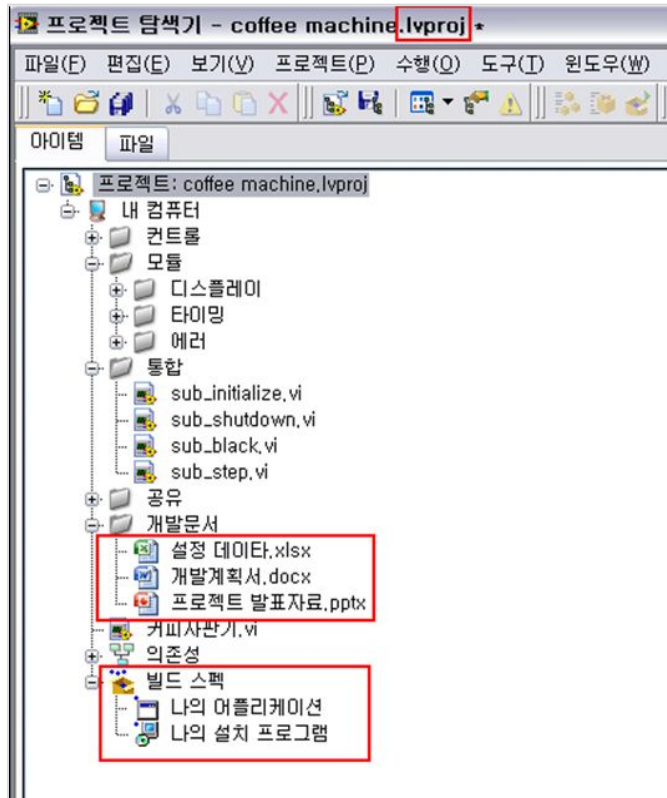
VI (Virtual Instruments)



- 프론트패널 : 사용자 인터페이스 공간
- 블록다이어그램 : 소스 코드 작성 공간

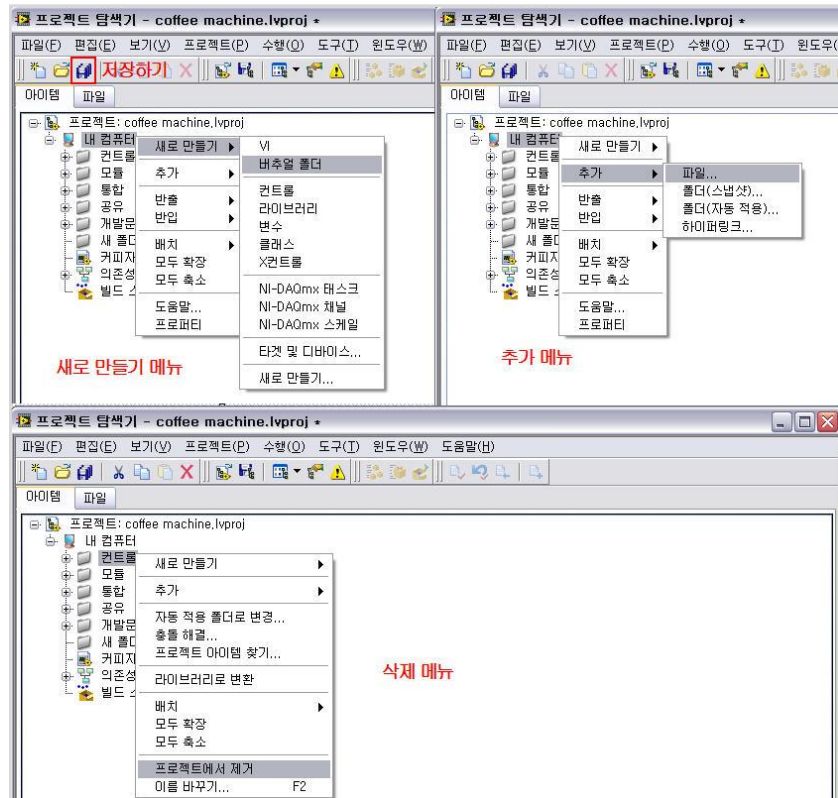
실습1-1-1) VI 만들기
실습1-2-1) VI 저장하기

프로젝트 (.lvproj)

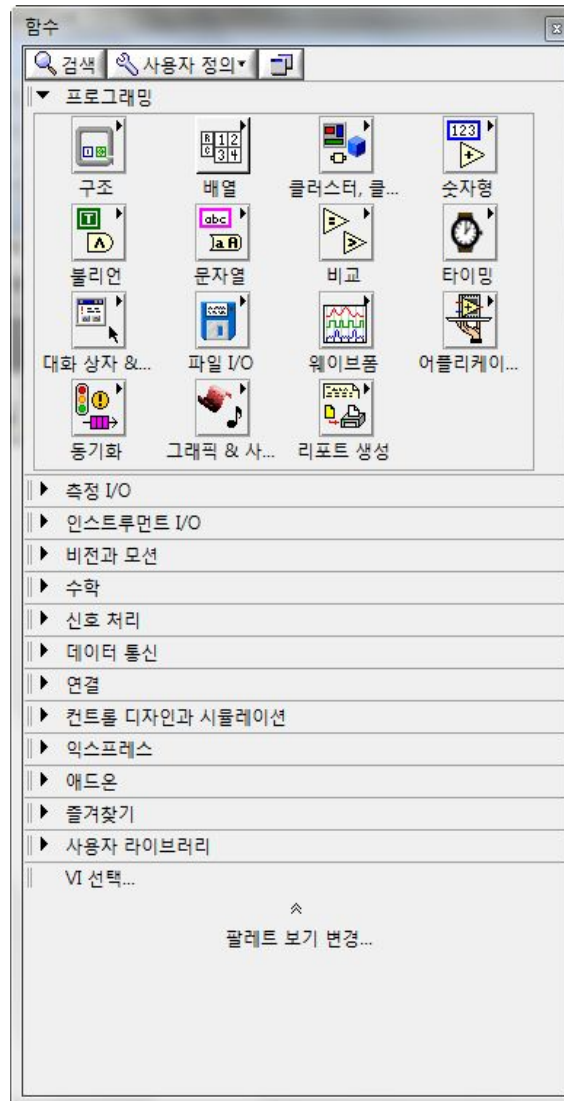
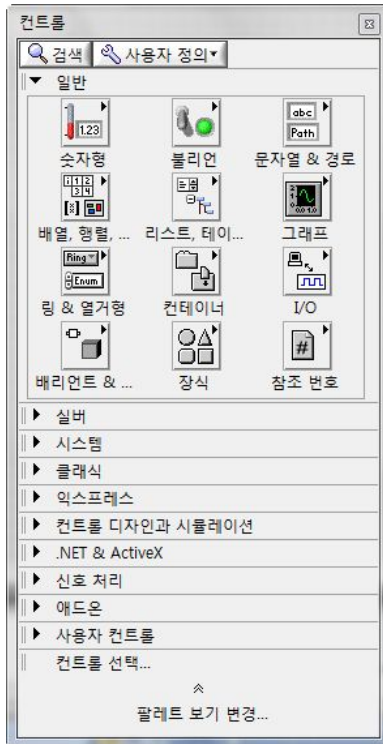


- 프로젝트 산출물 관리가 편리함
- 실행파일 (**exe**)과 설치파일 생성시 필요함

프로젝트 파일 생성, 추가, 삭제



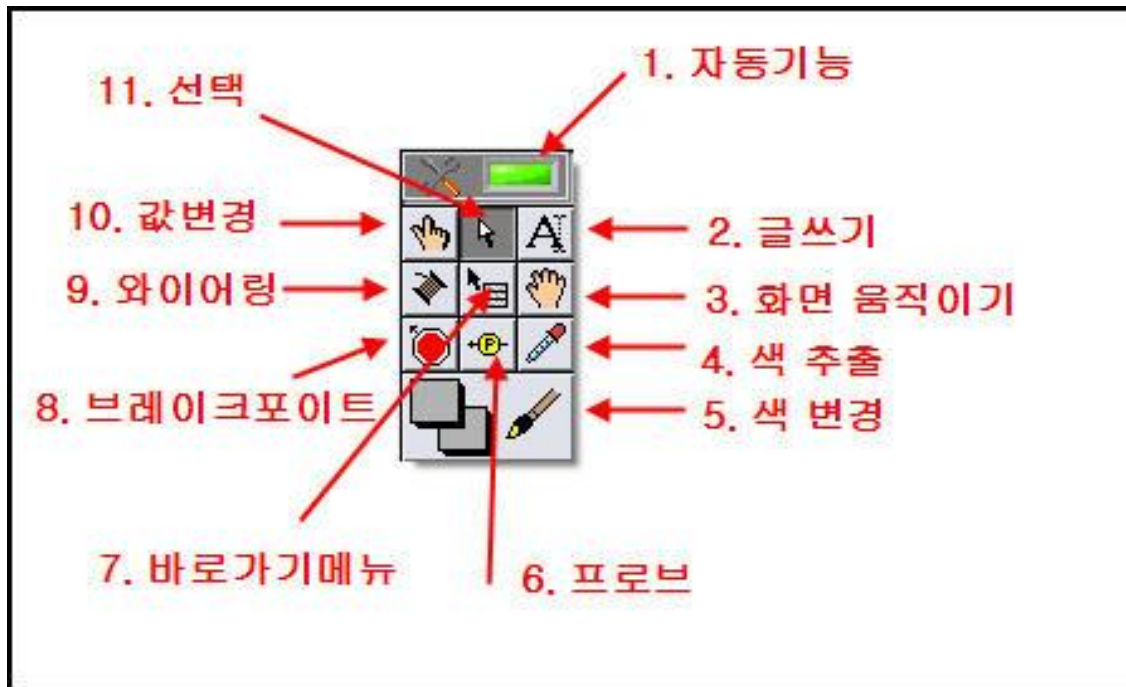
컨트롤/함수/도구 팔레트



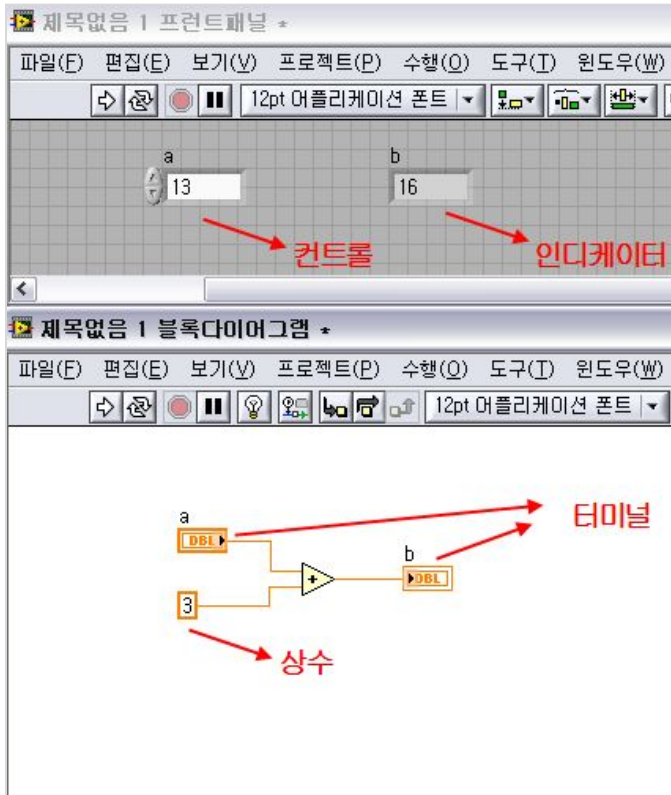
- 컨트롤 팔레트 : 컨트롤과 인디케이터 제공
- 함수 팔레트 : 소스 코드 제공
- 도구 팔레트 : 마우스 기능 변경

실습 1-6-1) 팔레트 사용해보기

도구 팔레트 기능



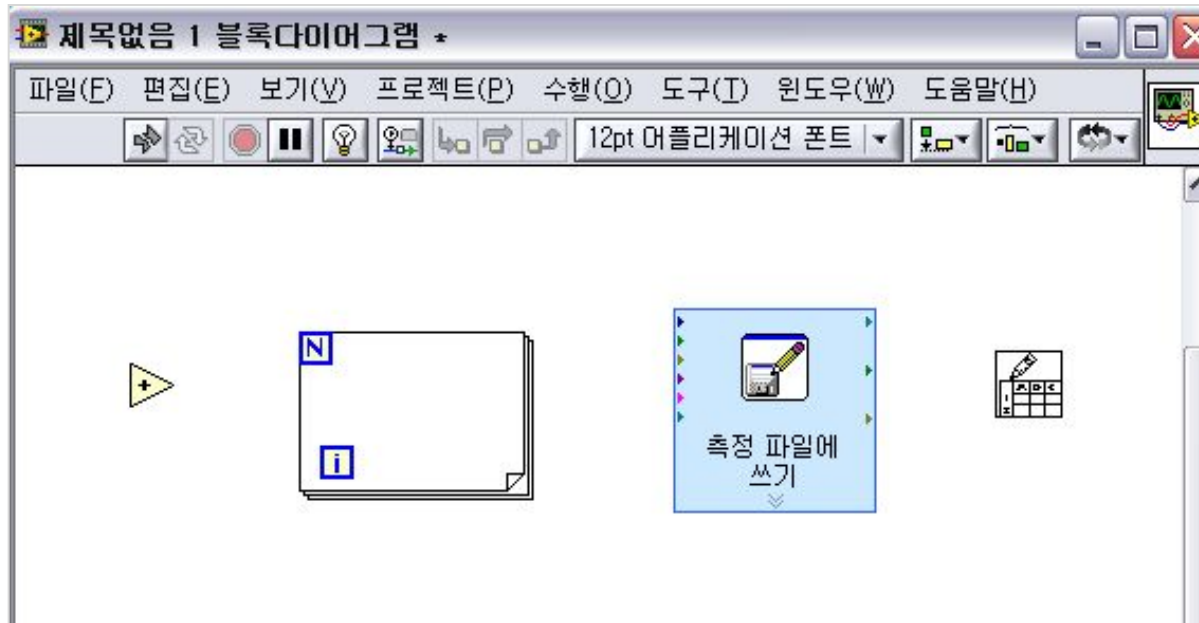
컨트롤/인디케이터/터미널/상수



- 컨트롤 : 입력
- 인디케이터 : 출력
- 터미널 : 컨트롤과 인디케이터와
매칭
- 상수 : 변하지 않는 값

실습1-7-1) 컨트롤 인디케이터 터미널 만들기

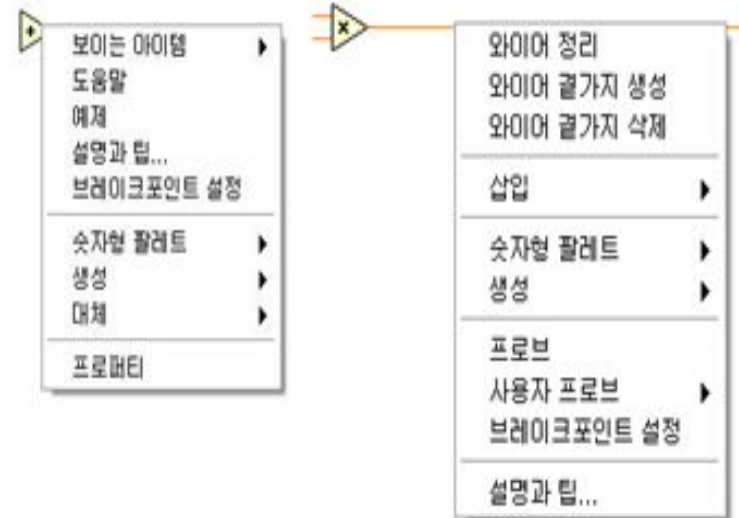
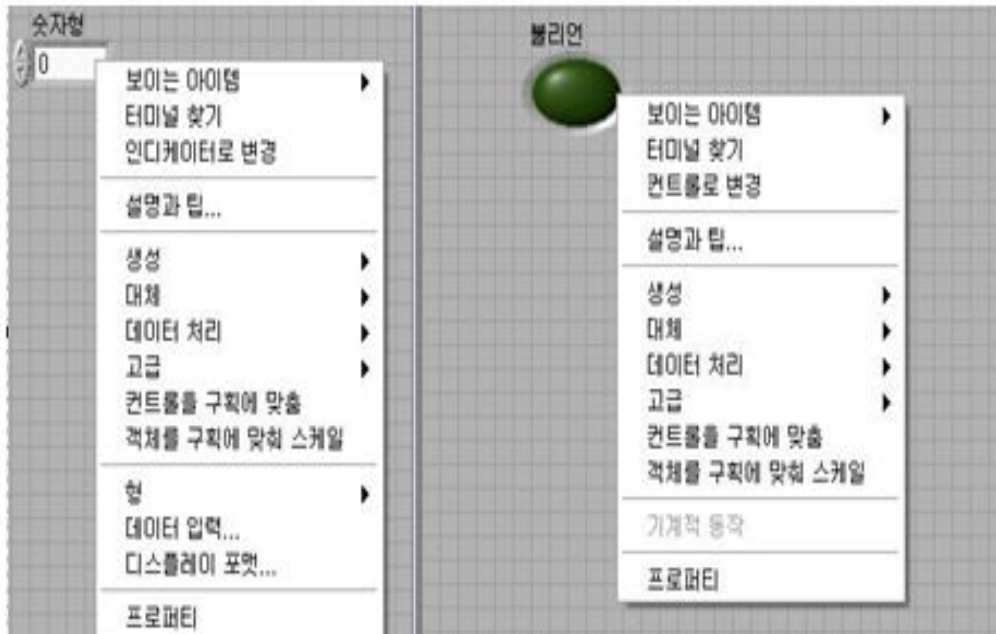
노드



- 노드 : 함수 팔레트에서 상수를 제외한 모든 함수들을 의미함.

실습1-8-1) 노드와 와이어링

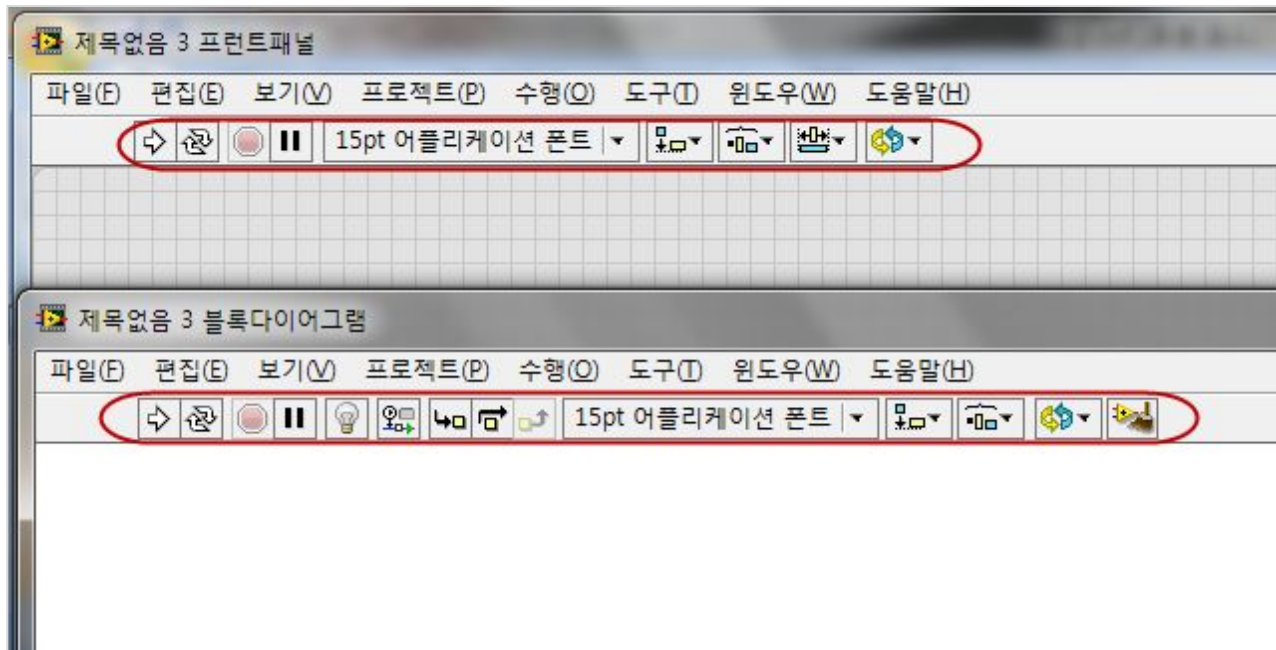
바로가기 메뉴



- 속성 변경 또는 편리한 기능 등을 모아 놓은 메뉴.

실습1-9-1) 바로가기 메뉴 사용하기

도구 모음



도구 모음 기능



실행 버튼



연속 실행 버튼



실행 강제 종료 버튼



일시 정지 버튼



텍스트 셋팅 메뉴



객체 정렬 메뉴



객체 간격 조절 메뉴



순서 재설정 메뉴



객체 크기 조절 메뉴



실행 하이라이트 버튼



단계별 실행 시작 버튼



단계별 실행 시작 버튼

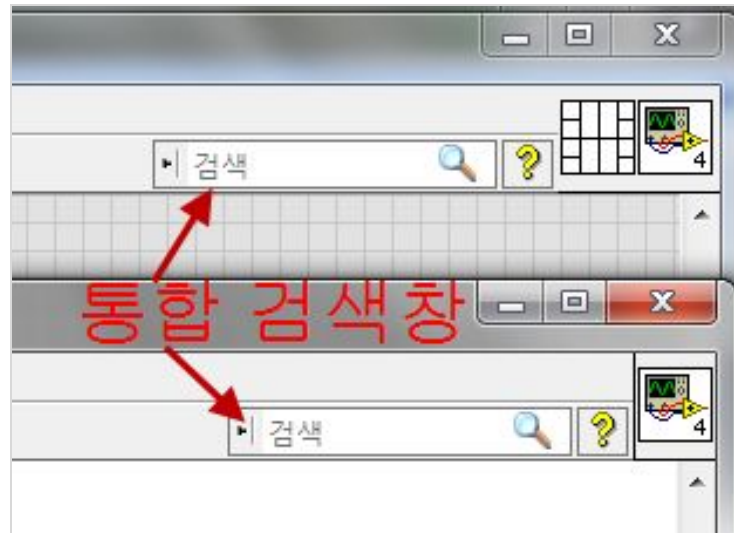


단계별 실행 나가기 버튼



다이어그램 정리

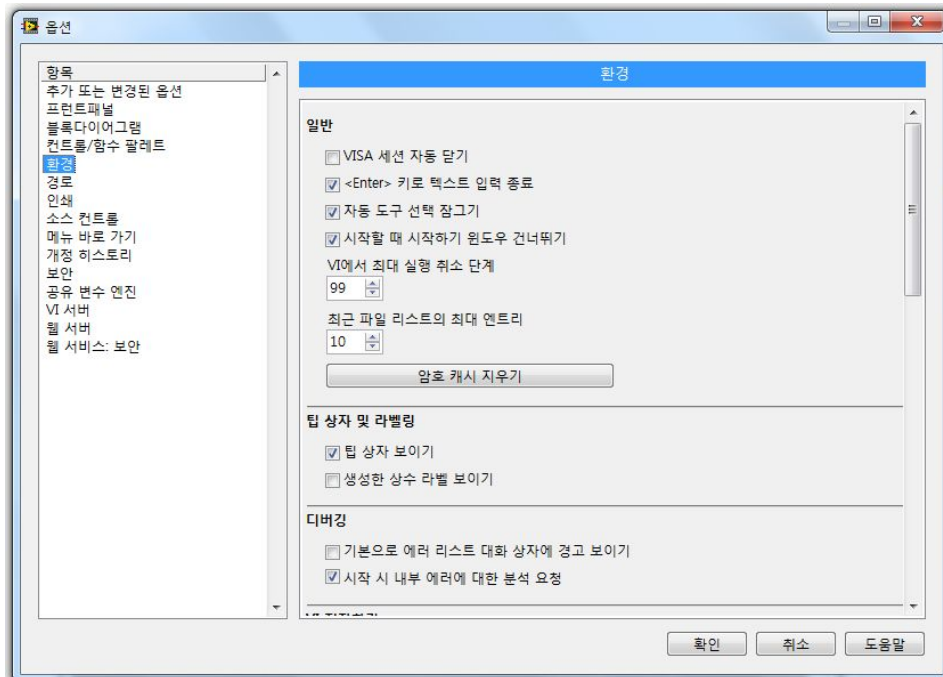
통합 검색창



- 통합 검색창에 키워드를 주면 **ni.com**, 팔레트, 도움말을 검색하여 결과를 보여줌.

실습1-10-1) 도구 모음 사용

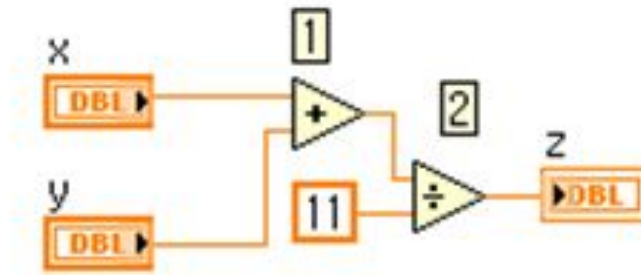
LabVIEW 환경 설정



- LabVIEW의 전체적 환경 설정에 영향을 줌.

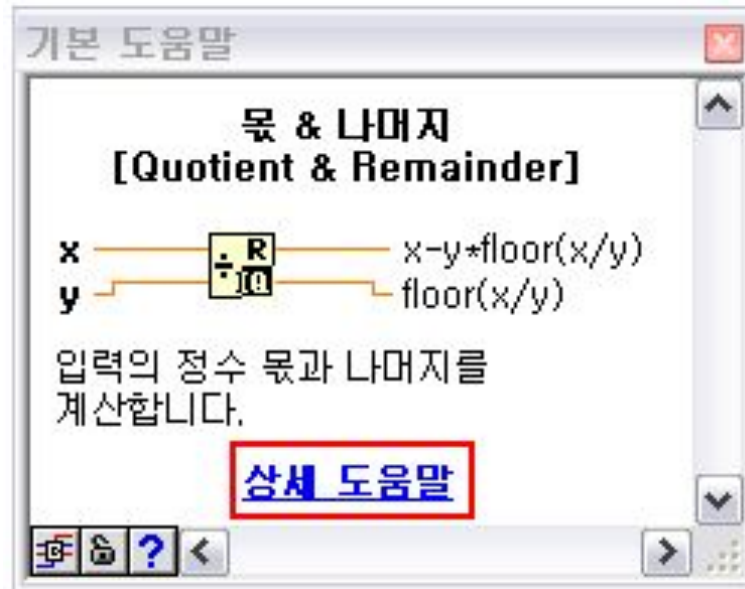
데이터 흐름

- ◆ 노드가 실행되려면 모든 입력으로부터 모두 값이 들어와야 한다.
- ◆ 노드 실행이 끝나야만 출력을 내보낼 수 있다.



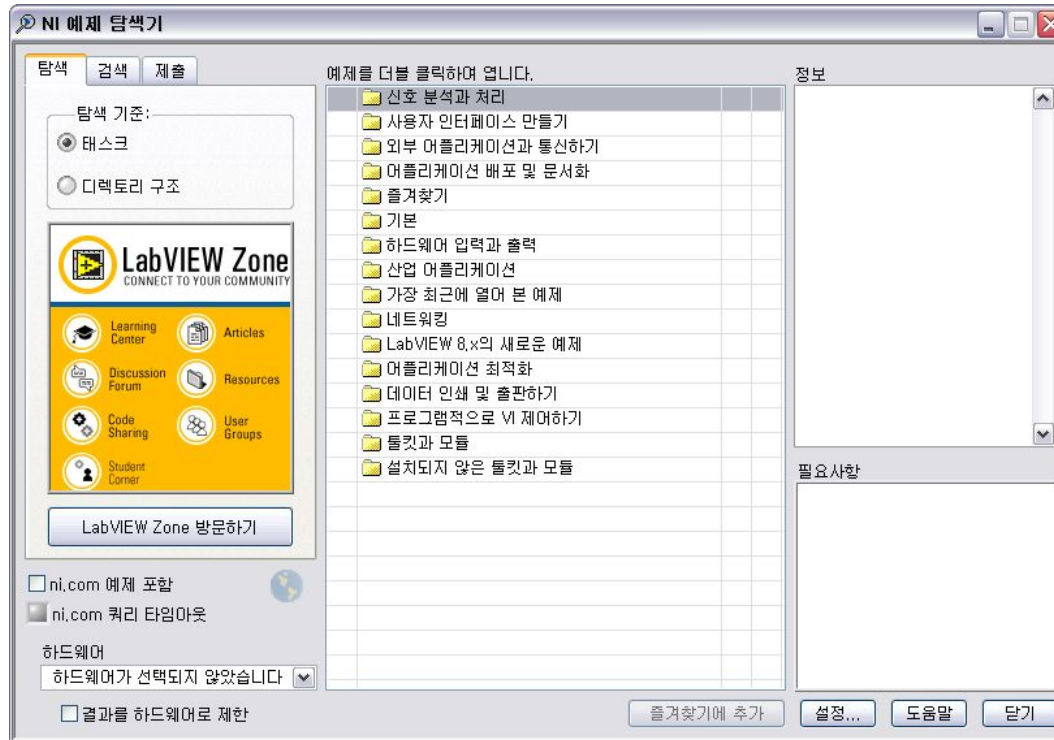
실습1-12-1) 데이터 흐름

기본 도움말

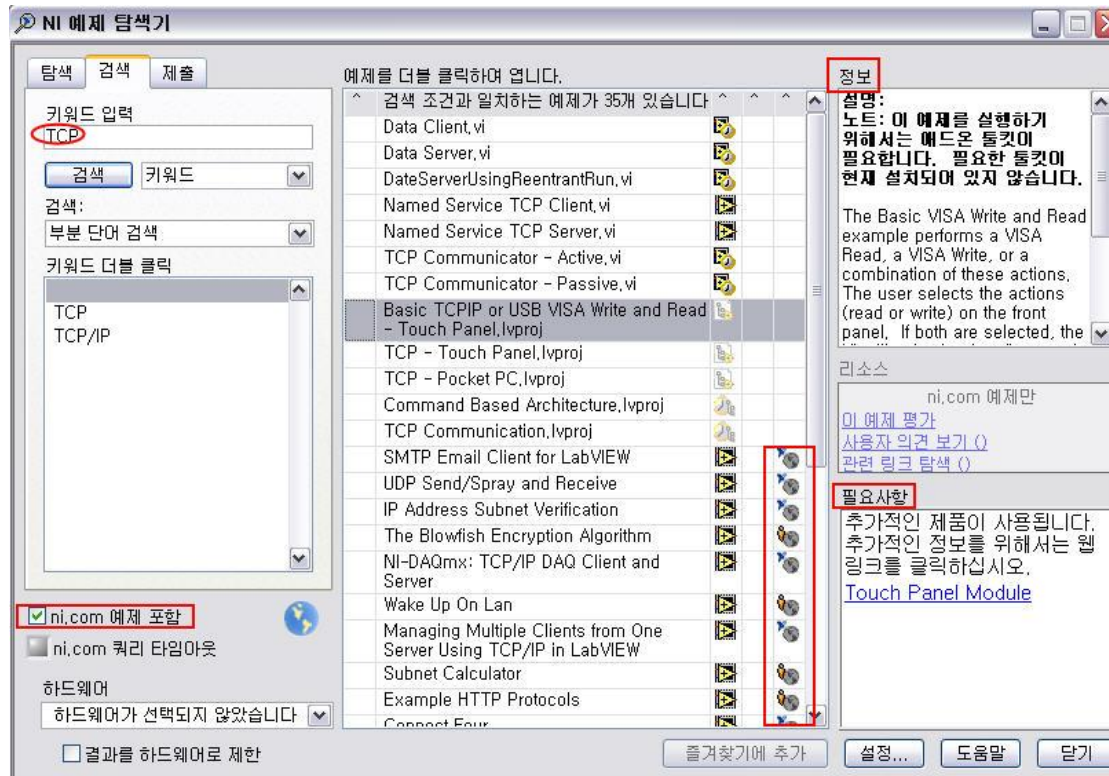


실습1-13-1) 기본 도움말

예제 찾기



예제 검색



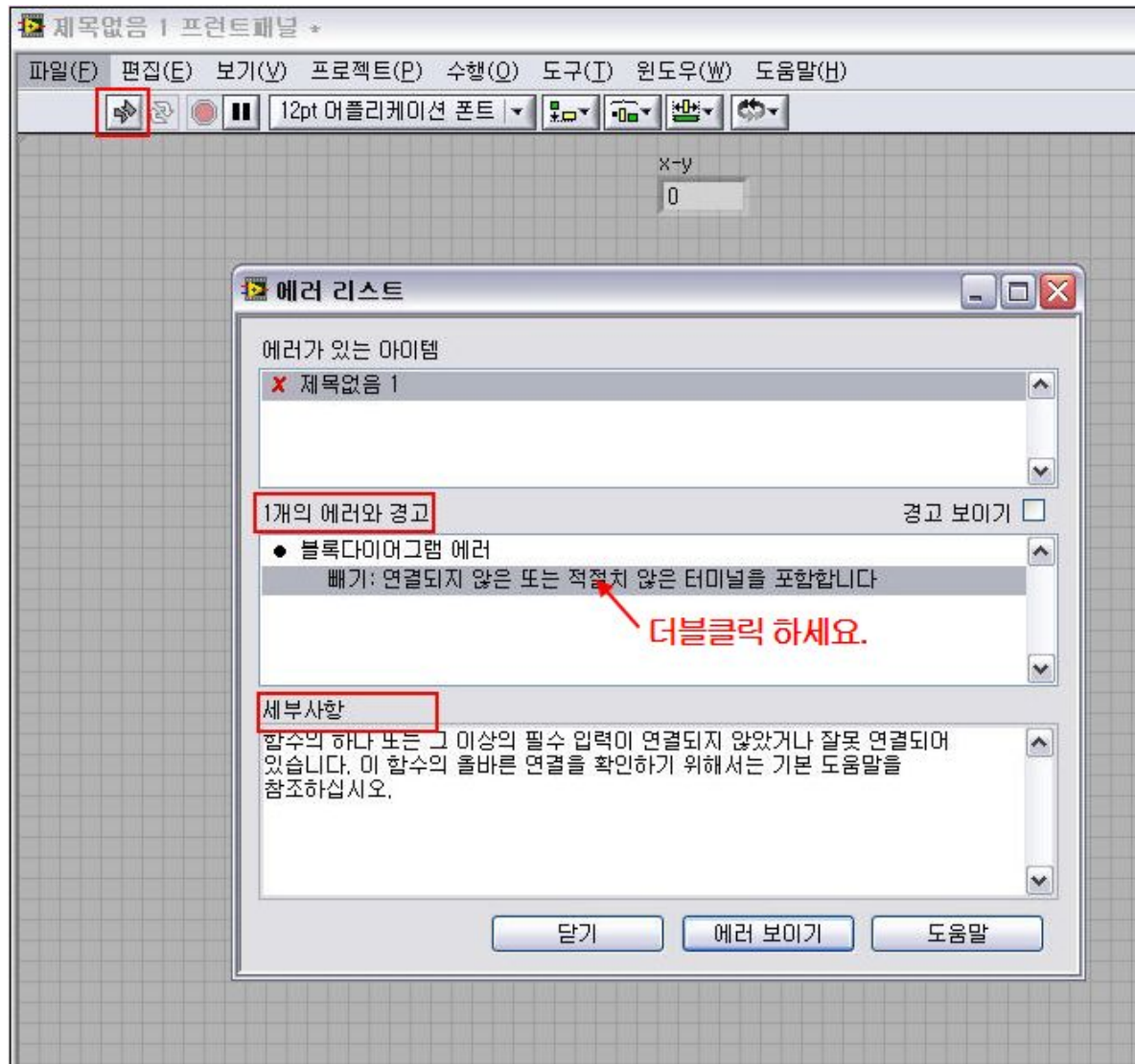
실습1-14-1) 예제 찾기

1장 요약

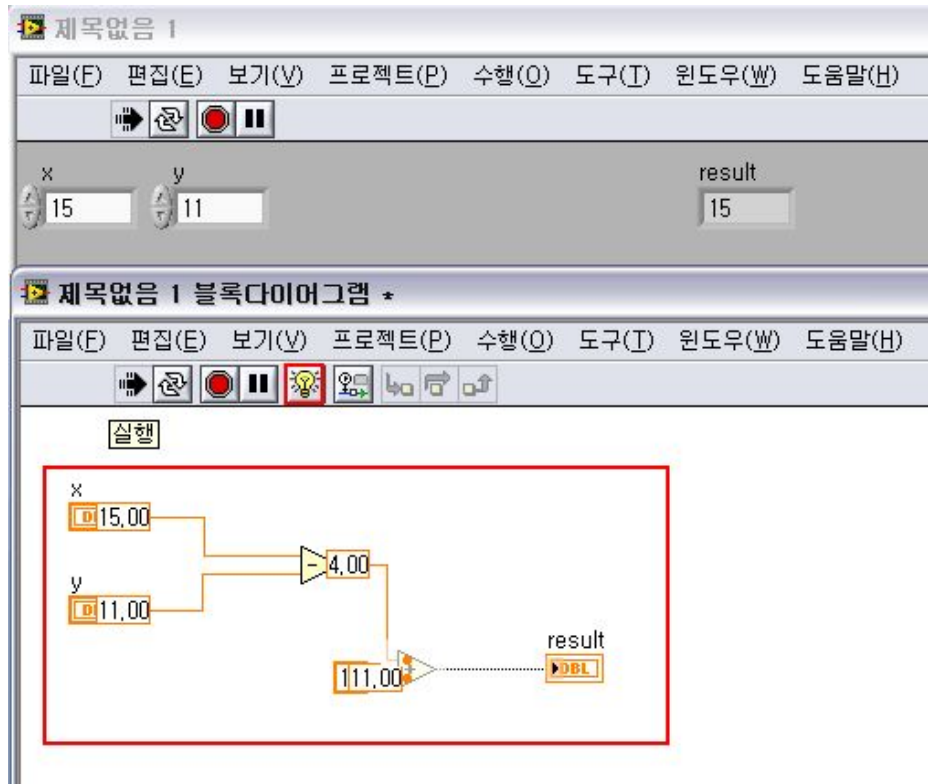
- **LabVIEW 확장자 *.VI**
- **프런트패널** - 사용자 인터페이스 구성 공간
- **블록다이어그램** - 소스코드 작성하는 공간
- **프로젝트 파일 *.lvproj**
- **컨트롤, 함수, 도구 팔레트**
- **컨트롤, 인디케이터, 상수, 노드**
- **데이터 흐름**
- **기본 도움말**

2. 디버깅

에러가 있는 VI

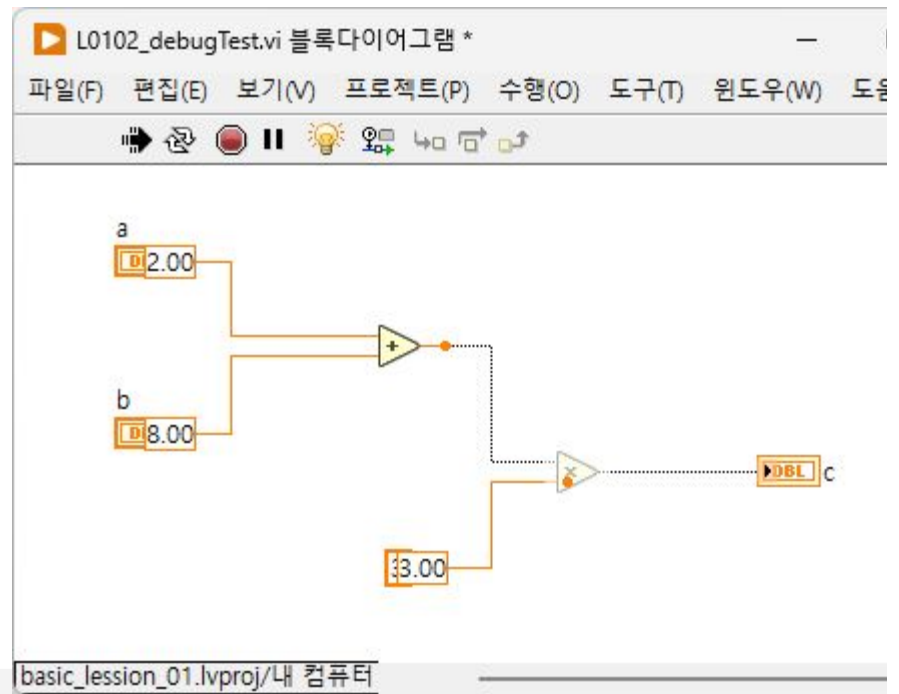
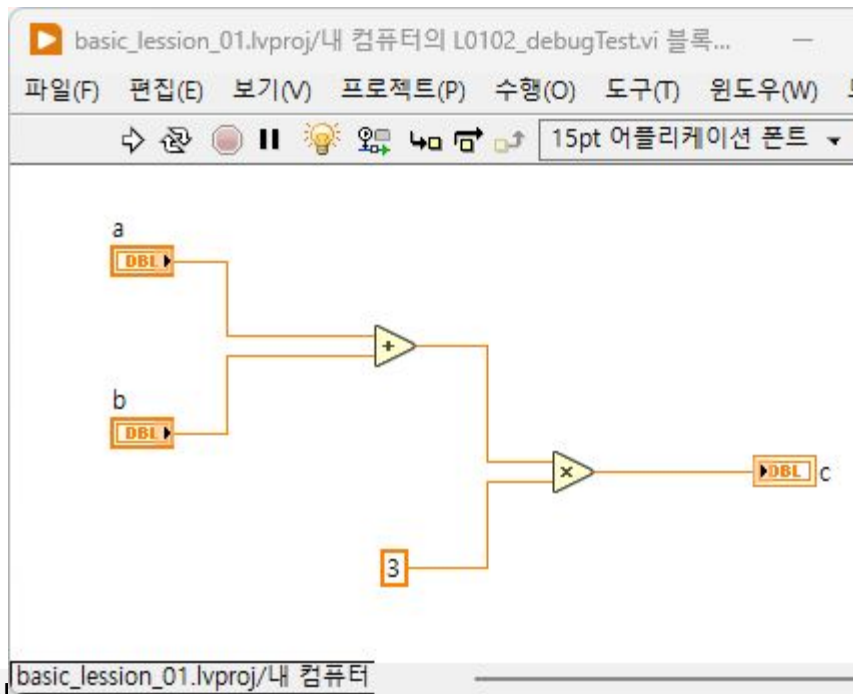
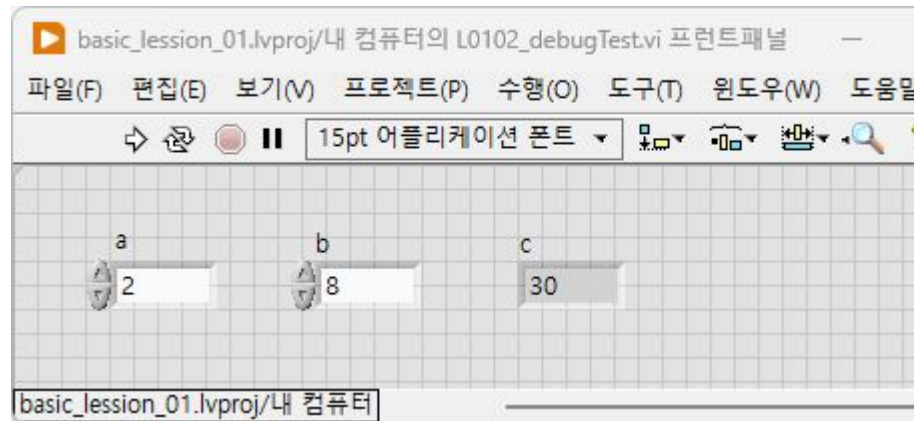


실행하이라이트

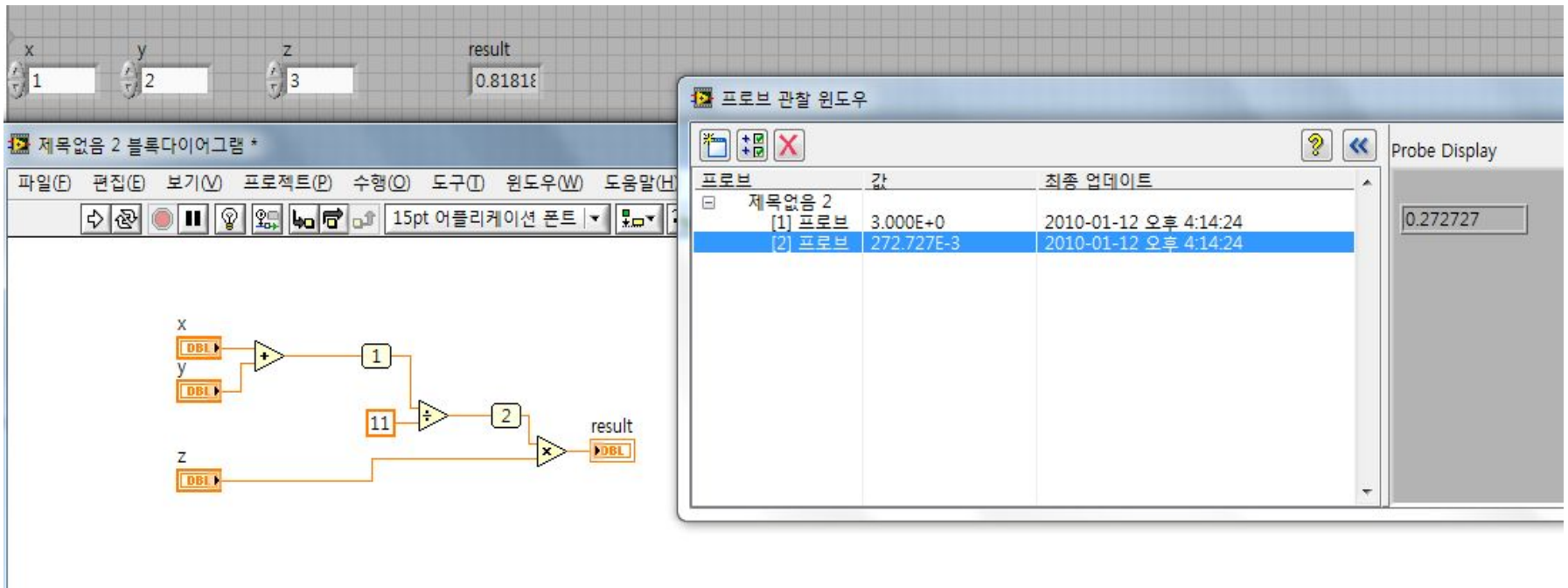


- 천천히 실행되면서
데이터의 흐름이 눈에
보임

실습2-2-1) 실행하이라이트



프로브



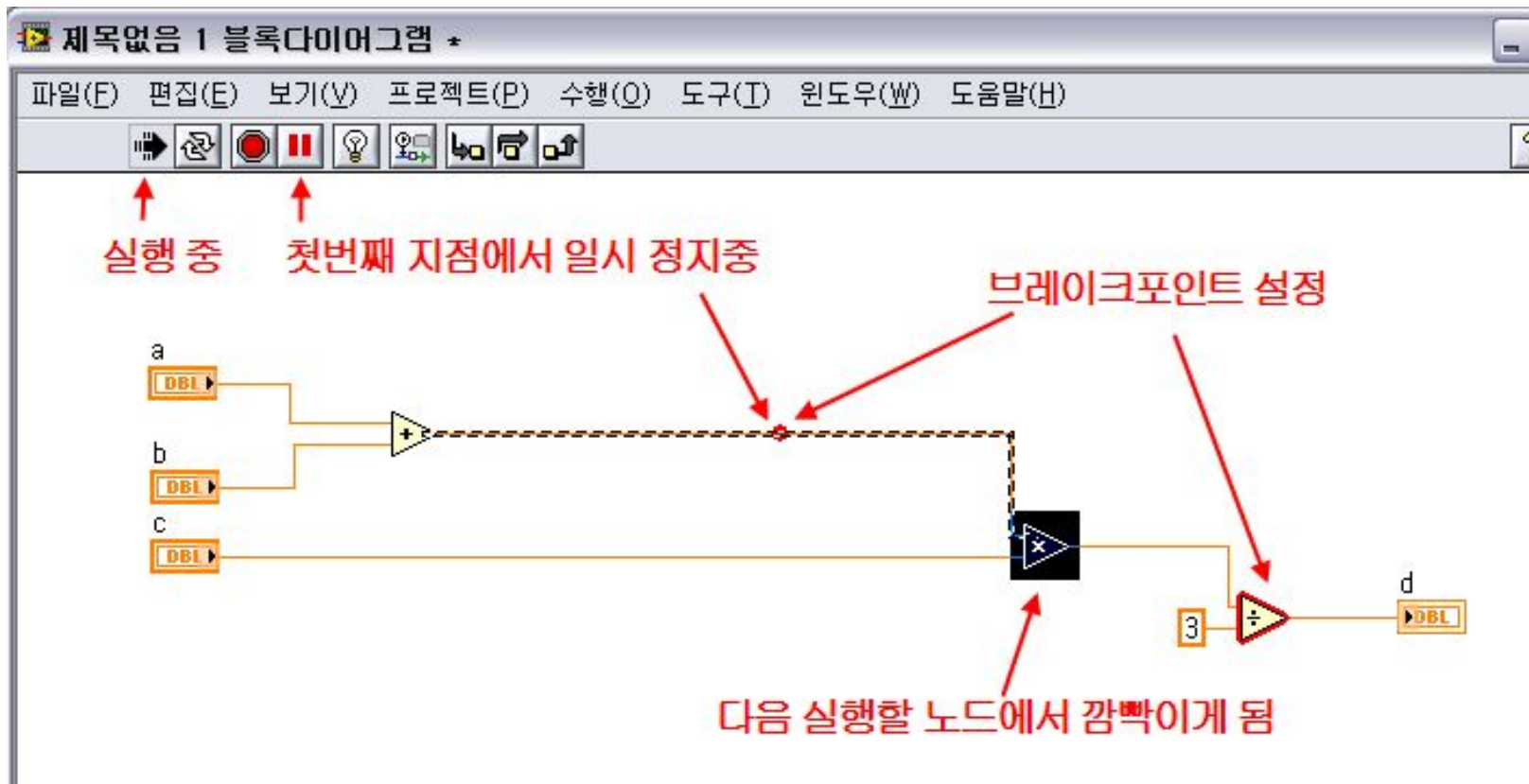
The image displays the LabVIEW software interface. On the left, a block diagram titled "제목없음 2 블록다이어그램 *" shows three input variables: x, y, and z. x and y are connected to an adder block (1), and the result is connected to a multiplier block (2) along with z. The final result is displayed in a numeric indicator labeled "result" with the value 0.81818. On the right, the "프로브 관찰 윈도우" (Probe Watch Window) is open, showing a table of probe data. The table has three columns: "프로브" (Probe), "값" (Value), and "최종 업데이트" (Last Update). The data shows two probes: "[1] 프로브" with a value of 3.000E+0 and "[2] 프로브" with a value of 272.727E-3. The "Probe Display" section on the right shows the value 0.272727.

프로브	값	최종 업데이트
[1] 프로브	3.000E+0	2010-01-12 오후 4:14:24
[2] 프로브	272.727E-3	2010-01-12 오후 4:14:24

- 원하는 지점의 값을 모니터링 함.

실습2-3-1) 프로브

브레이크포인트



- 원하는 지점에서 일시 정지 함.

실습2-4-1) 브레이크포인트

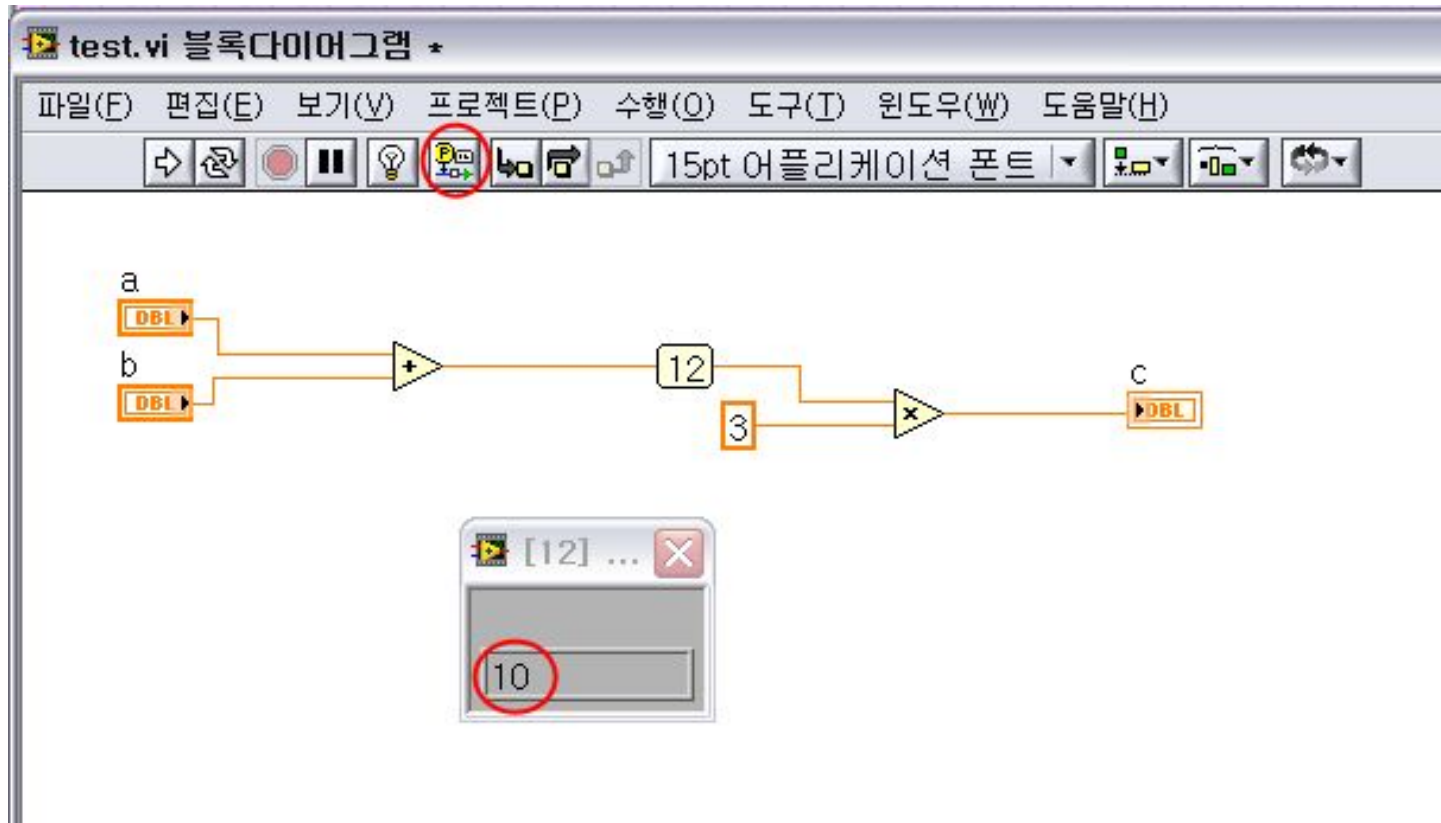
단계별 실행



- 노드 하나씩 차례로 실행됨.

실습2-5-1) 단계별 실행

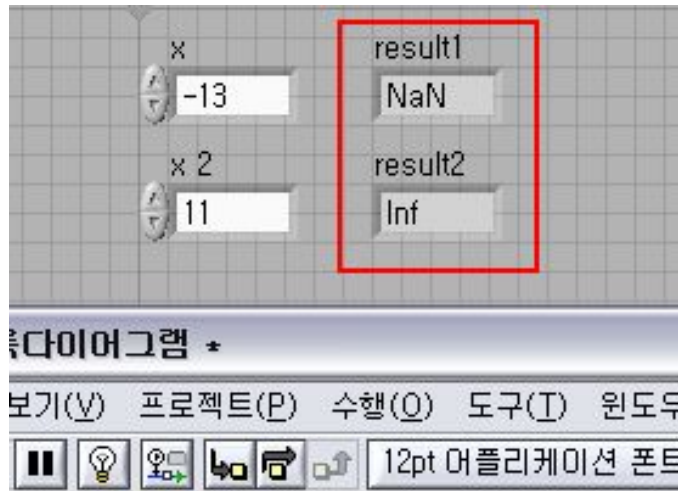
와이어 값 유지



- 실행이 끝난 다음 와이어에 값이 유지되어 있음.

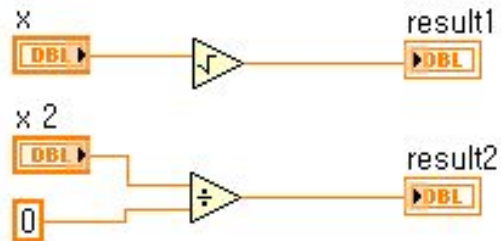
실습2-6-1) 와이어 값 유지

비정상적 데이터

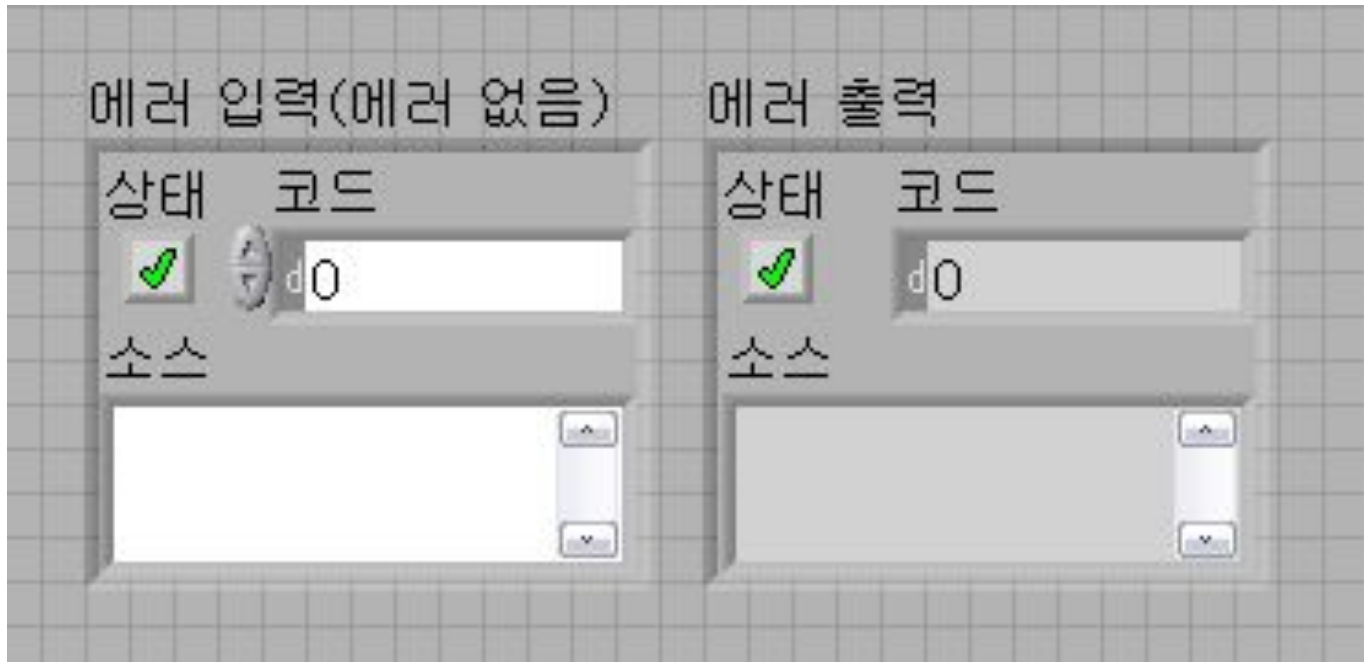


•**NaN** : 숫자가 아님

•**Inf** : 무한대

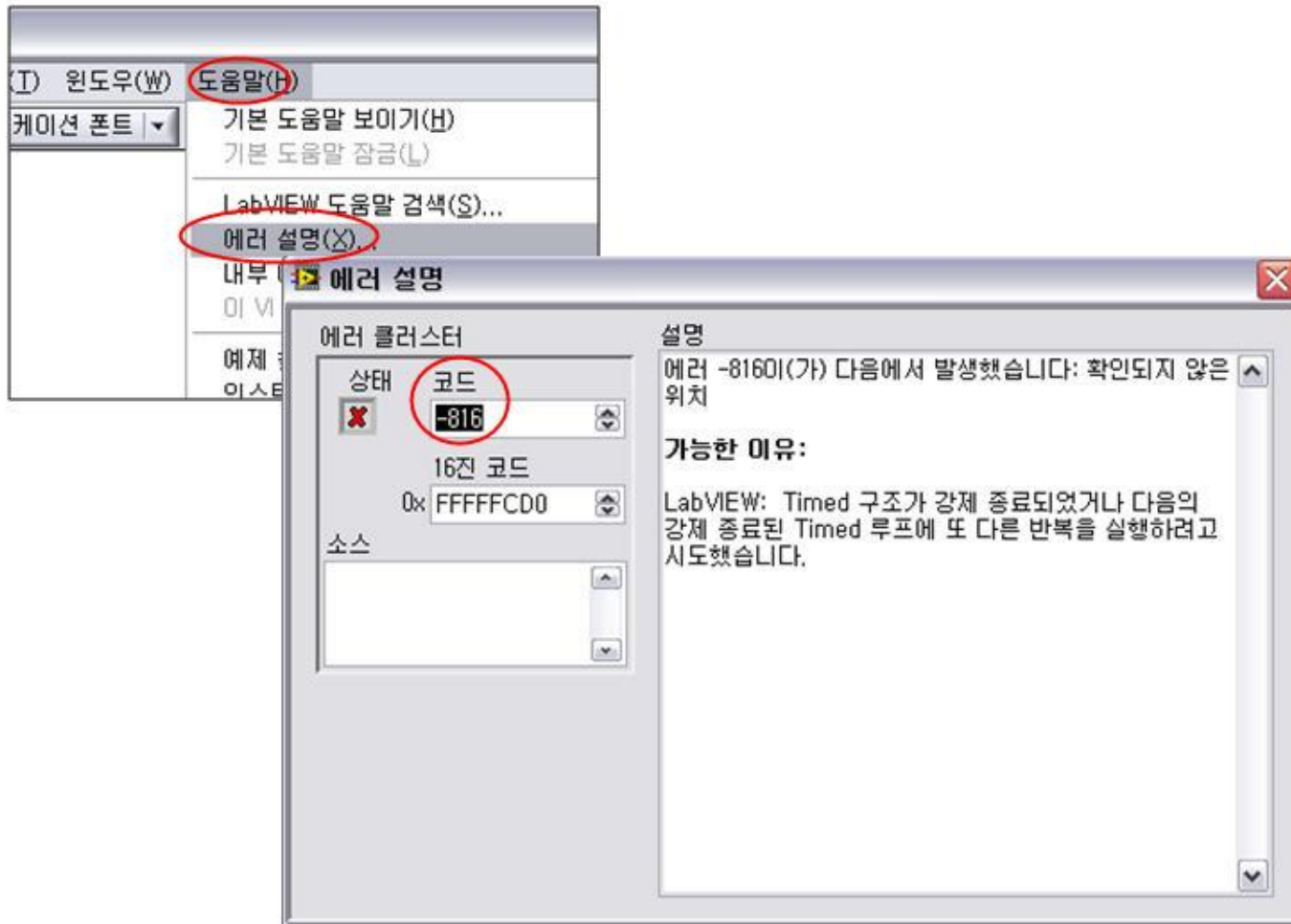


에러 클러스터



- 상태 : 에러 발생 유무
- 코드 : 에러 번호
- 소스 : 에러가 발생한 노드명

에러 검색

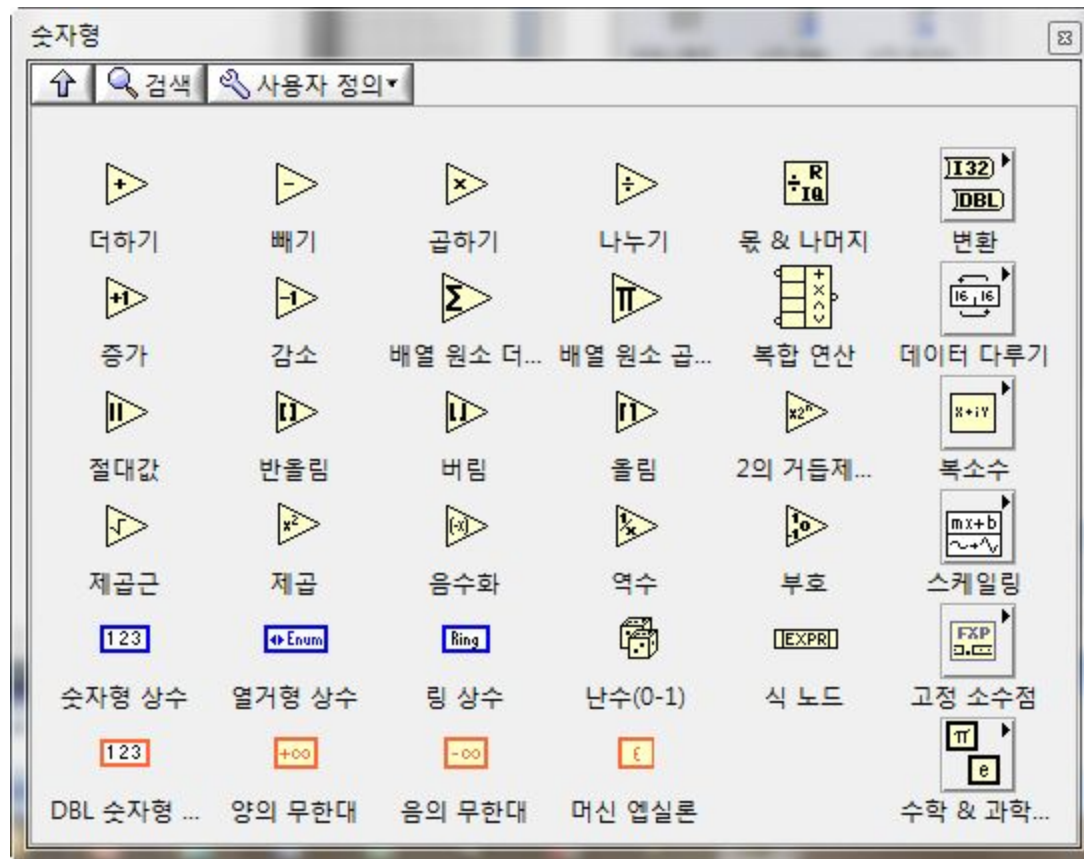


2장 요약

- 실행하이라이트
- 프로브
- 브레이크포인트
- 단계별 실행
- 와이어 값 유지
- **Inf, NaN**
- 에러 클러스터

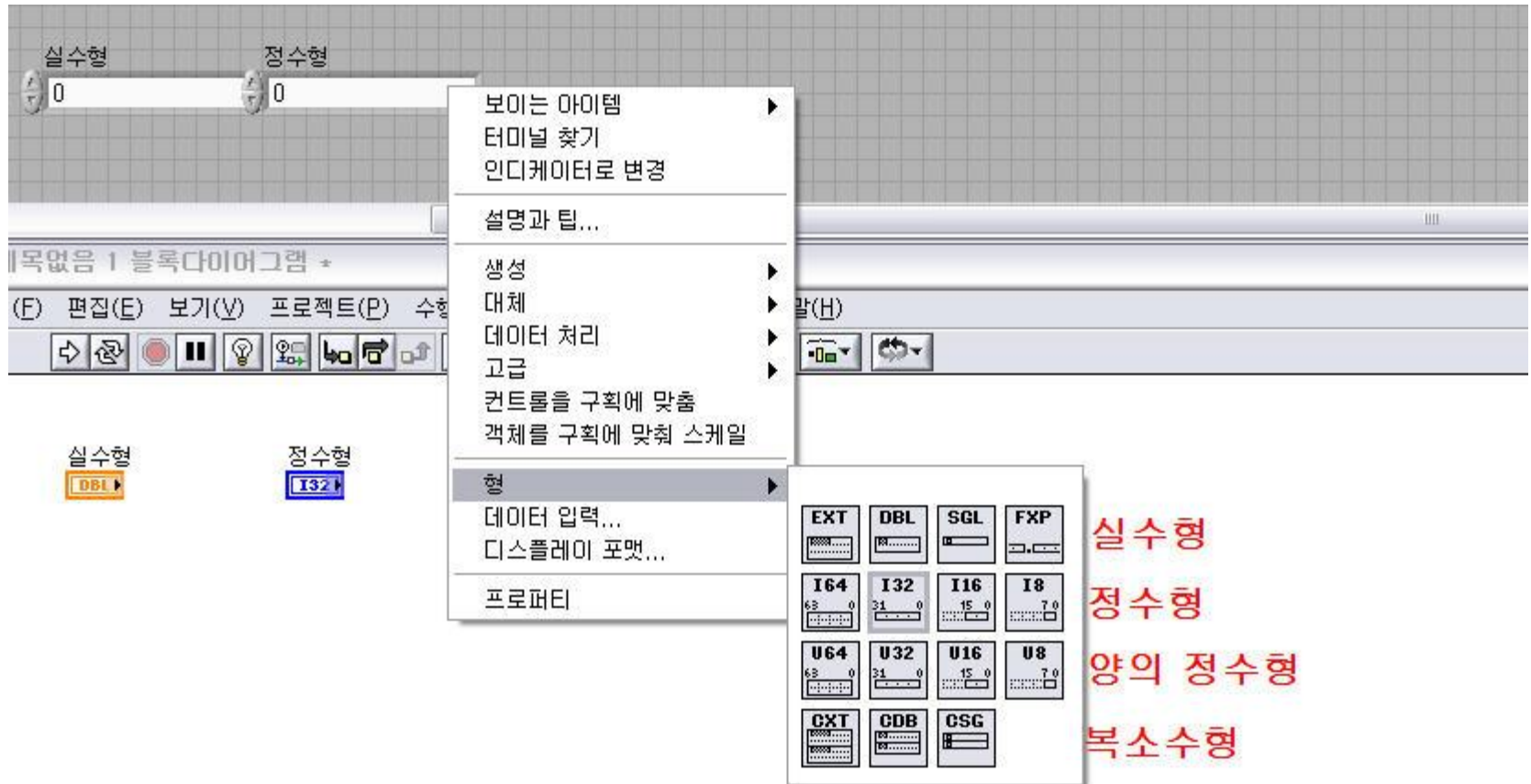
3. 데이터 타입

숫자형 (Numeric)



- 숫자형 - 숫자 데이터를 다룸.
- 주황색은 실수형, 파란색은 정수형을 나타냄

숫자형의 형 변경



실수형
정수형
양의 정수형
복소수형

숫자형 데이터 타입 - 정수

◆ 부호있는 정수

- 32-bit (I32): -2147483648 ~ 2147483647
- 16-bit (I16): -32768 ~ 32767
- 8-bit (I8): -128 ~ 127

◆ 부호없는 정수

- 32-bit (U32): 0 ~ 4,294,967,295
- 16-bit (U16): 0 ~ 65536
- 8-bit (U8): 0 ~ 256

숫자형 데이터 타입 - 실수형

숫자

◆ 실수형:

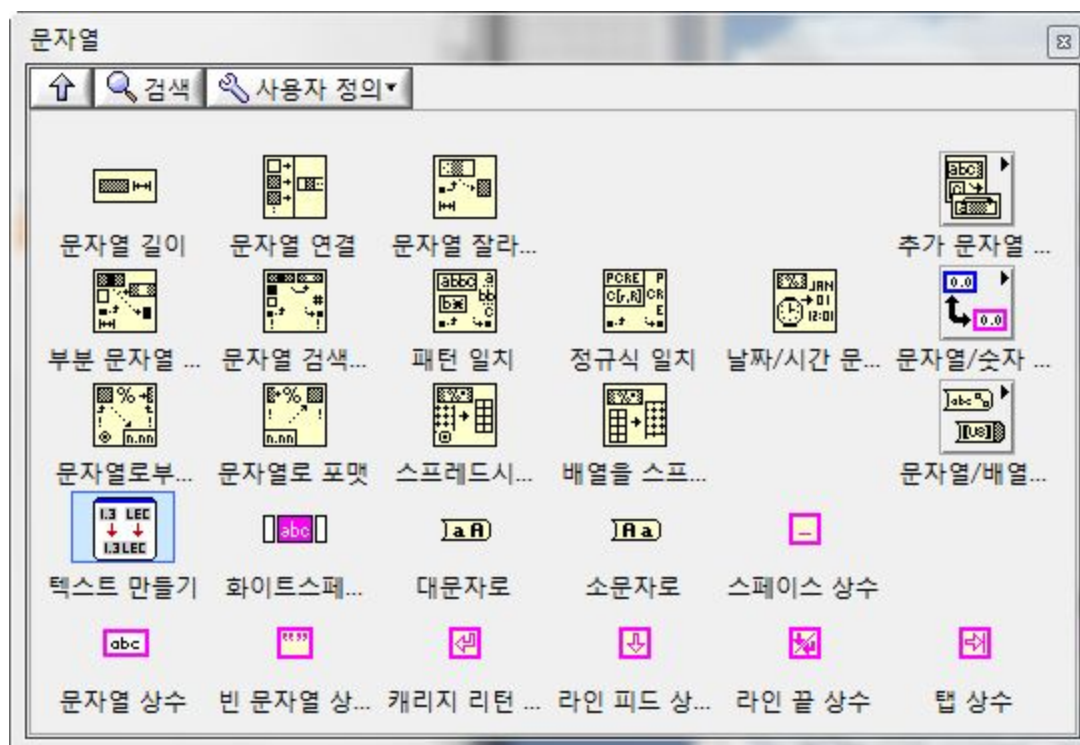
- 확장형 정밀도 [EXT]: $-1.19e+4932 \sim 1.19e+4932$
- 배정도 [DBL]: $-1.79e+308 \sim 1.79e+308$
- 단정도 [SGL]: $-3.40e+38 \sim 3.40e+38$

◆ 복소수 실수형:

- 복소수 실수형 데이터 타입은 실수형 데이터 타입과 같은 정밀도를 가집니다. 유일한 차이점은 복소수 실수형 데이터 타입은 실수와 허수부를 가진다는 것입니다.

실습3-1-1) 숫자형 사용법

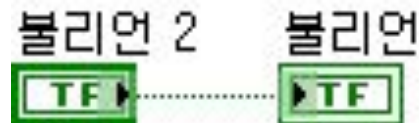
문자열 (String)



- 문자열 - **ASCII** 코드로 나타낸 문자를 표시하기 위한 데이터 타입. 분홍색을 나타냄.
- 각종 통신(시리얼, **GPIB**, **TCP/IP** 등)에서 사용되는 데이터 타입.

실습3-2-1) 문자열 사용법

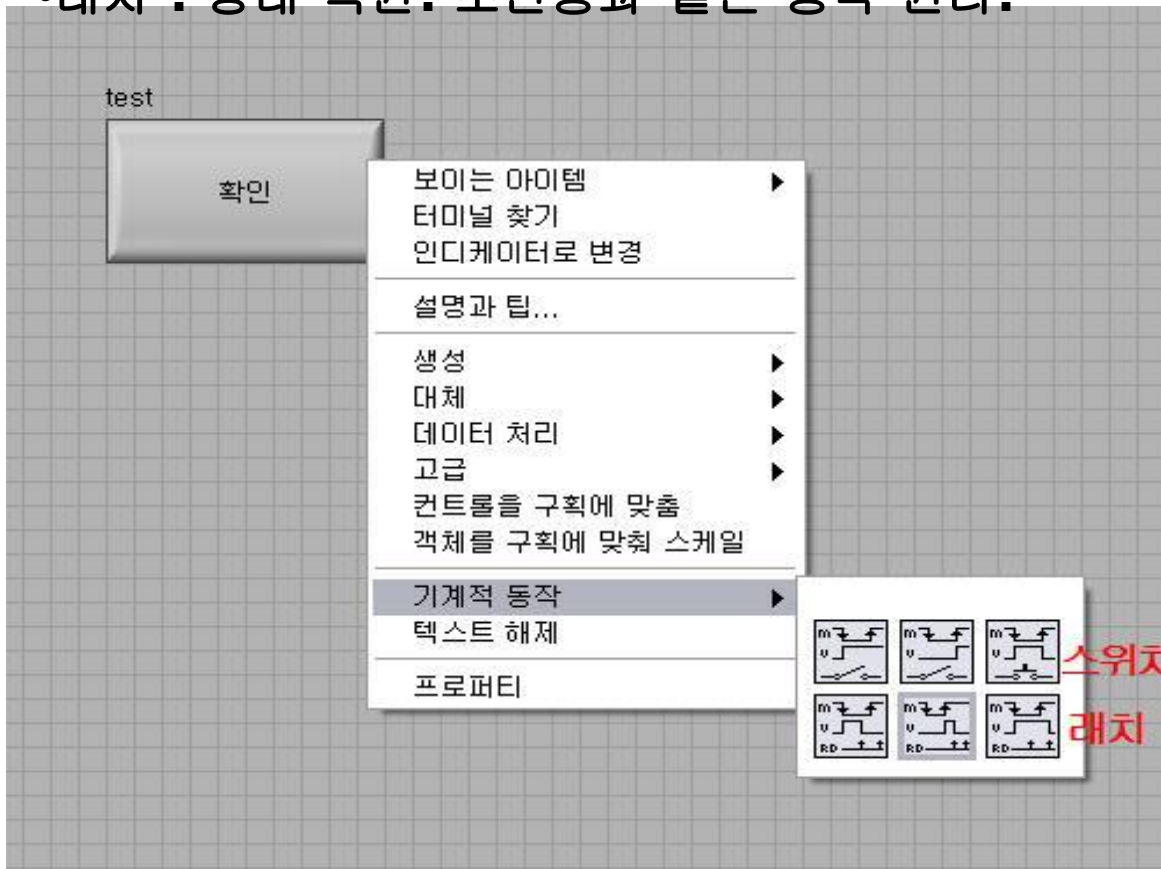
불리언 (Boolean)



- 불리언 - 참, 거짓 데이터를 표현되는 논리 연산을 위한 데이터, 초록색을 나타냄

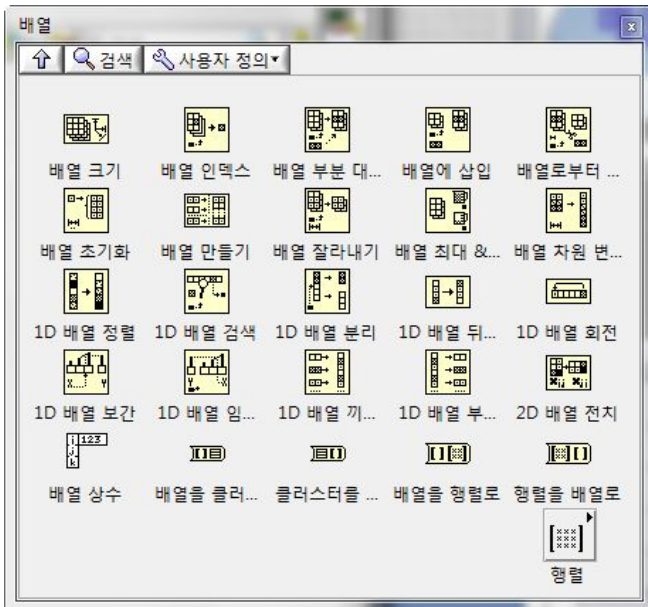
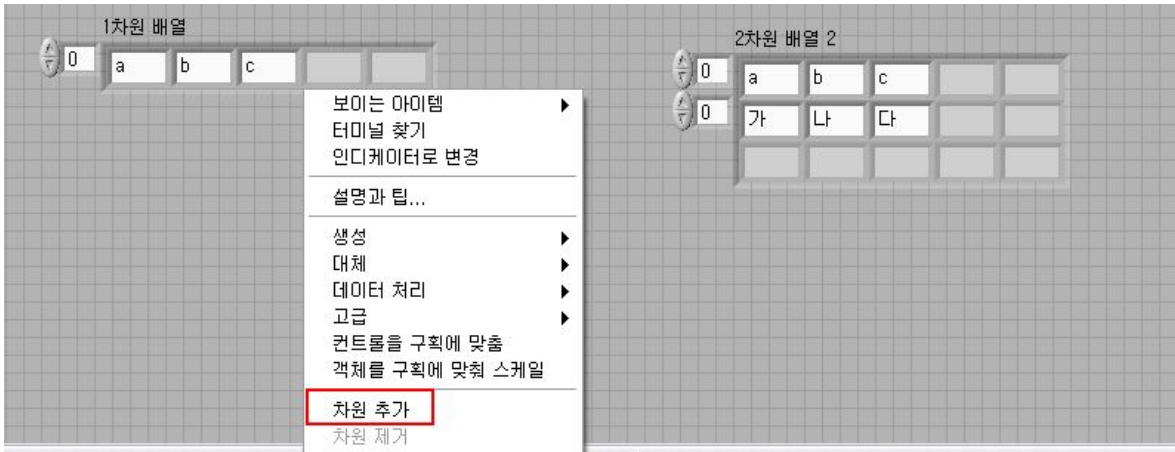
불리언의 기계적 동작

- 스위치 : 상태 유지.전등불의 전원이 같은 동작 원리
- 래치 : 상태 복원. 초인종과 같은 동작 원리.



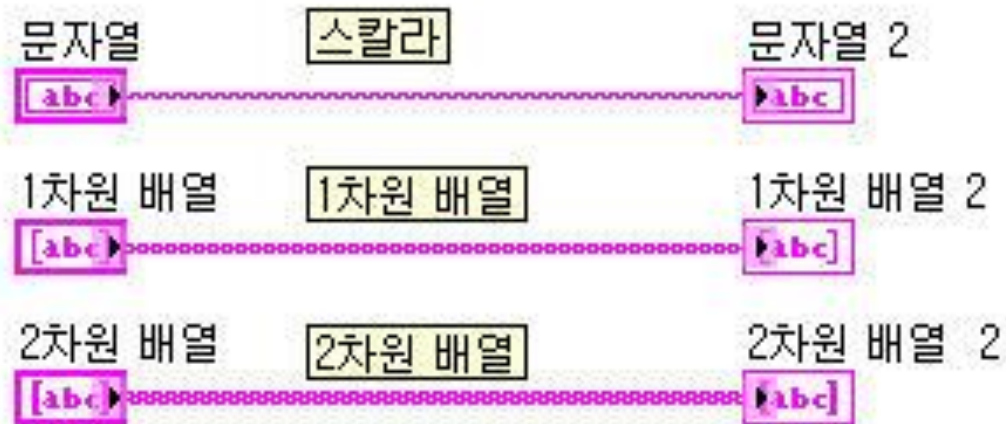
실습3-3-1) 불리언 사용법
실습3-3-2) 복합연산 사용법

배열 (Array)



- 배열 : 같은 데이터 타입의 묶음
- 원소 : 배열 구성원
- 인덱스 : 원소의 주소를 의미
- 차원

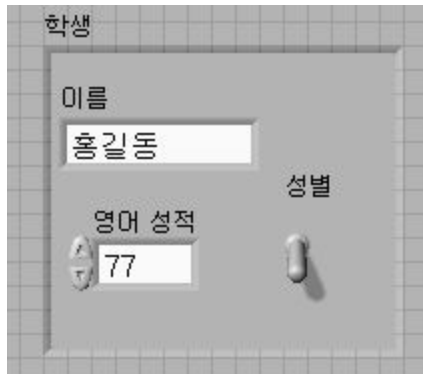
배열 와이어



- 와이어의 굵기가 배열의 차원에 따라 달라짐.

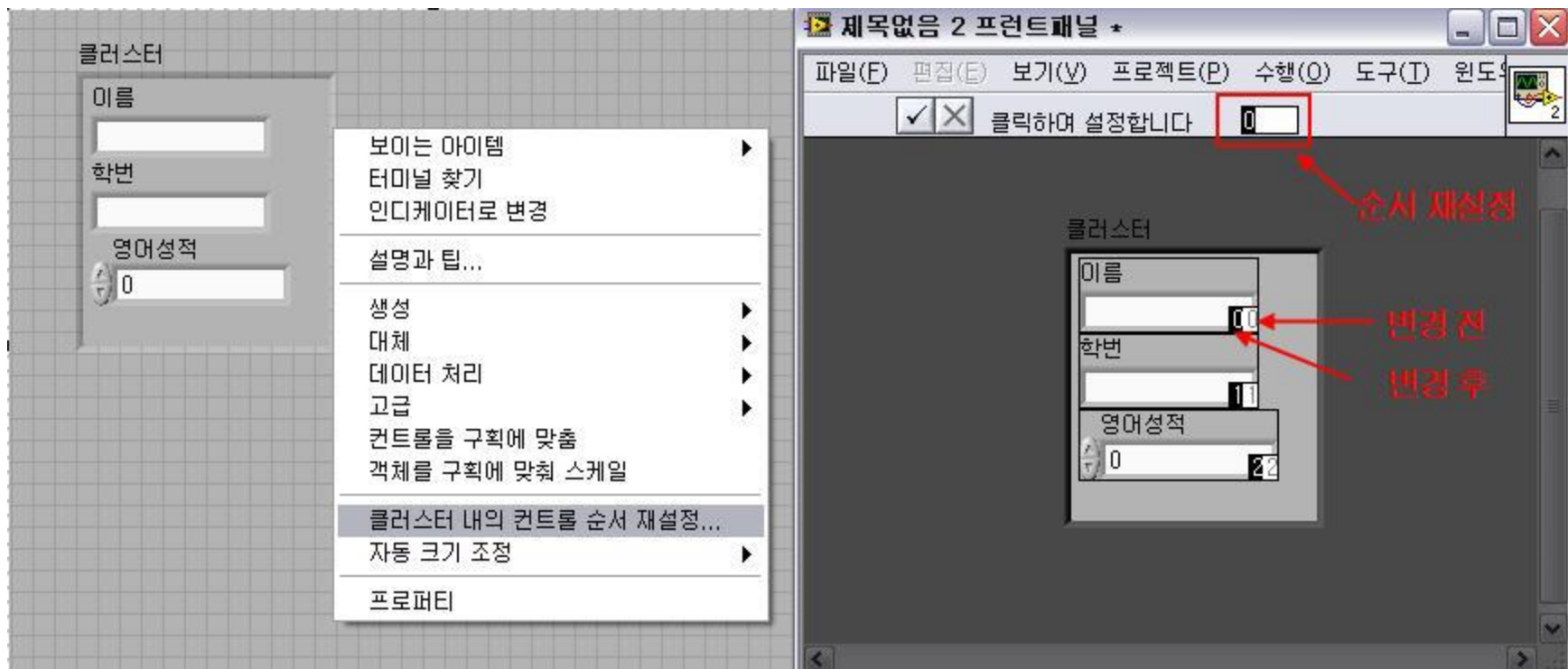
실습3-4-1) 배열 생성 및 노드 사용법
실습3-4-2) 배열 생성 및 노드 사용법
실습3-4-3) 배열 만들기 노드 사용법

클러스터 (Cluster)



- 클러스터 : 다양한 데이터 타입의 묶음

클러스터 내의 순서 설정



클러스터 노드



- 묶기 : 클러스터 생성할 때 사용
- 이름으로 묶기 : 클러스터 업데이트할 때 사용
- 풀기 : 클러스터 원소 전체 풀어낼 때 사용
- 이름으로 풀기 : 클러스터 원소 선택적으로 풀어낼 때 사용

실습3-5-1)

클러스터 생성 및 노드 사용법

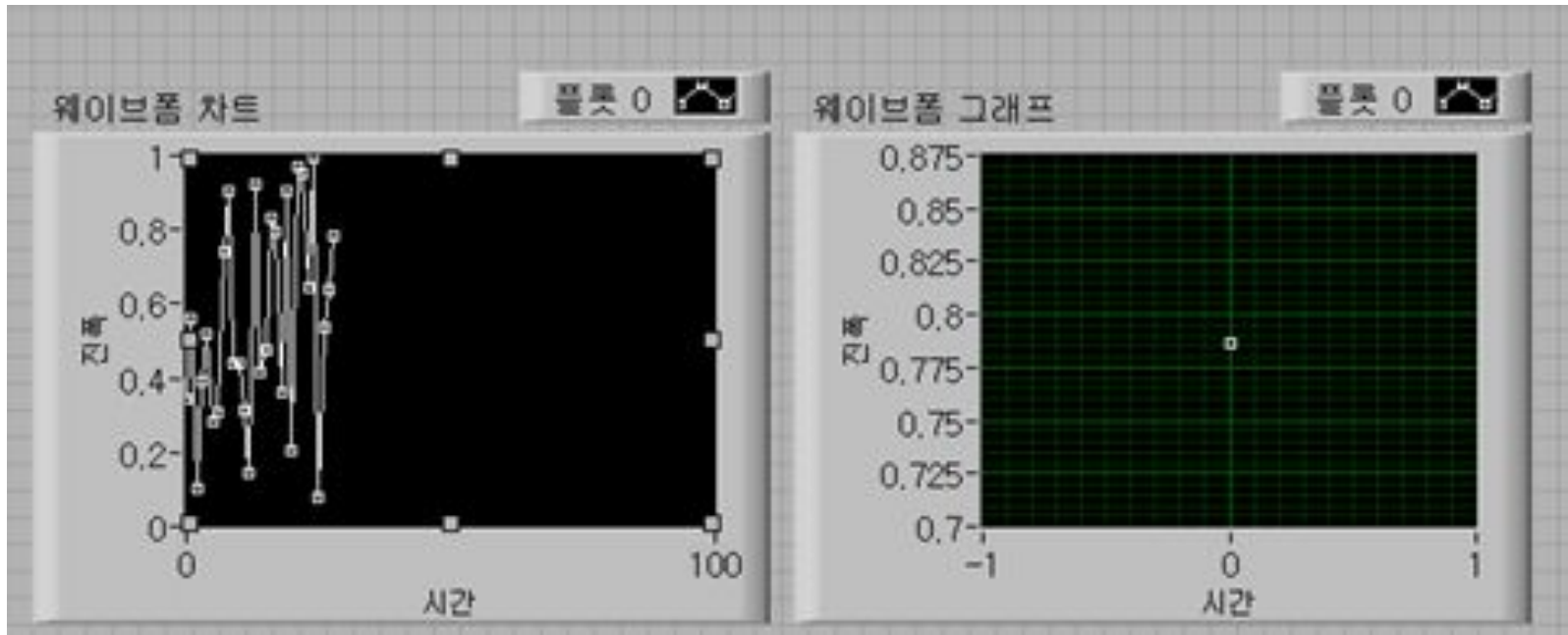
실습3-5-2) 클러스터와 배열

웨이브폼 데이터



- **t0** : 데이터 획득 시작 시간
- **dt** : 데이터 획득 간격
- **Y[]** : 획득한 데이터

웨이브폼 차트/웨이브폼 그래프



- 웨이브폼 차트 : 데이터값 누적
- 웨이브폼 그래프 : 현재 데이터만 디스플레이

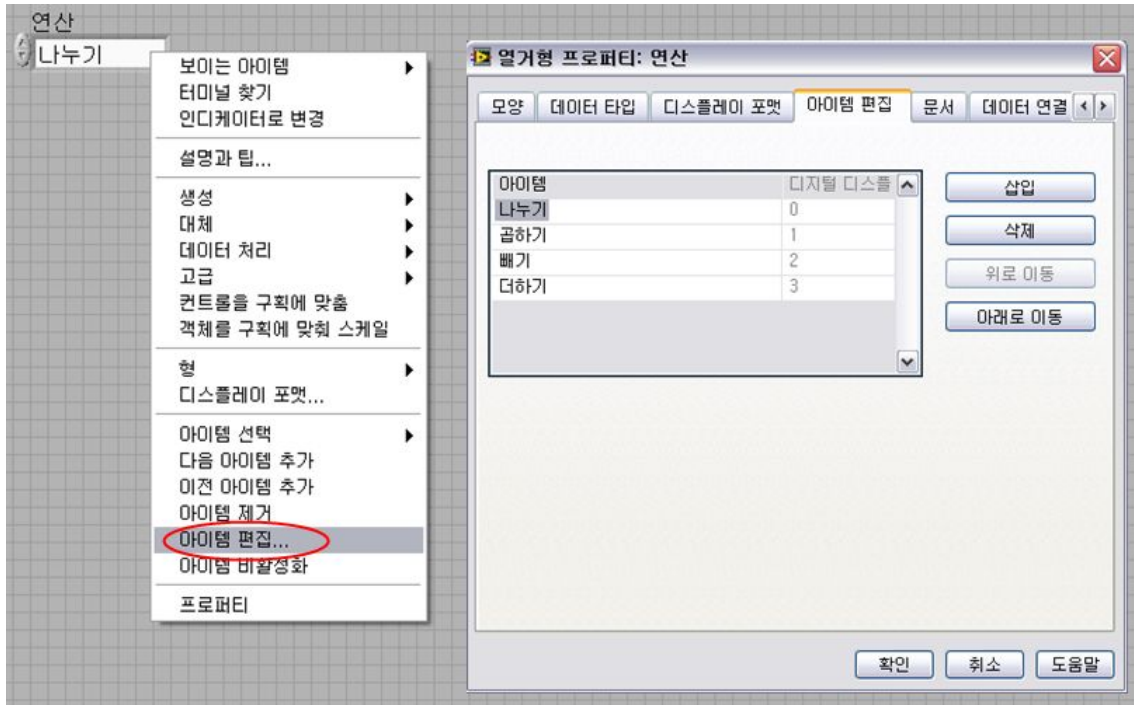
실습3-6-1) 웨이브폼 차트와 그래프

다이나믹 데이터 타입



- 다이나믹 데이터 : 익스프레스 전용 데이터 타입

열거형 (Enum)



- 정의된 아이템 중 하나를 선택해서 사용
- 4장에서 배울 케이스 구조와 함께 많이 사용

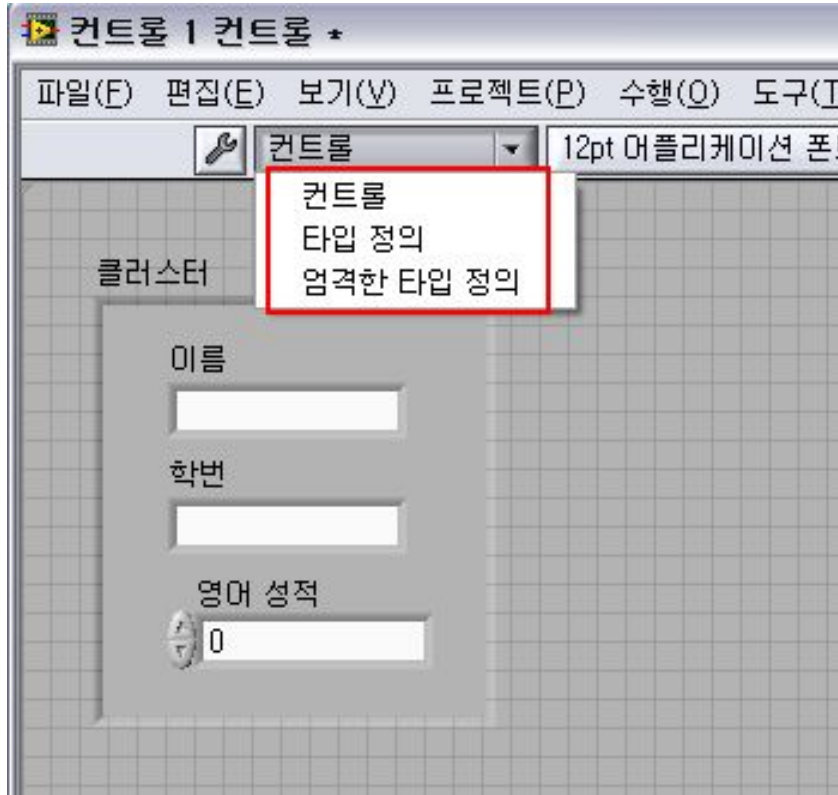
실습3-8-1) 열거형

배리언트 (Variant)



- 프로그램의 유연성을 제공함. 프로그램 유지 관리 시 용이함.
- 단 **RT os**에서는 사용할 수 없음.

타입 정의 (Type Definition)



•사용자가 정의하는 컨트롤로써
여러 프로젝트에서 손쉽게 가져다가
재사용 가능함.

- ✓컨트롤 - 단순한 사용자 정의
- ✓타입 정의 - 파일을 변경하면 사용하는
모든 컨트롤들의 프로퍼티가 함께
변경됨.
크기 등 제외
- ✓엄격한 타입 정의 - 크기를 비롯해 모든
컨트롤들의 프로퍼티가 함께 변경됨.
이름, 캡션, 초기값 등 제외

*터미널에 타입 정의된 의미로 검정색 삼각형 표시가
됨.

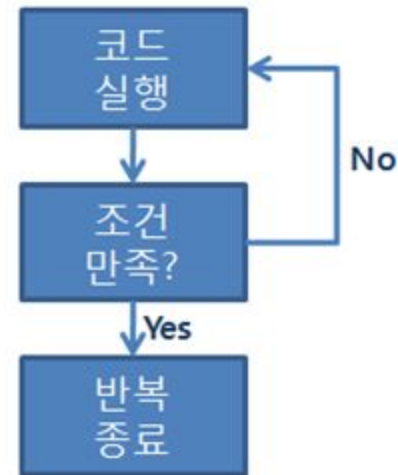
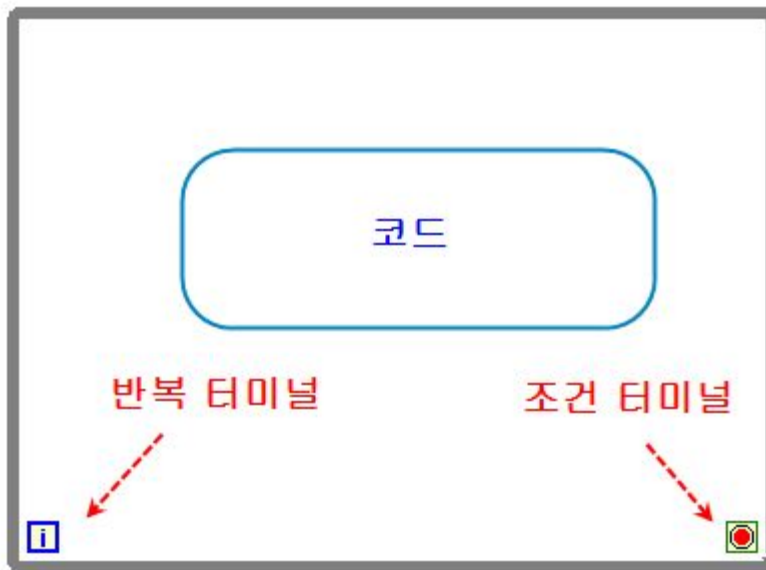


3장 요약

- 숫자형, 문자열, 불리언
- 배열 - 같은 데이터 타입의 묶음
- 클러스터 - 다양한 데이터 타입의 묶음
- 웨이브폼 차트, 웨이브폼 그래프
- 다이내믹 데이터 타입
- 열거형
- 배리언트
- 타입 정의

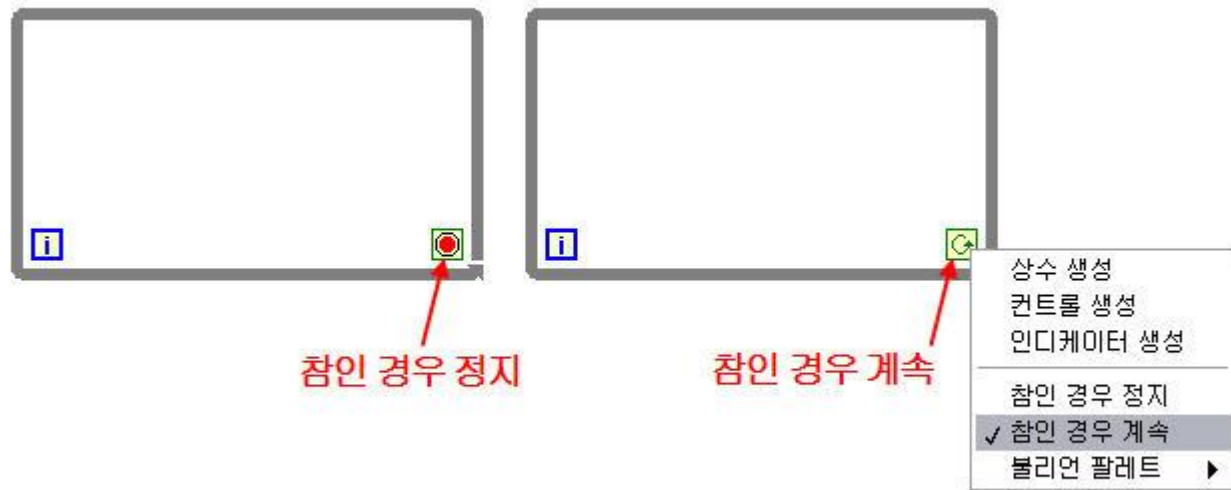
4. 구조

While 루프



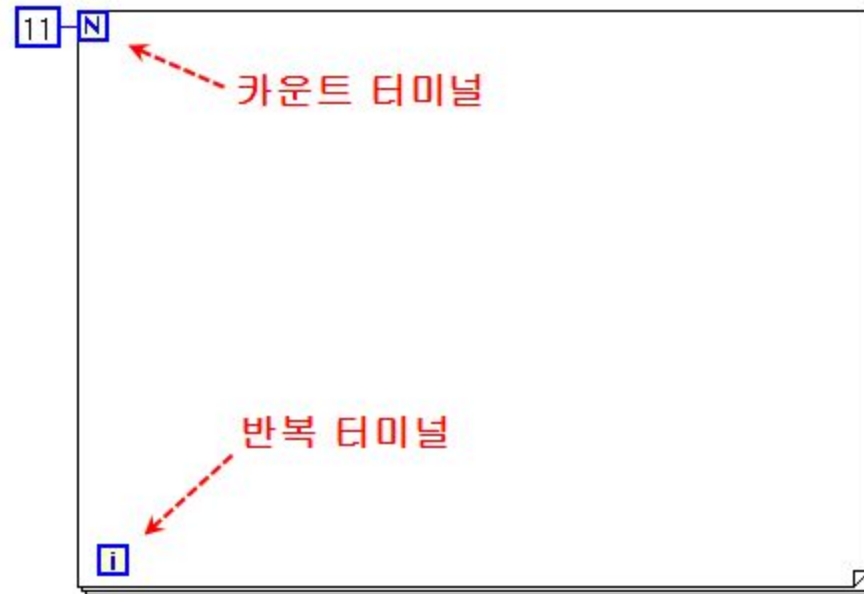
• **While** 루프 : 반복을 위한 구조

조건 터미널



실습4-1-1) **WHILE** 루프

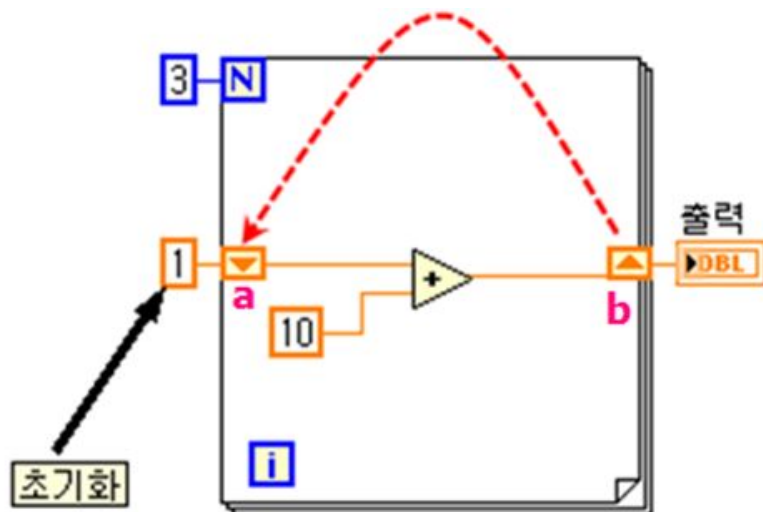
For 루프



- **For** 루프 : 반복 횟수를 지정하여 사용. 카운트 터미널에 반복횟수를 지정함.

실습4-2-1) **FOR** 루프

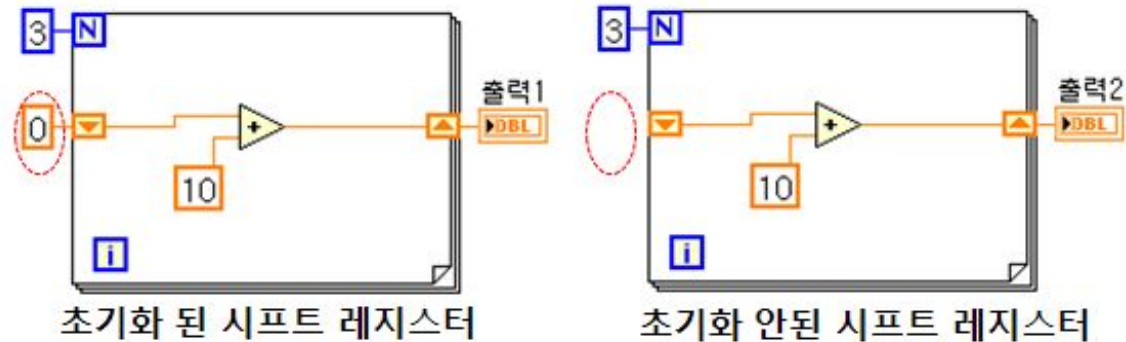
시프트 레지스터



•시프트 레지스터 : 과거값을 저장

반복 횟수	지점 a 의 값	지점 b 의 값
1st	1	11
2nd	11	21
3rd	21	31

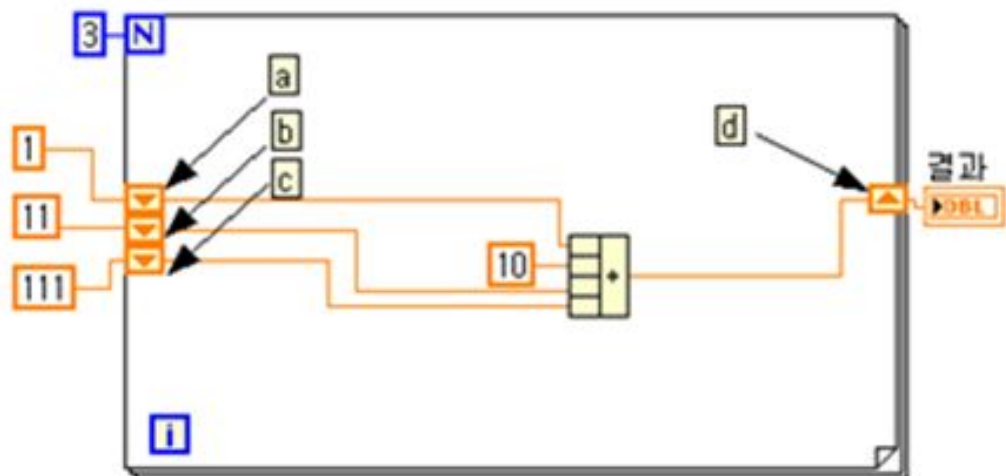
시프트 레지스터 초기화 유무



실행 횟수	초기화	초기화 안 함
1 st 실행	출력1 : 30	출력2 : 30
2 nd 실행	출력1 : 30	출력2 : 60
3 rd 실행	출력1 : 30	출력2 : 90

- 시프트 레지스터의 초기화 유무에 따라 결과값이 달라짐.

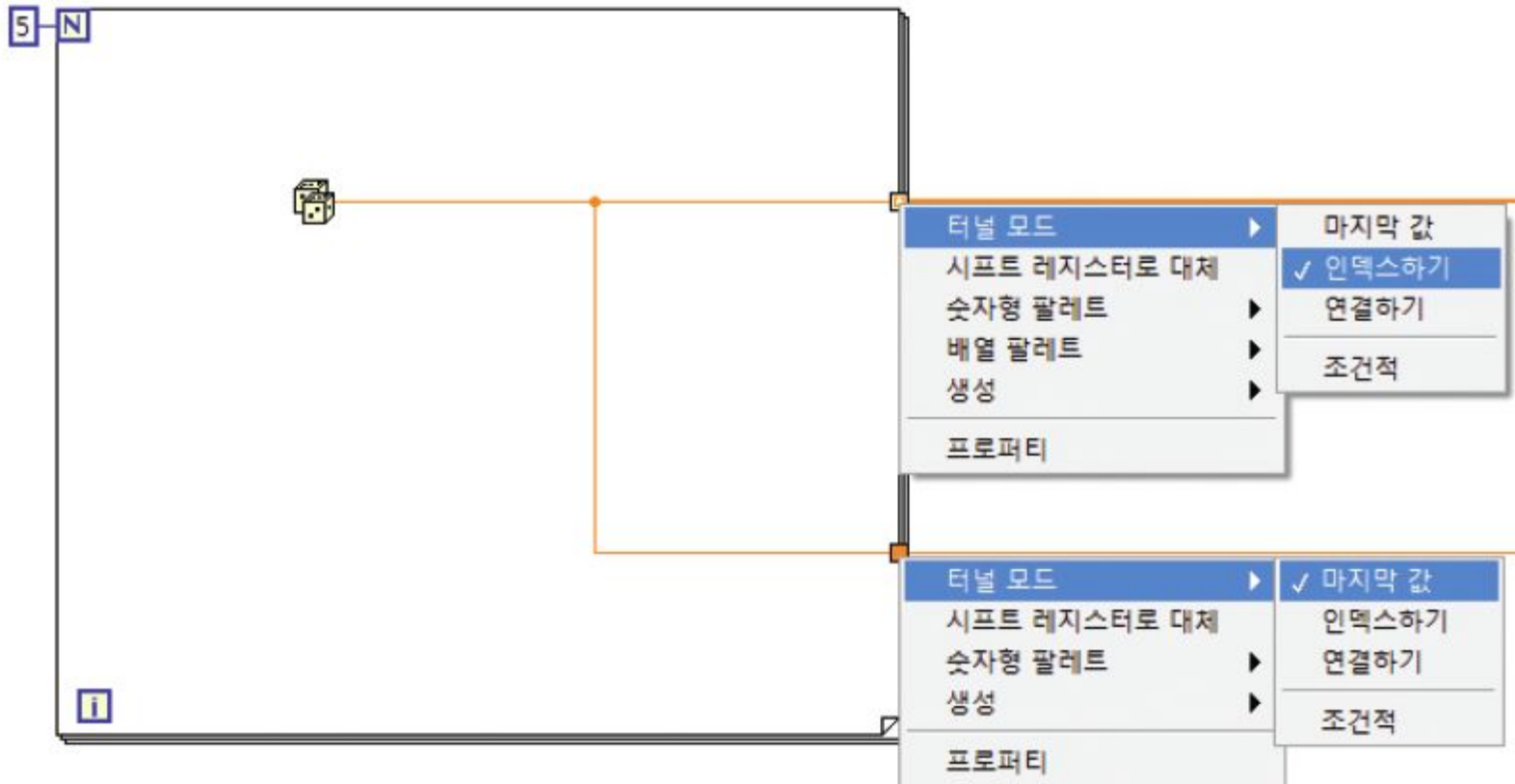
다층 레지스터



반복 횟수	지점 a 의 값	지점 b 의 값	지점 c 의 값	지점 d 의 값
1st	1	11	111	133
2nd	133	1	11	155
3rd	155	133	1	299

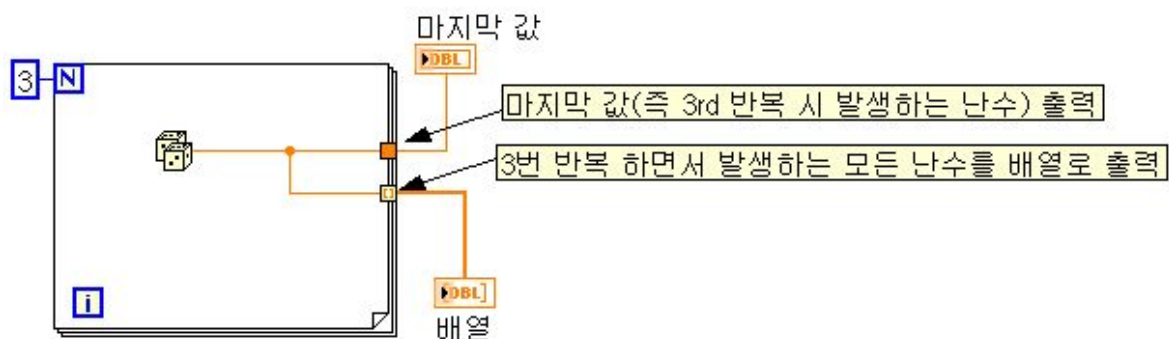
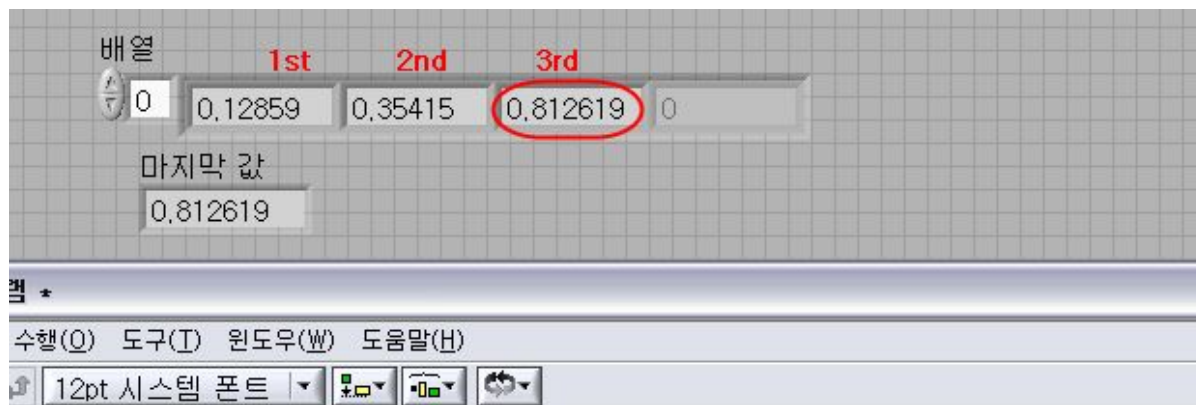
실습4-3-1) 시프트 레지스터

인덱싱 활성화/비활성화



- 인덱싱 활성화 / 비활성화 유무에 따라 출력이 달라짐.

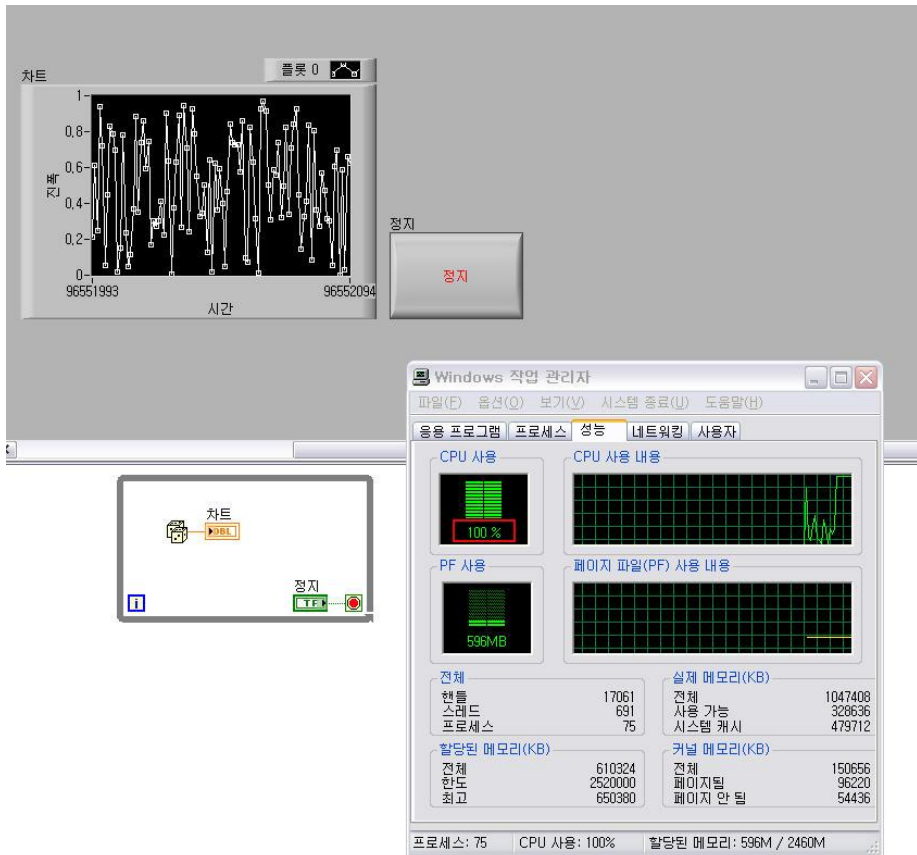
인덱싱 활성화/비활성화 출력



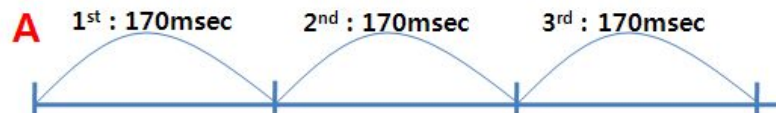
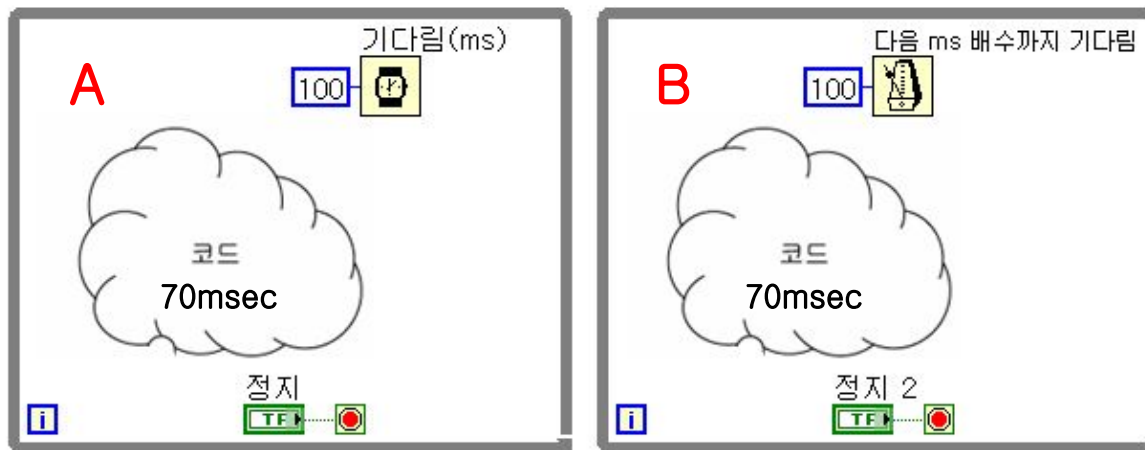
실습4-4-1) 인덱싱 활성화/비활성화

타이밍 노드 필요성

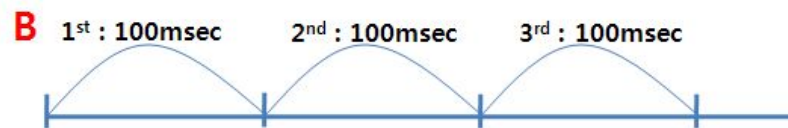
- 반복 구조에 타이밍 노드가 없으면 **CPU** 점유율이 최대로 상승함.



타이밍 노드



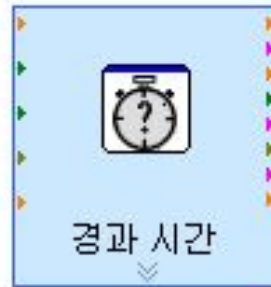
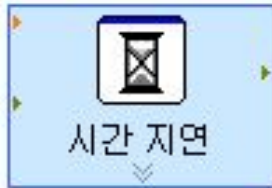
•기다림 : 절대적 기다림.



•다음ms배수까지 기다림 : 상대적 기다림.

*타이밍 노드와 나머지 노드들간의 순서를 결정해줘야지만 위 그림과 같이 정확하게 동작함.

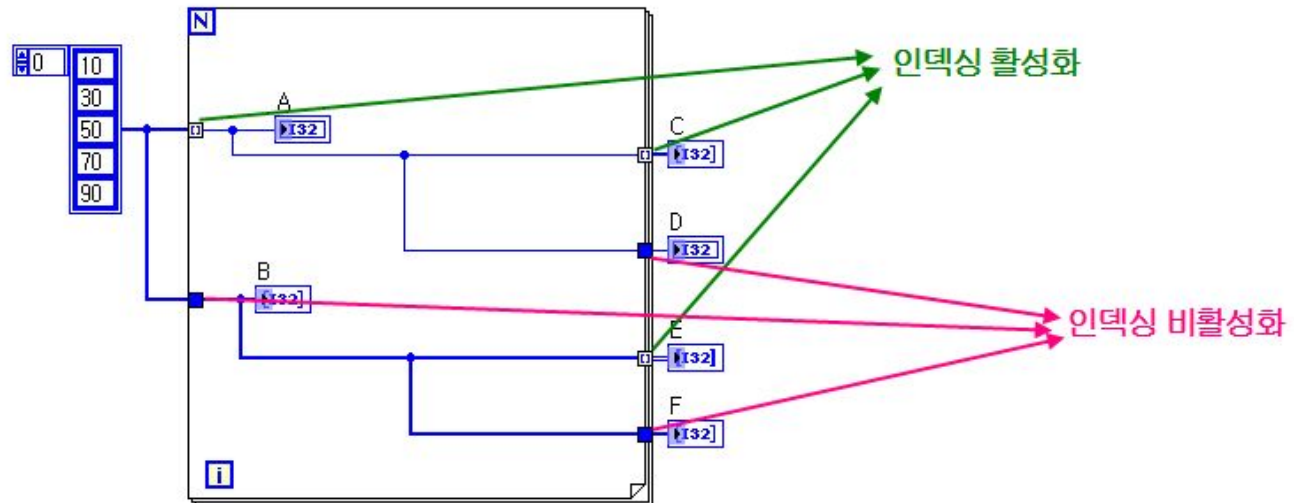
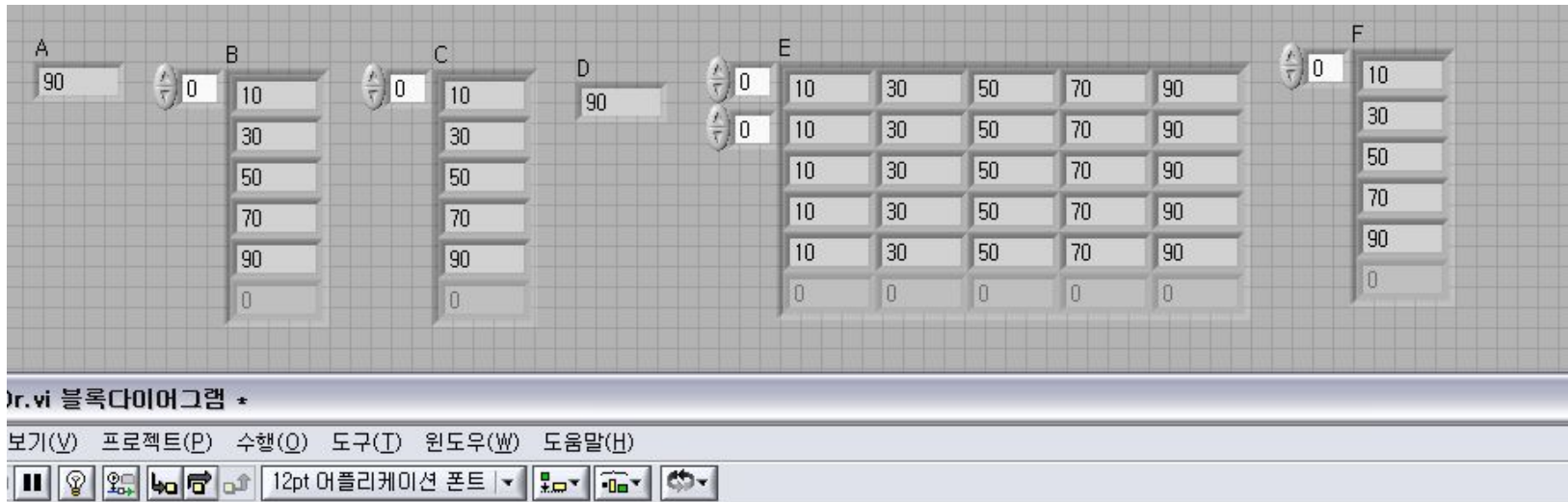
다양한 타이밍 노드들



- 시간 지연 : 기다림 노드와 기능이
같음.
- 경과 시간 : 설정한 시간을 알려줌.
- 틱 카운트 : 초시계와 같은 기능.

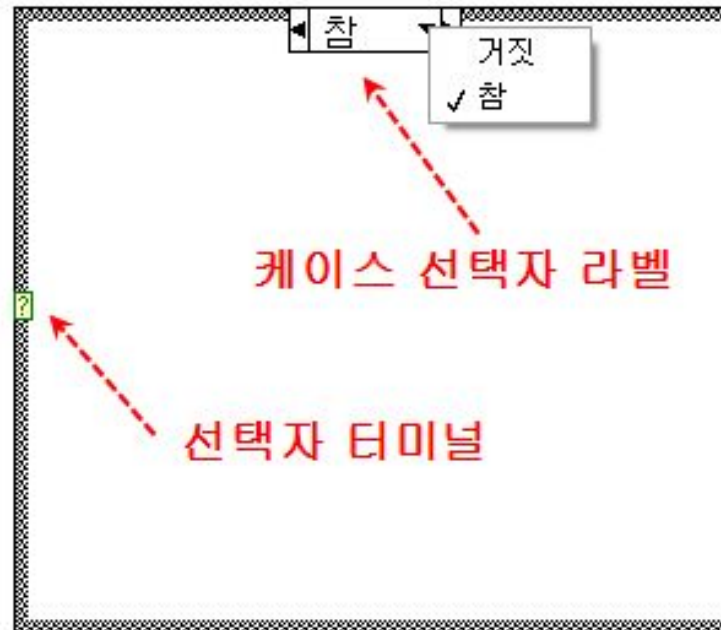
실습4-5-1) 경과 시간 노드

배열과 For루프



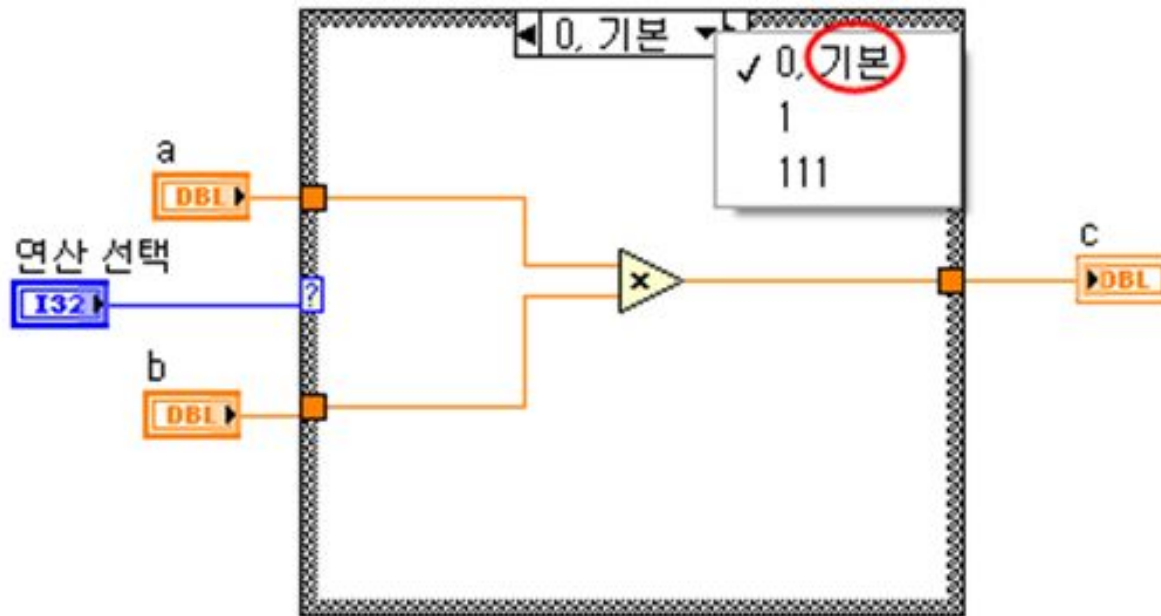
실습4-6-1) 배열과 FOR 루프
실습4-6-2) 배열과 FOR 루프

케이스 구조



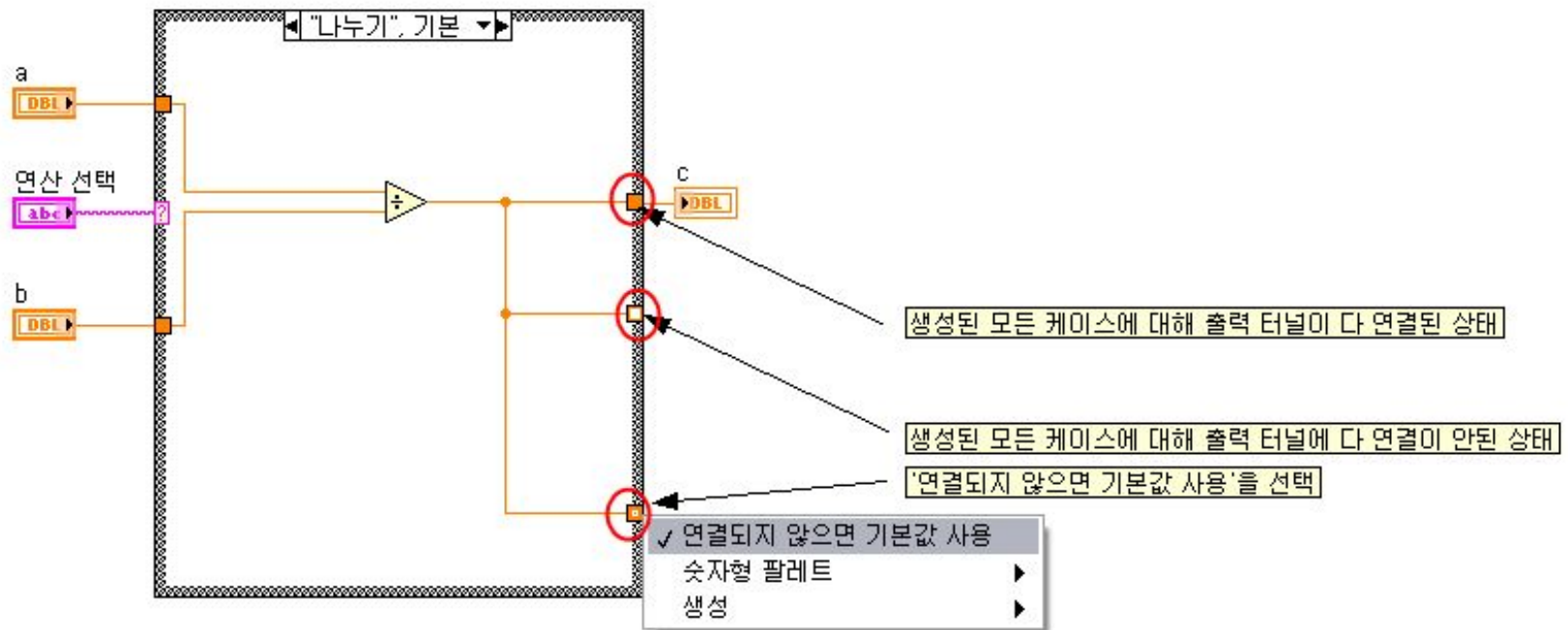
- 케이스 구조 : 선택자 터미널 입력에 따라 실행되는 코드가 달라짐.

기본 케이스



- 기본 케이스 : 예외 입력 때 실행되는 코드.

케이스 구조의 출력 터널



•케이스 출력 터널은 항상 막혀 있어야 함.

실습

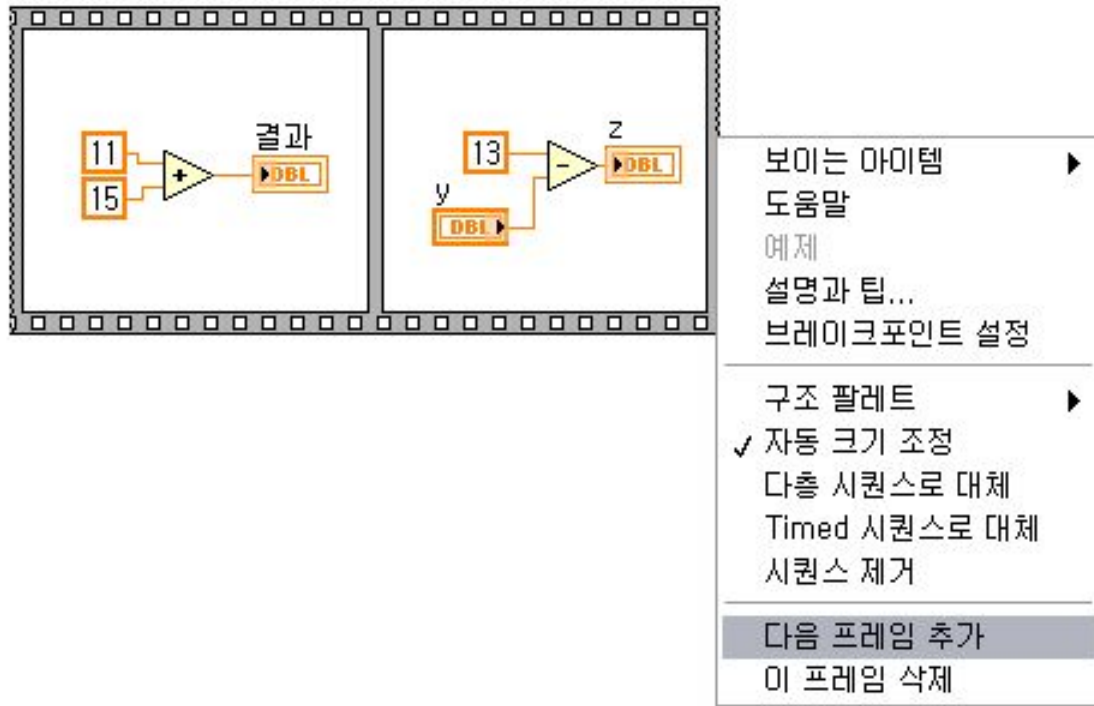
while 구조안에 case 구조를 넣는다.

사칙연산을 수행한다.

[중지] 버튼을 누르면 종료한다.

실습4-7-1) 케이스 구조 사용법

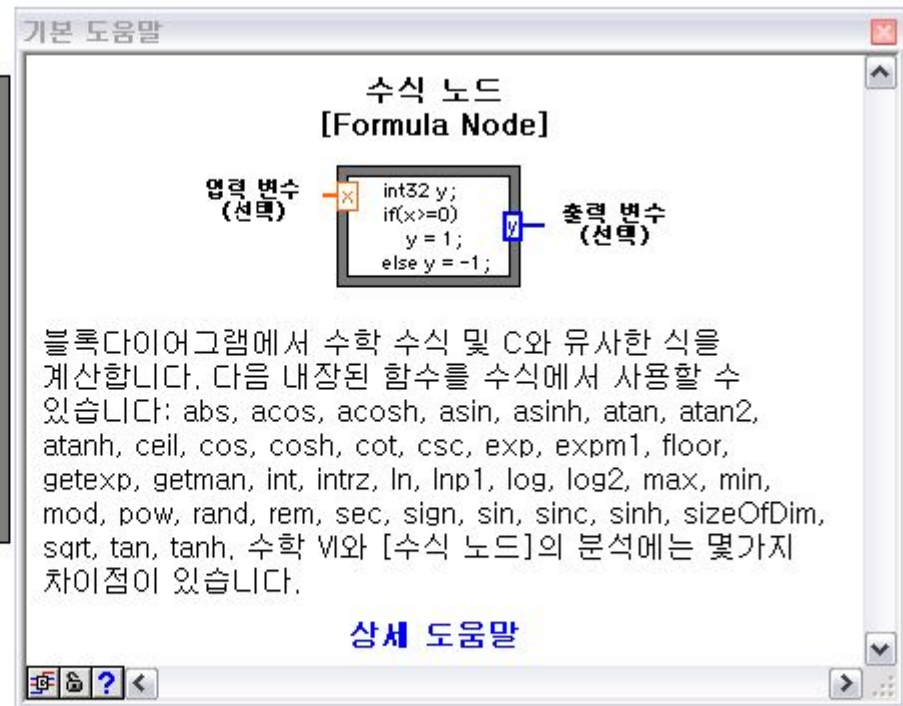
시퀀스 구조



- 시퀀스 구조 : 강제적으로 실행 순서를 결정함.

실습4-8-1) 시퀀스 구조 사용법

수식 노드



- 수식 노드 : 복잡한 수식을 C 프로그래밍과 비슷한 기법으로 구현.

실습4-9-1) 수식 노드 사용법

이벤트 구조 종류

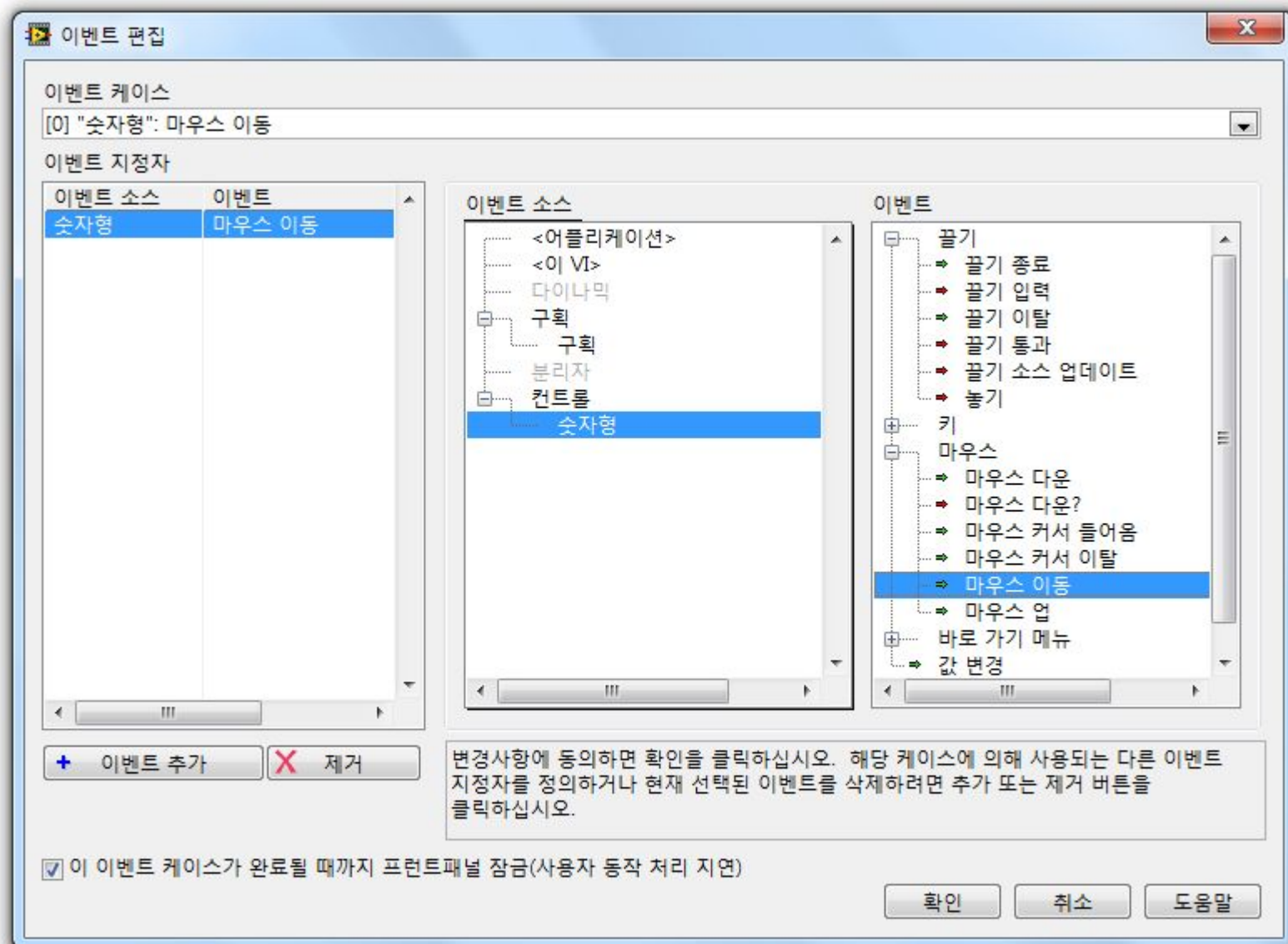


이벤트 구조



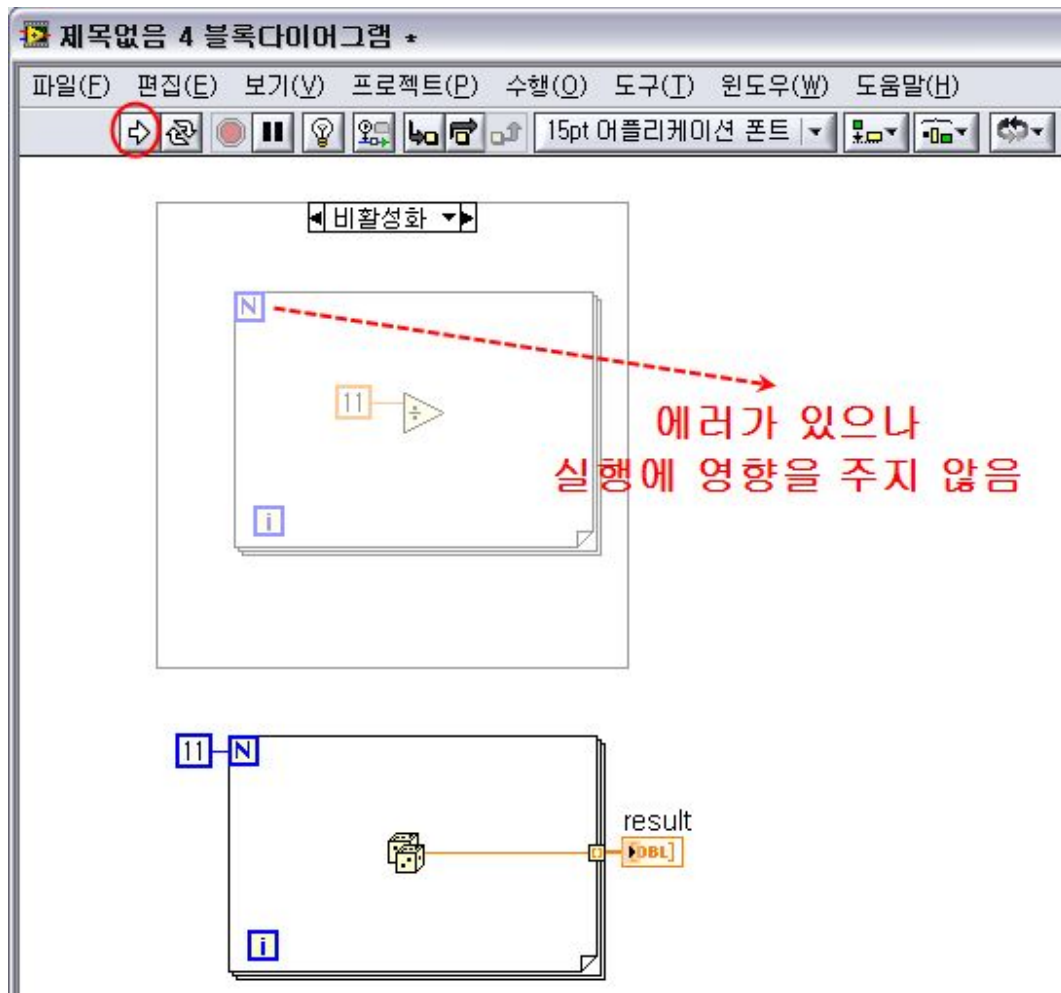
- 이벤트 구조 : 마우스나 키보드를 통해 발생하는 이벤트에 동기화 하여 코드 실행.

이벤트 편집



실습4-10-1) 정적 이벤트 사용법

다이어그램 비활성화 구조



실습4-11-1)

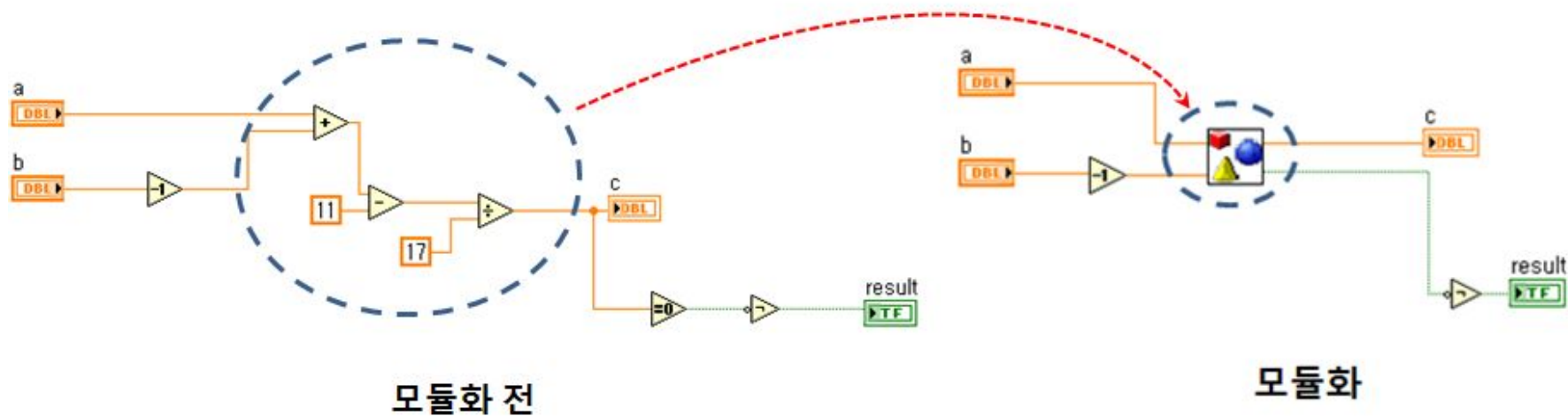
다이어그램 비활성화 구조 사용법

4장 요약

- 반복 구조 **While vs For**
- 케이스 구조
- 시퀀스 구조
- 수식 노드
- 이벤트 구조
- 다이어그램 비활성화 구조

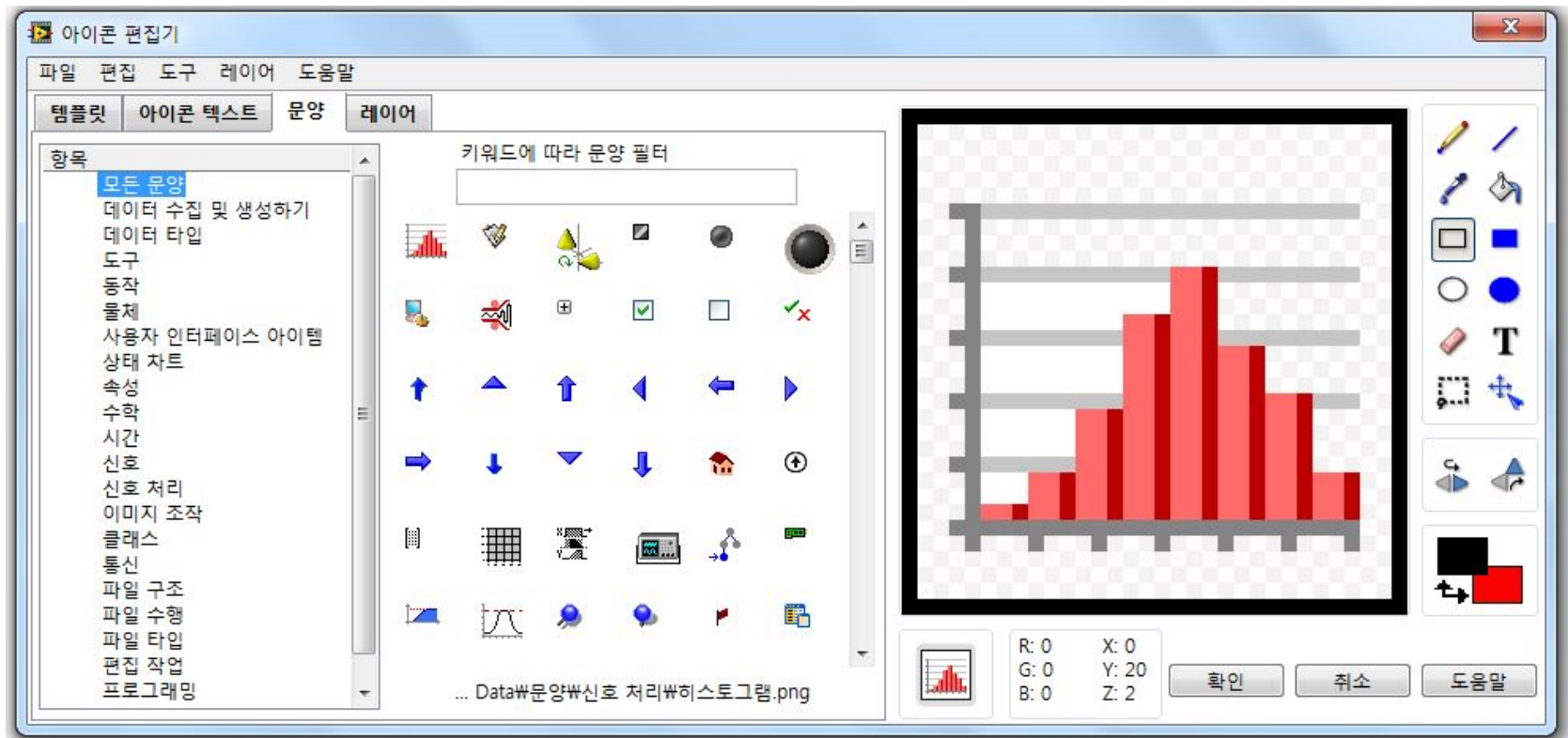
5. 모듈화 프로그램

subVI

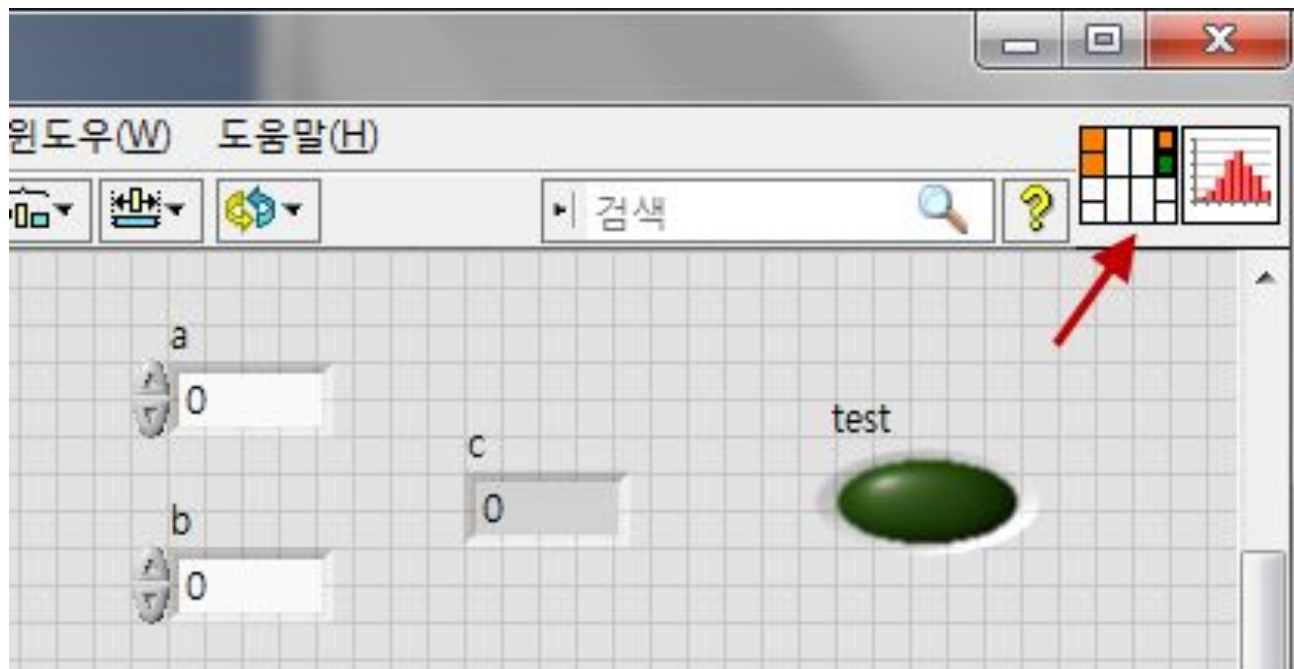


- 코드를 손쉽게 재사용 가능함.

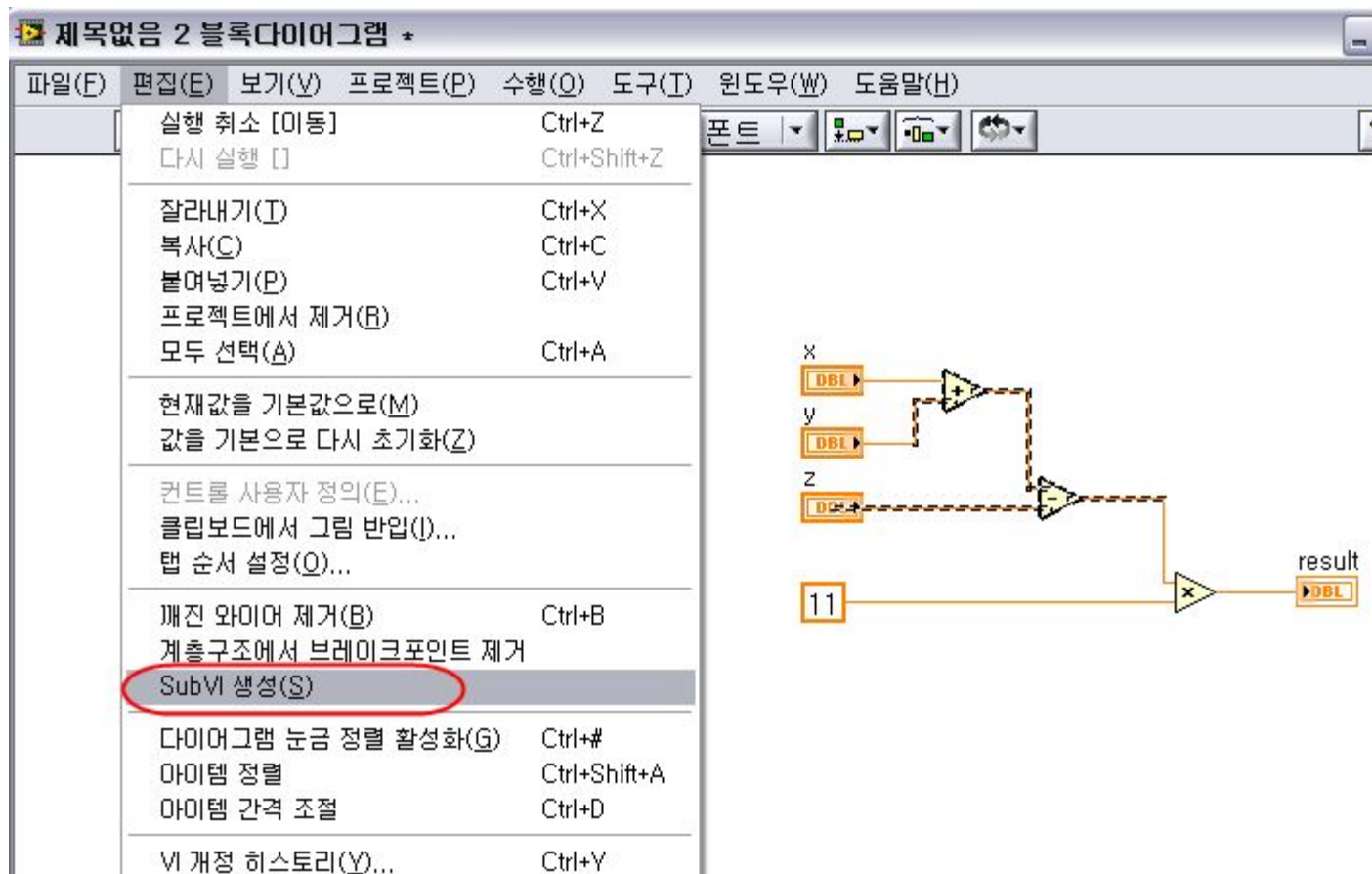
subVI 생성 1단계 - 아이콘 편집



subVI 생성 2단계 - 컨넥터 연결



자동으로 subVI 생성



실습5-4-1) **SUBVI** 생성

5장 요약

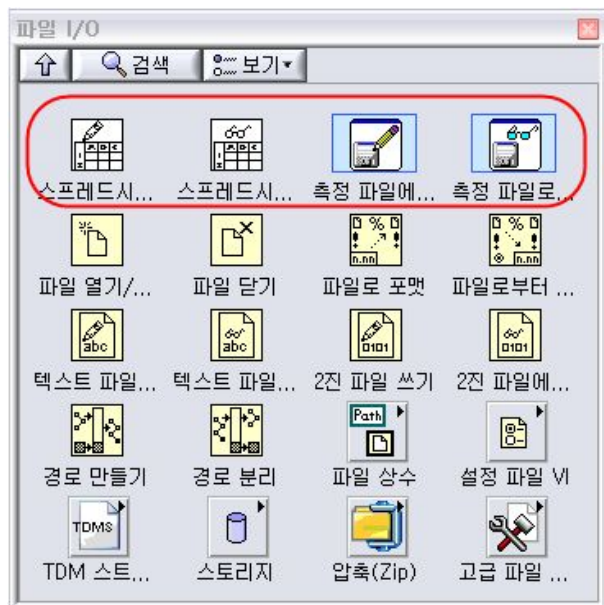
- **subVI** 의미
- **subVI** 생성 순서 - 아이콘 디자인과 커넥터 연결

6. 파일 입출력

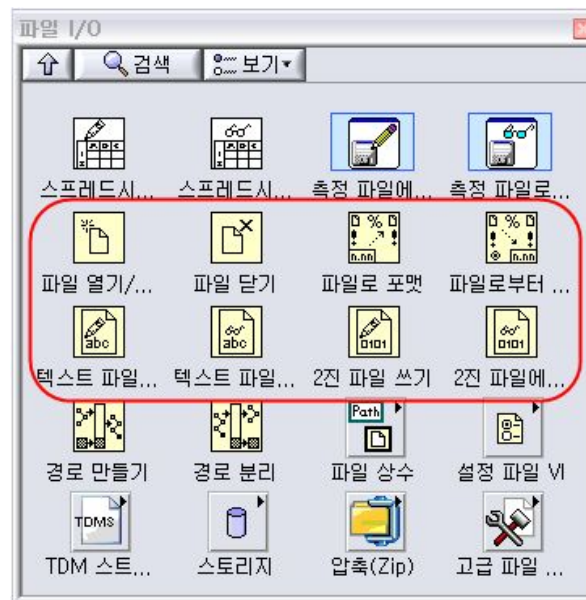
파일 포맷

- ◆ 아스키 : 메모장 등에서 데이터 확인 가능
- ◆ 바이너리 : 속도나 용량이 효율적임
- ◆ **LVM : LabVIEW용 아스키**
- ◆ **TDMS : LabVIEW용 바이너리**

상위/하위레벨 노드들

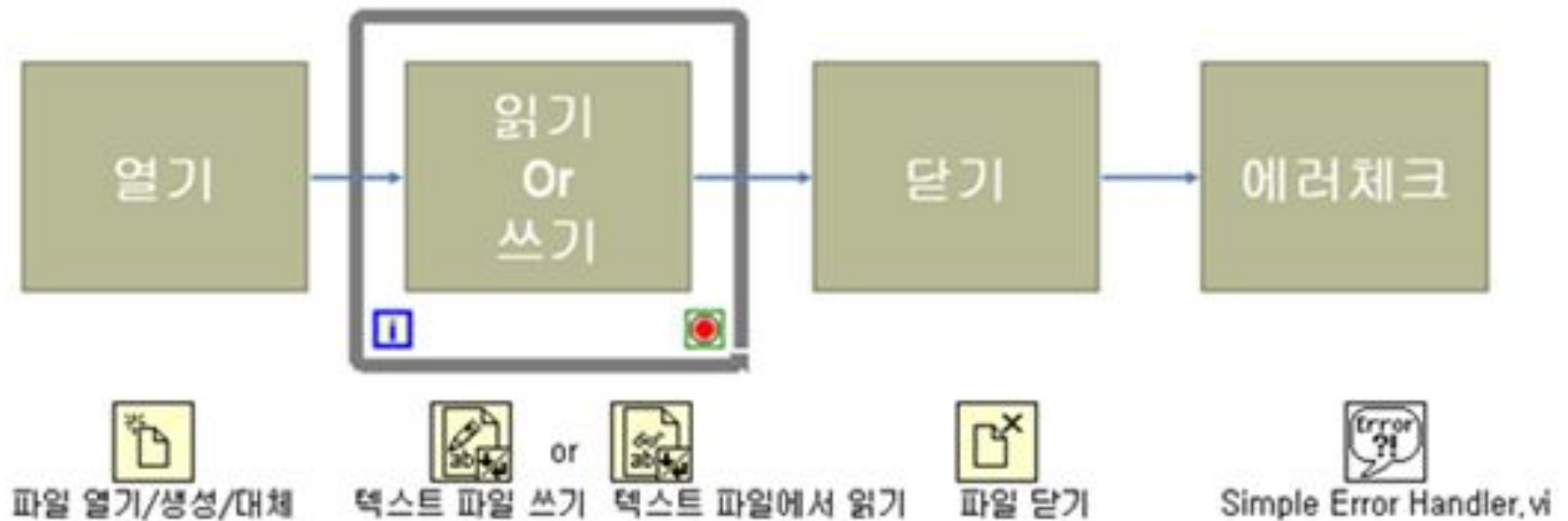


상위레벨

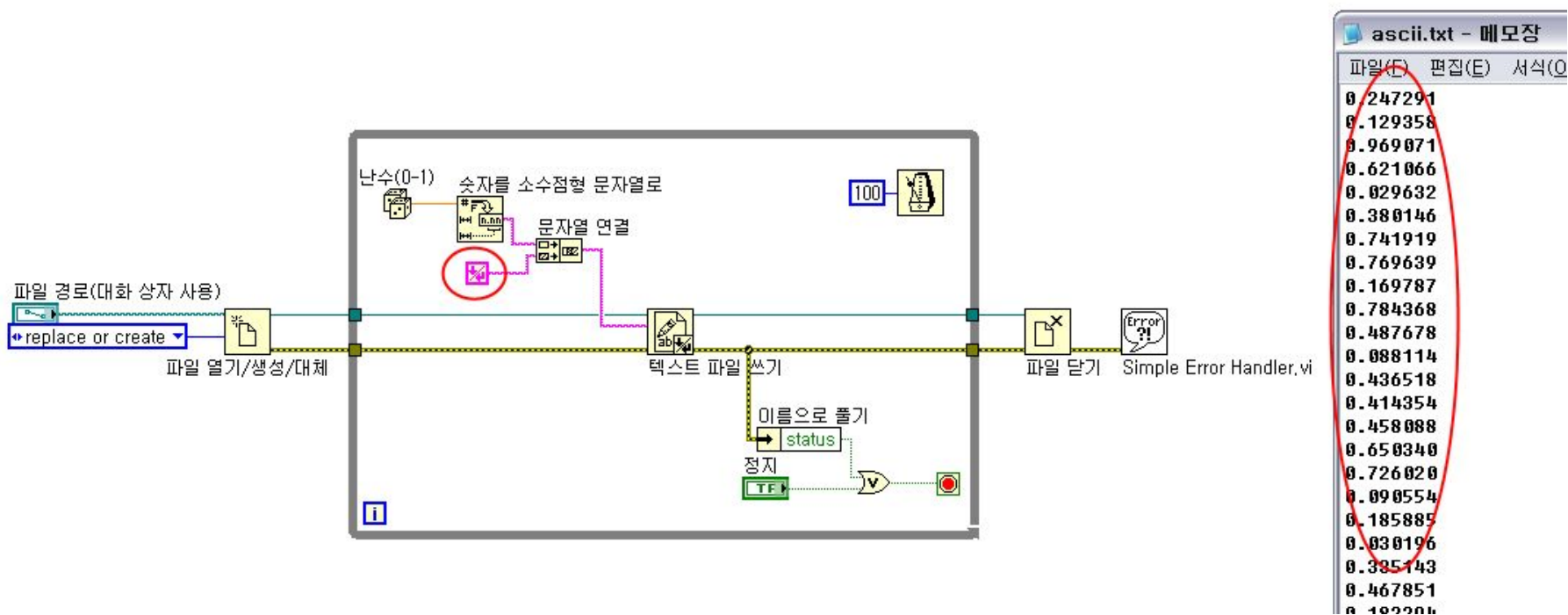


하위레벨

하위레벨 노드 프로그램 순서



아스키 파일 쓰기



실습6-3-1) 아스키 파일 쓰기

바이너리 파일 쓰기

파일 경로(대화 상자 사용)
D:\EduVIEW\book\기본\VI\binary.dat

데이터
0
0.873152
0.0886341
0.636527
0.0491971
0.122491

목없는 3 블록 다이어그램 -

F) 편집(E) 보기(V) 프로젝트(P) 수행(Q) 도구(T) 윈도우(W) 도움말(H)

12pt 시스템 폰트

100 N

난수(0-1)

데이터

데이터 변환 없이 바로 연결

파일 경로(대화 상자 사용)
replace or create

파일 열기/생성/대체

2진 파일 쓰기

파일 닫기

Simple Error Handler.vi

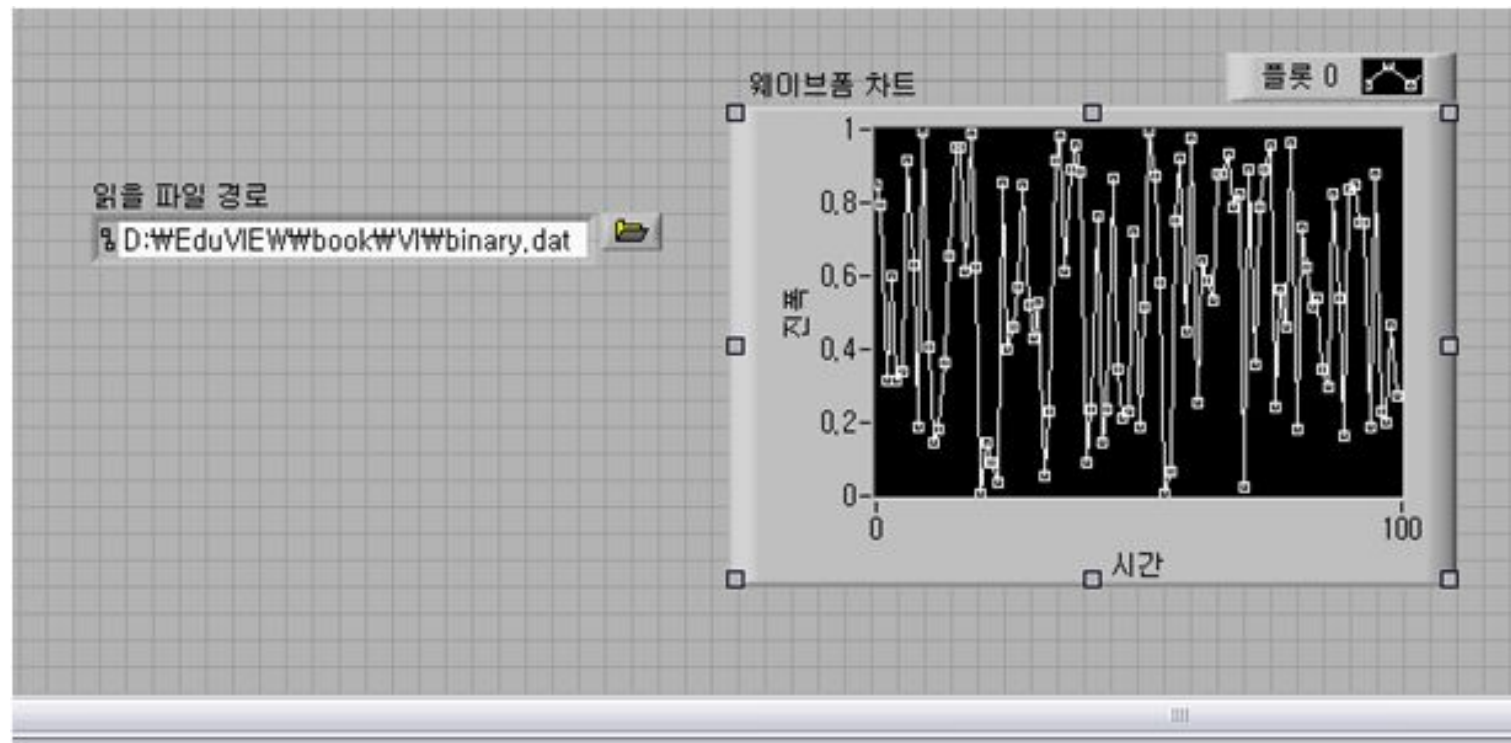
binary.dat - 메모장

파일(F) 편집(E) 서식(O) 보기(V) 도움말(H)

d?儀??z?0?載??M??V??섀E0 ??9 稔?濕?침???極?陸F 6x1

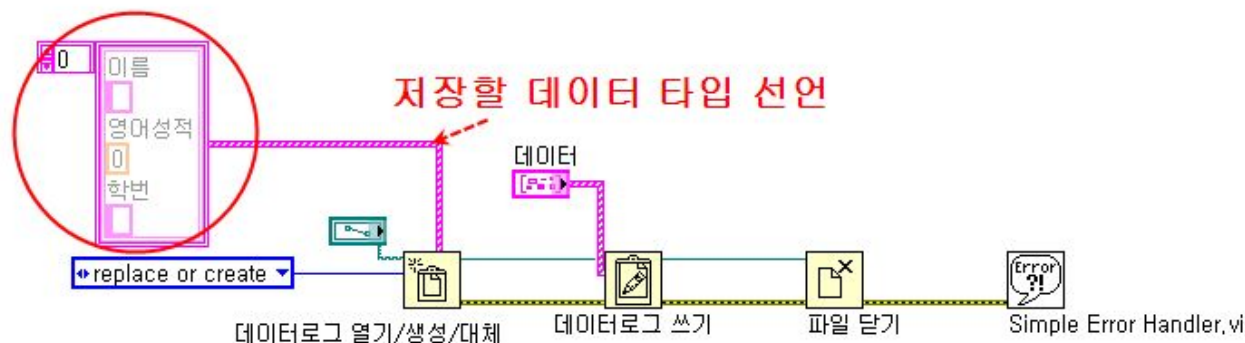
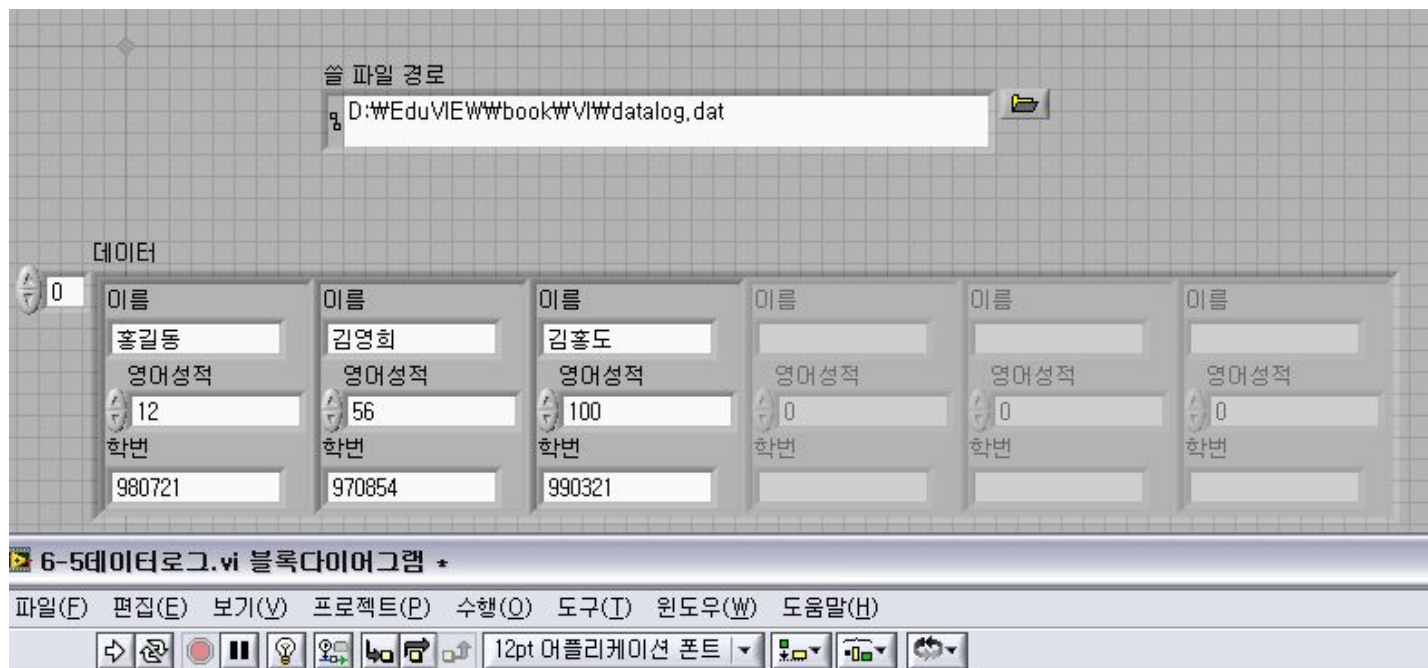
실습6-4-1) 바이너리 파일 쓰기

바이너리 파일 읽기



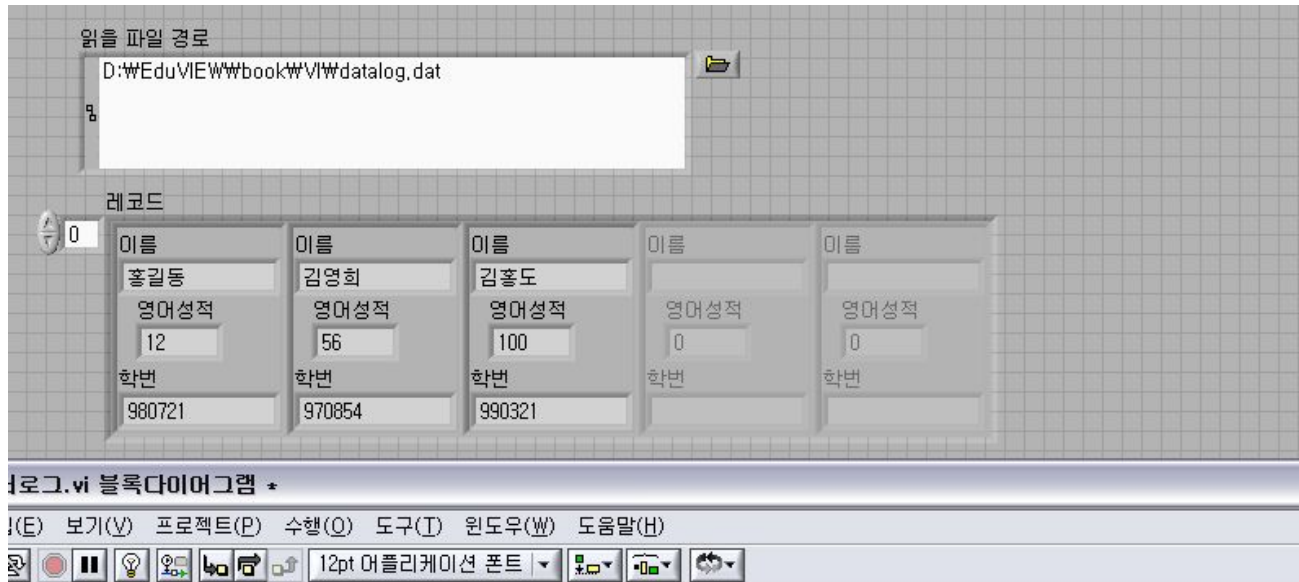
실습6-4-2) 바이너리 파일 읽기

데이터로그 파일 쓰기



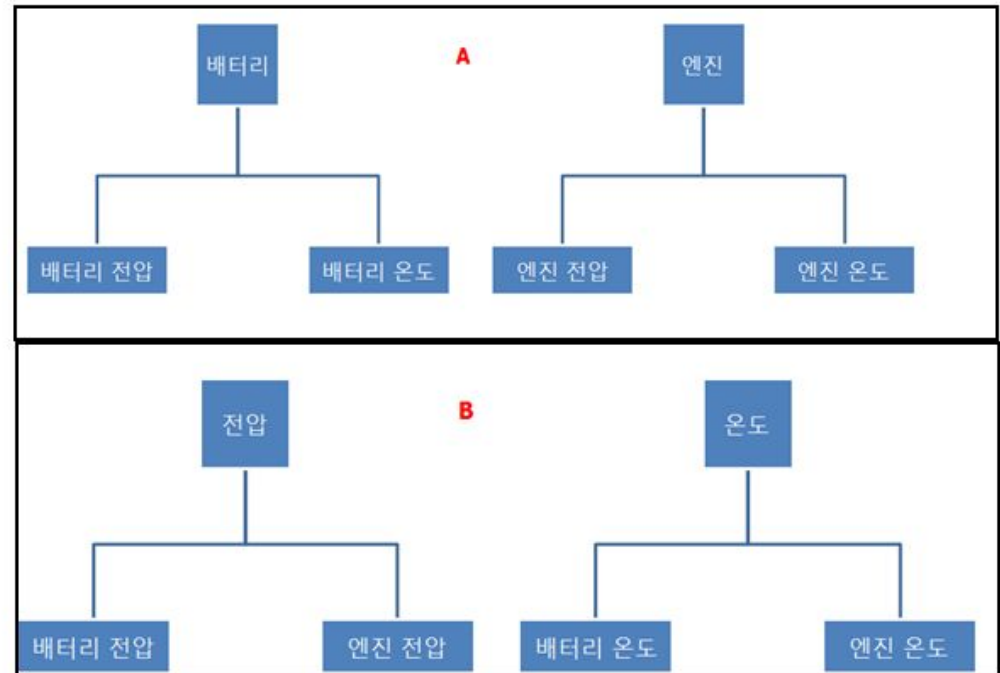
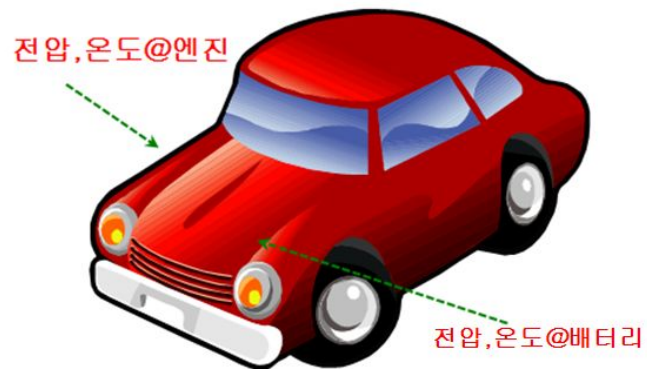
실습6-5-1) 데이터로그 파일 쓰기

데이터로그 파일 읽기



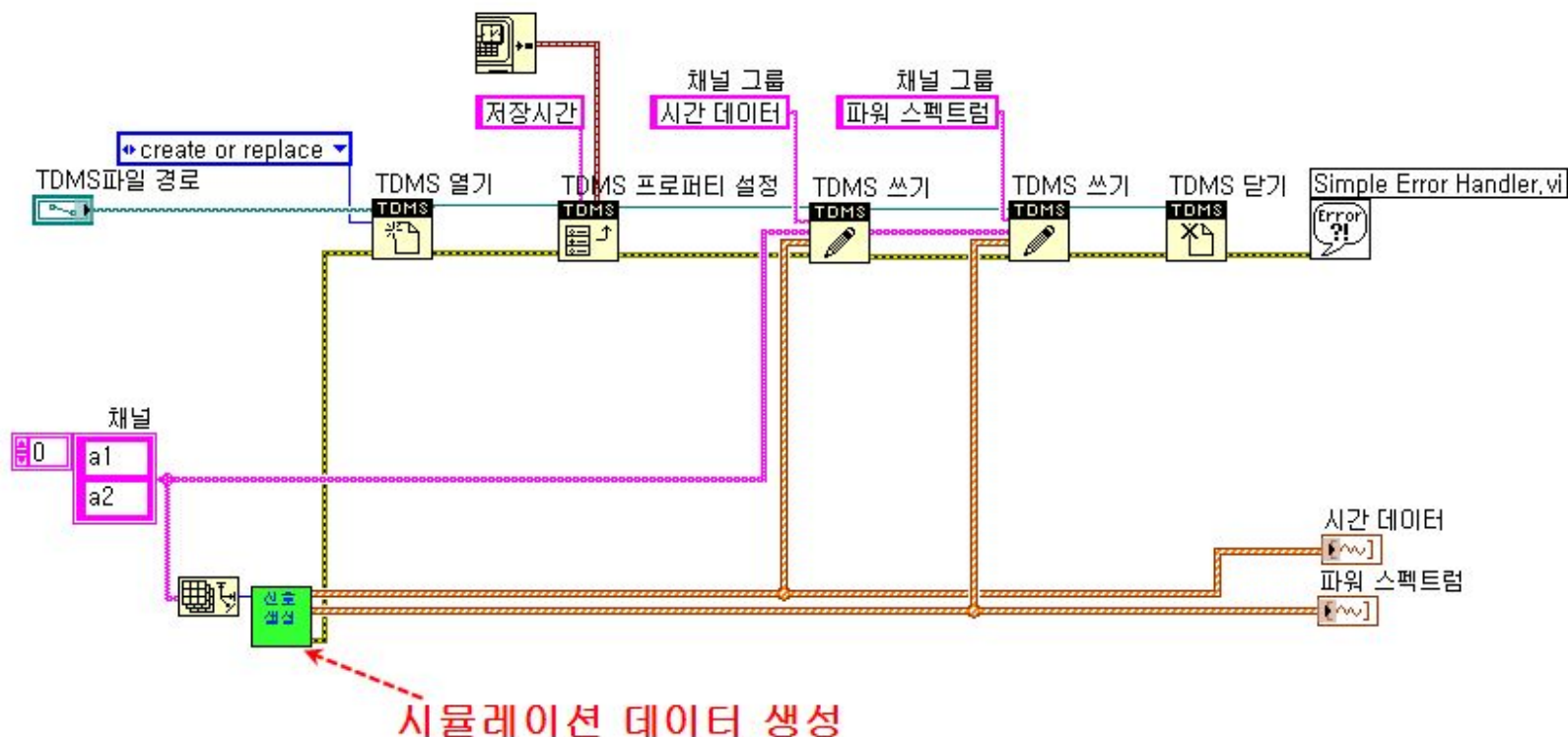
실습6-5-2) 데이터로그 파일 읽기

TDMS



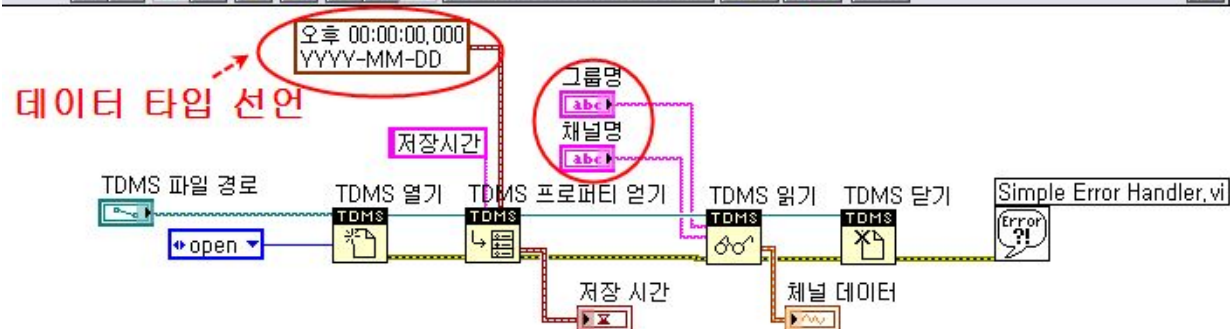
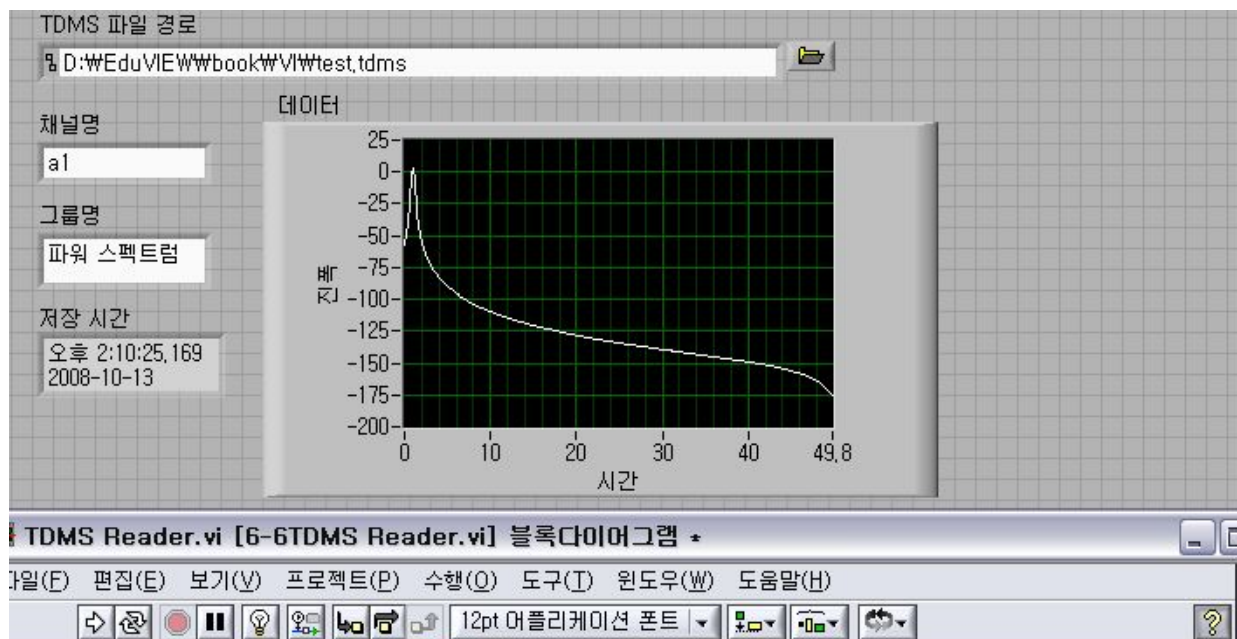
- 간단한 쿼리 적용하여 파일을 읽어 올 수 있어 데이터 수집과 함께 사용하면 유용함.
- 데이터 저장 할 때, '채널'과 '채널 그룹'을 지정함.

TDMS 파일 쓰기



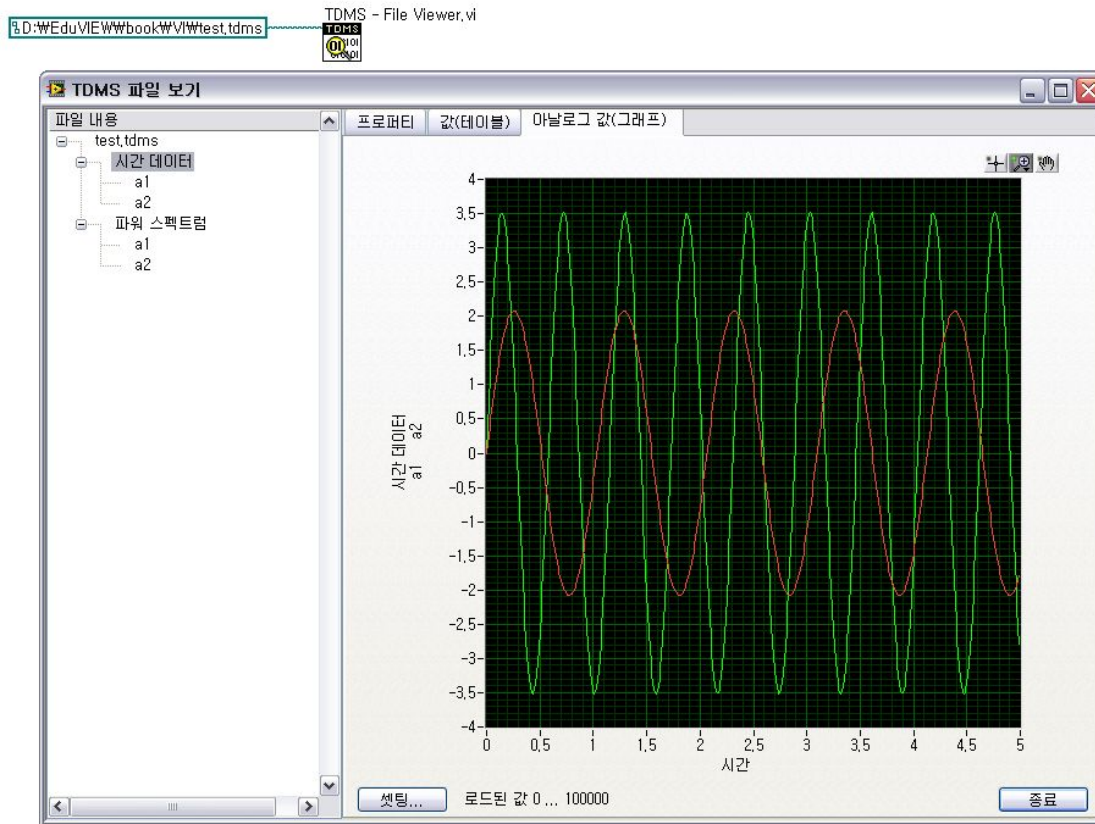
실습6-6-1) TDMS 파일 쓰기

TDMS 파일 읽기



실습6-6-2) TDMS 파일 읽기

TDMS 파일 뷰어



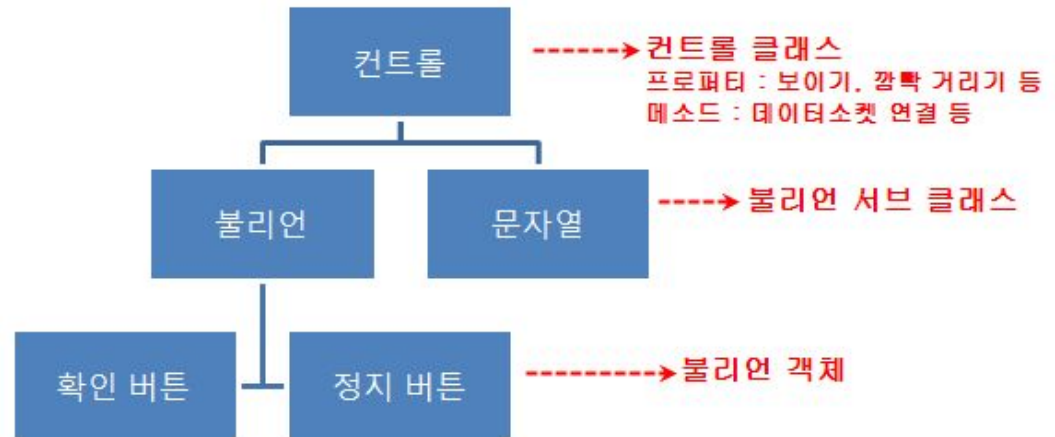
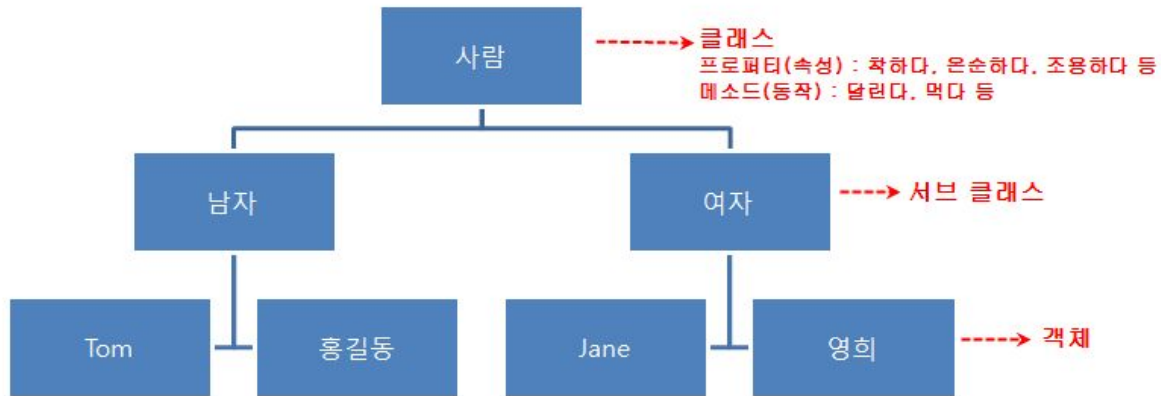
- 읽기 프로그램을 따로 작성하지 않아도 '**TDMS 파일 뷰어**'를 통해
TDMS 파일을 쉽게 읽어올 수 있음.

6장 요약

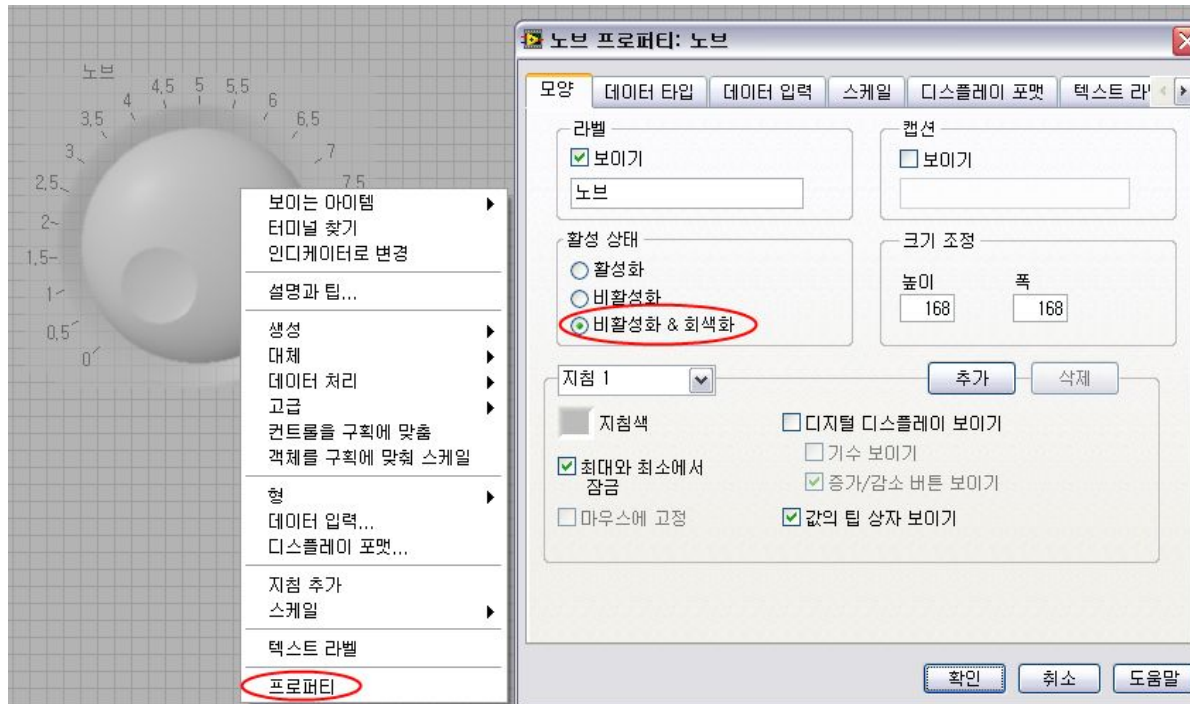
- 아스키 파일 읽고 쓰기
- 바이너리 포맷 읽고 쓰기
- 데이터로그 포맷 읽고 쓰기
- **TDMS** 파일 포맷 읽고 쓰기

7. 프런트패널 제어

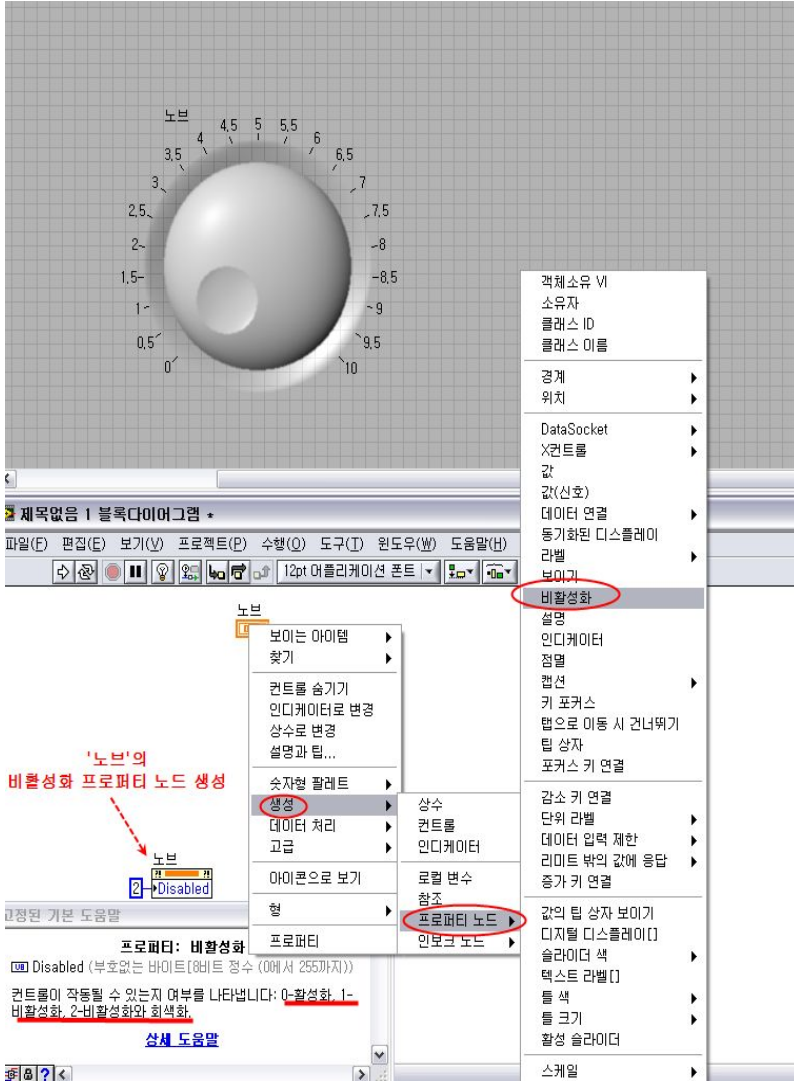
VI 서버



프로퍼티 설정 - 수동

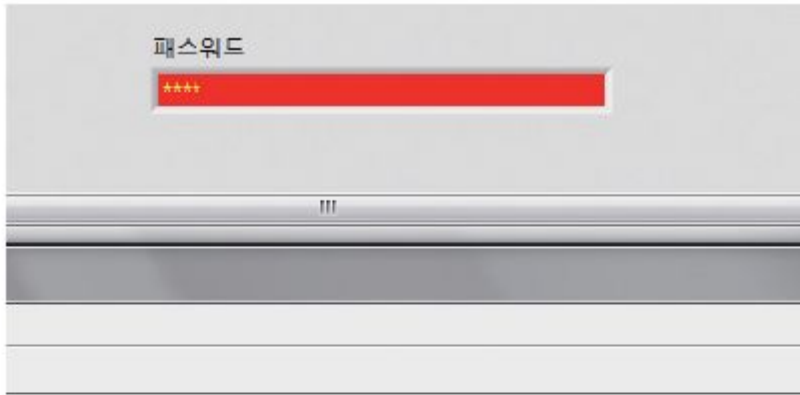


프로퍼티 설정 - 프로퍼티 노드



- 컨트롤이나 인디케이터의 라벨이 매우 중요.
- 터미널에서 바로가기 메뉴로 생성 및 설정

멀티 프로퍼티 노드

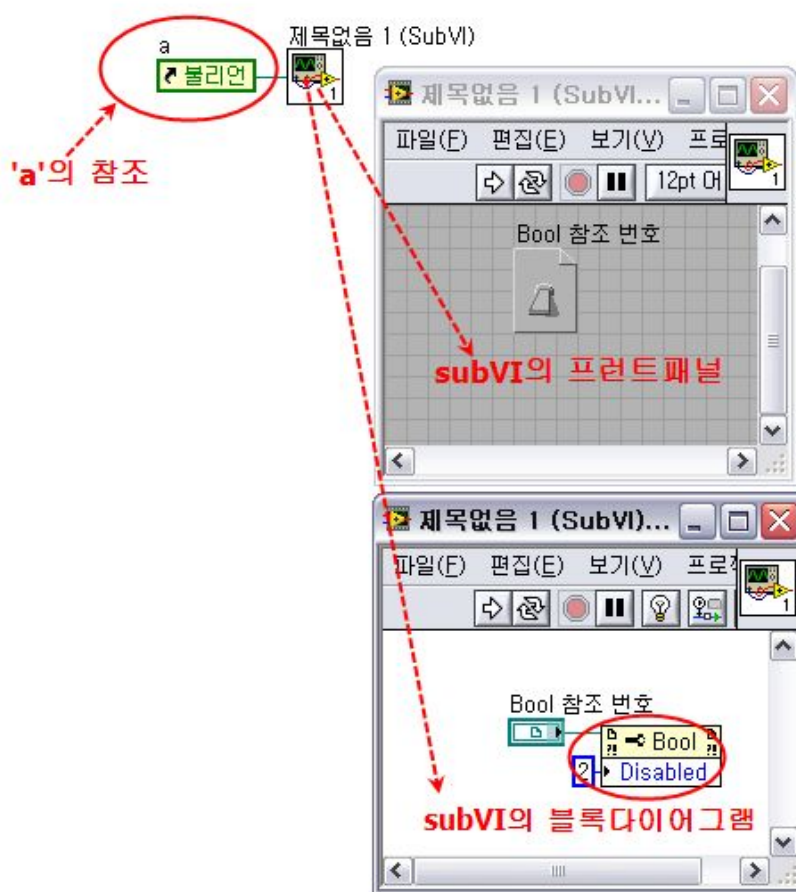


하나의 프로퍼티 노드를 이용하여 여러 개의
프로퍼티를 설정 가능함.
실행 순서는 위에서 차례로 실행됨.



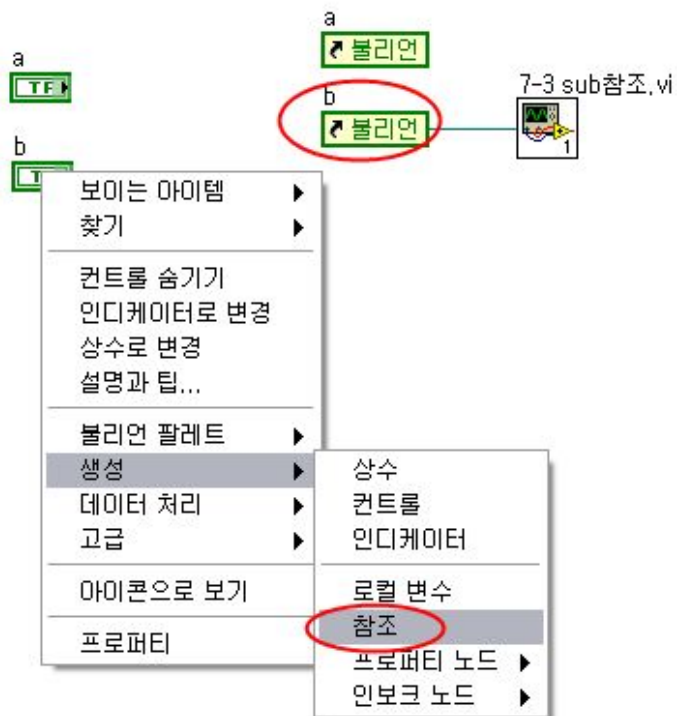
실습7-2-1) 프로퍼티 노드 사용법

참조



- **subVI**에서 프로퍼티나 메소드를 컨트롤 할 때 사용.

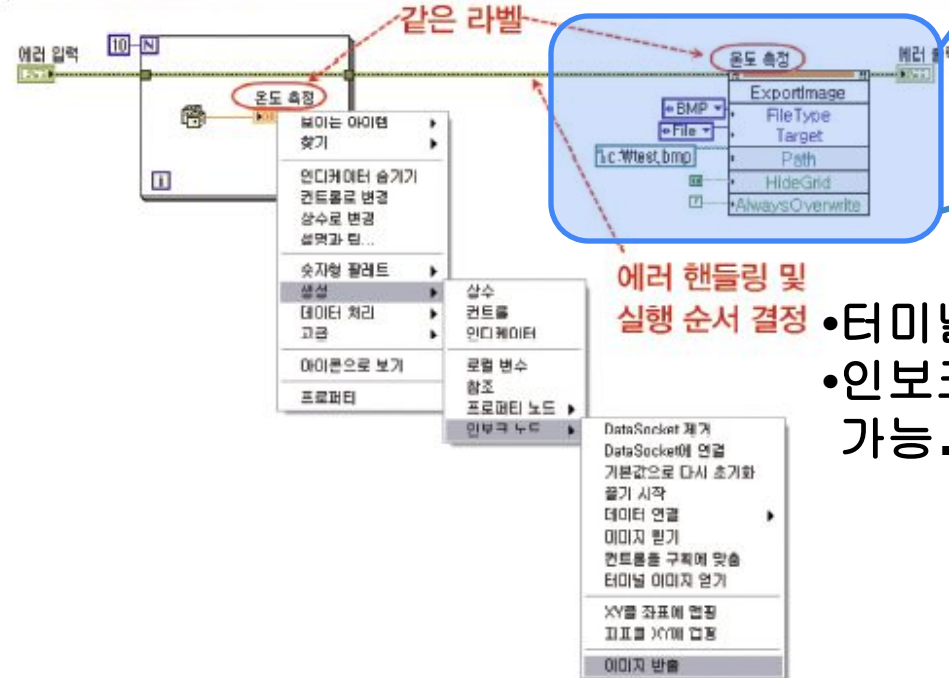
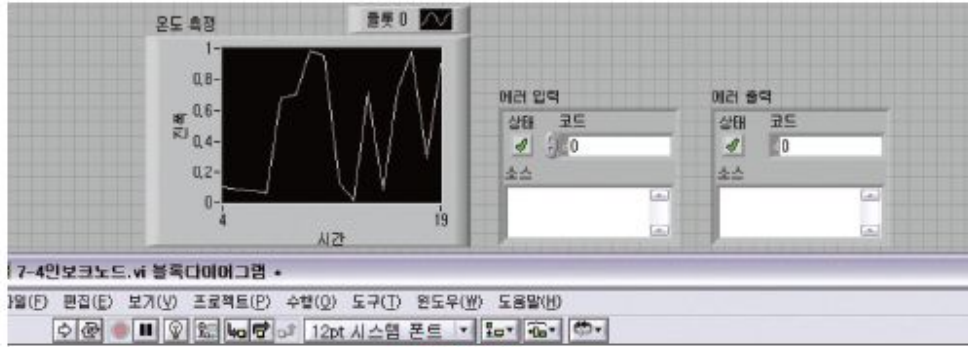
참조 생성법



- 라벨이 매우 중요
- 터미널에서 바로가기 메뉴로 생성함.

실습7-3-1) 참조 사용법

인보크 노드



- 터미널에서 바로가기 메뉴로 생성 및 설정
- 인보크 노드는 한 개의 메소드만 설정 가능.

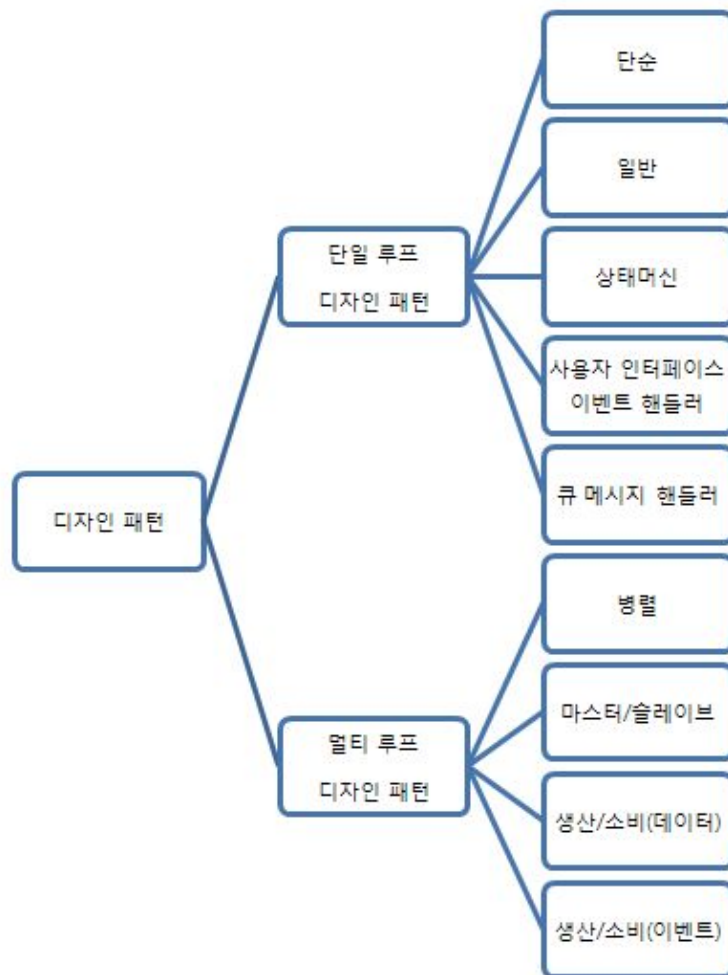
실습7-4-1) 인보크 노드 사용법

7장 요약

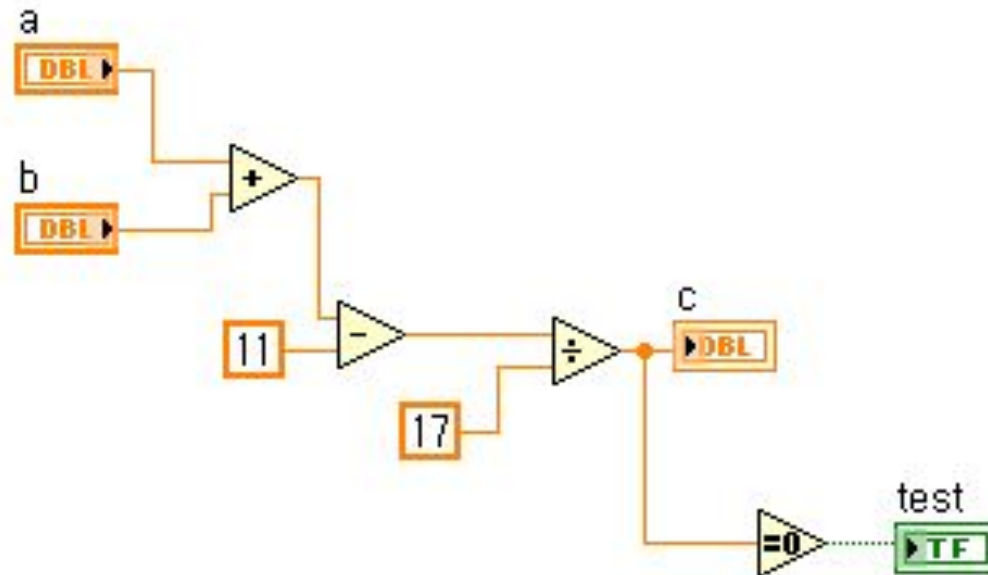
- **VI** 서버
- 프로퍼티 노드
- 참조
- 인보크 노드

8. 디자인 패턴

디자인 패턴 개요

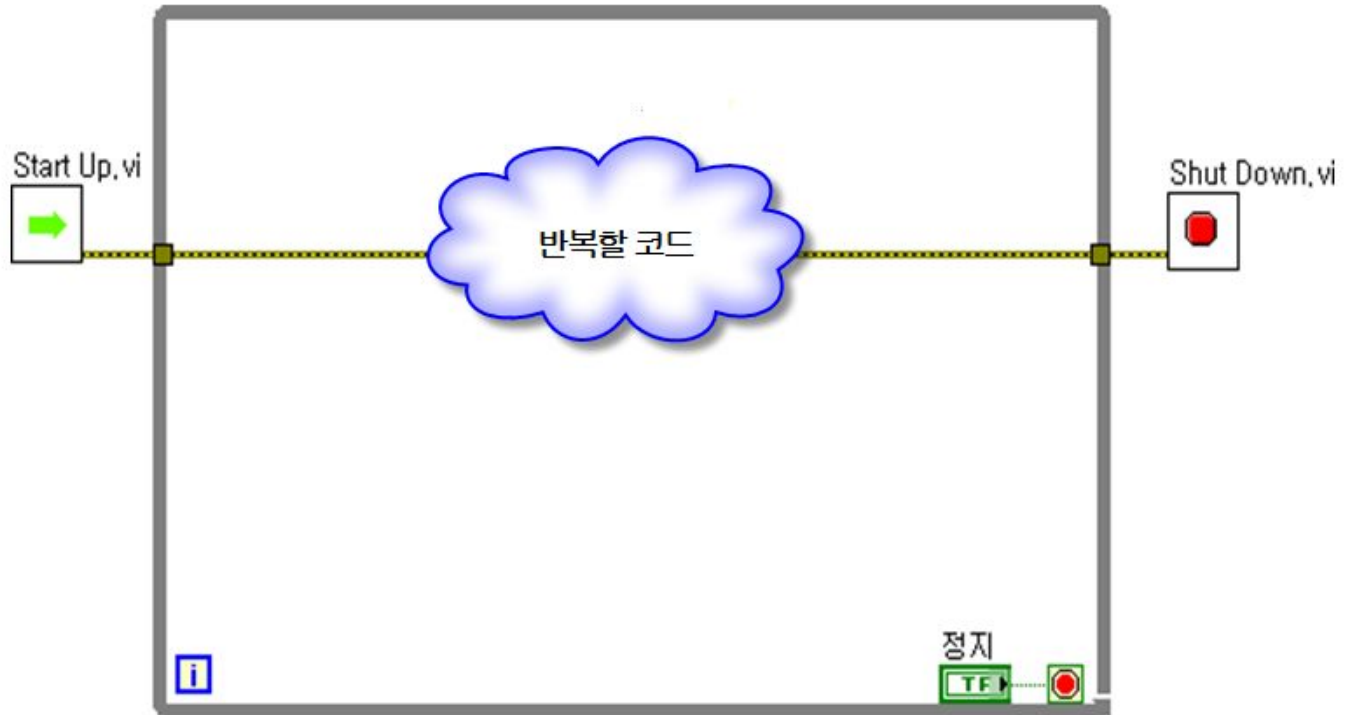


단순 (Simple) VI 디자인 패턴

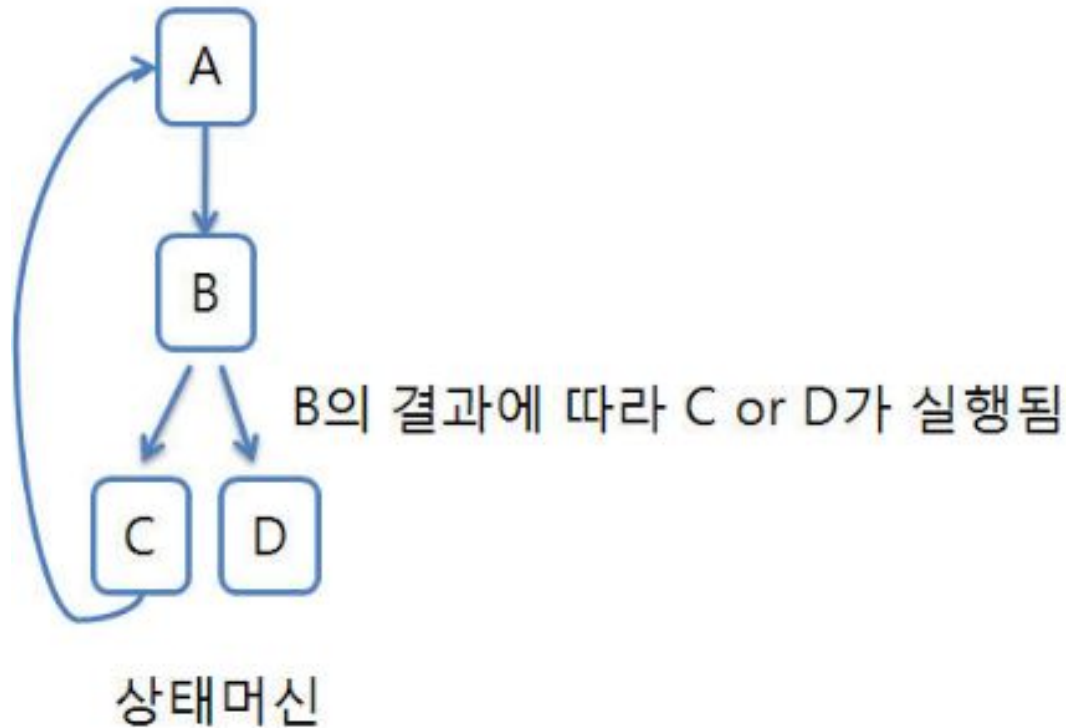


- **subVI**용으로 적합한 디자인 패턴

일반 (General) VI 디자인 패턴

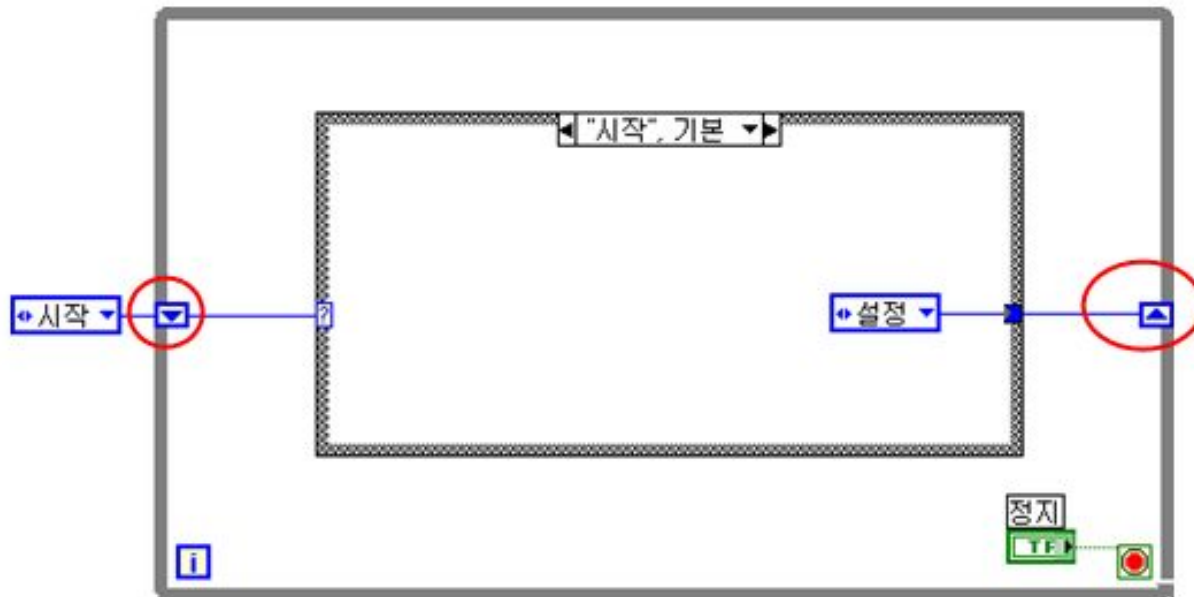


상태머신 적용 프로젝트



- **Task A** 수행한 뒤, **Task B** 수행하고 수행 결과에 따라 **Task C** 또는 **Task D** 수행하는 등의 프로젝트에 적합한 디자인 패턴

상태머신 디자인 패턴

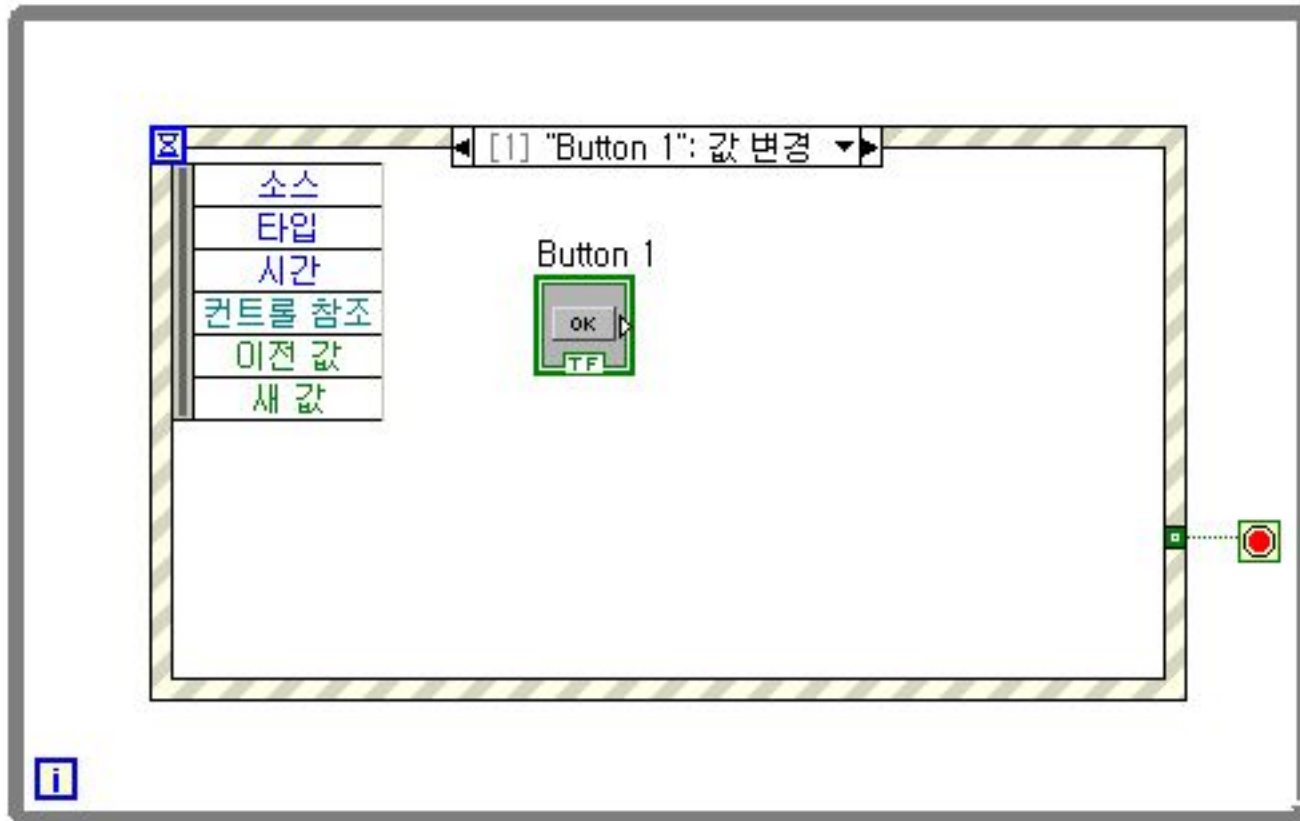


- **While**루프의 시프트 레지스터를 통해 다음 상태의 정보를 전달함.

실습8-4-1)

상태머신 디자인 패턴 사용법

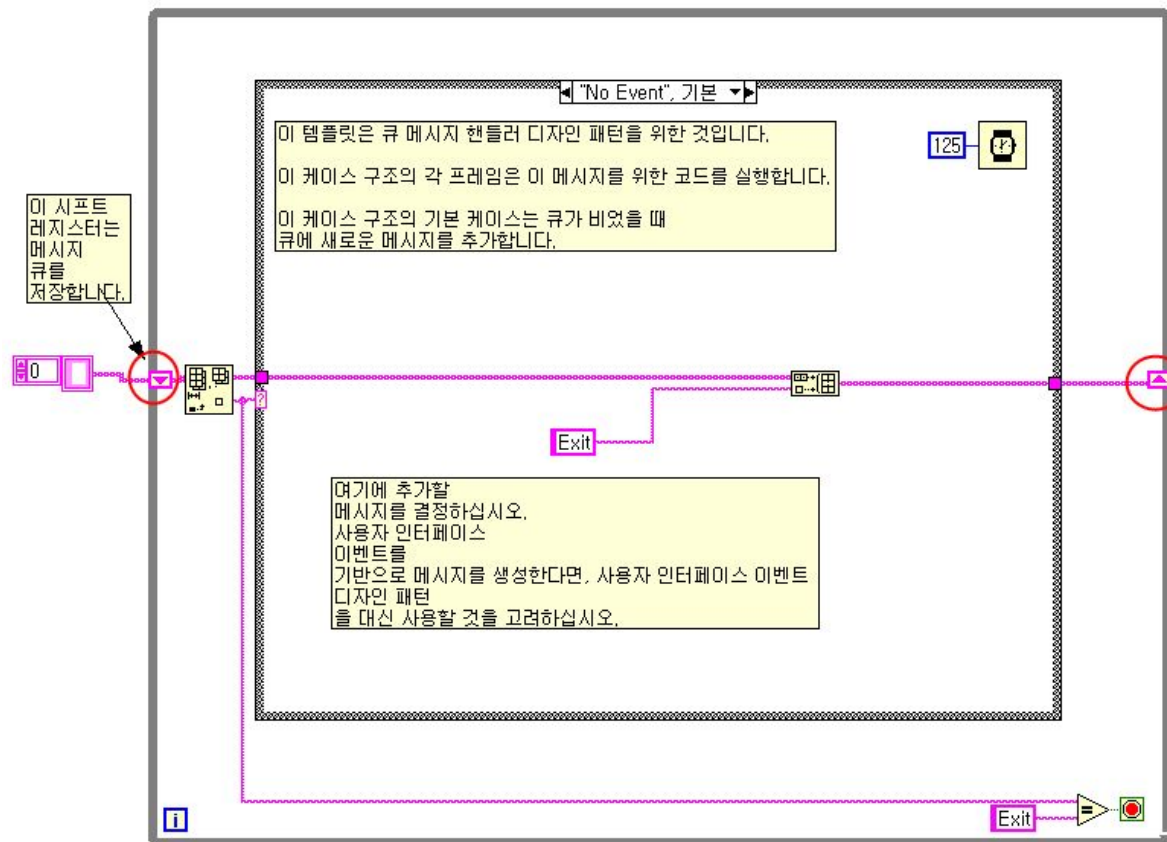
사용자 인터페이스 이벤트 핸들러



실습8-5-1)

사용자 인터페이스 이벤트 핸들러

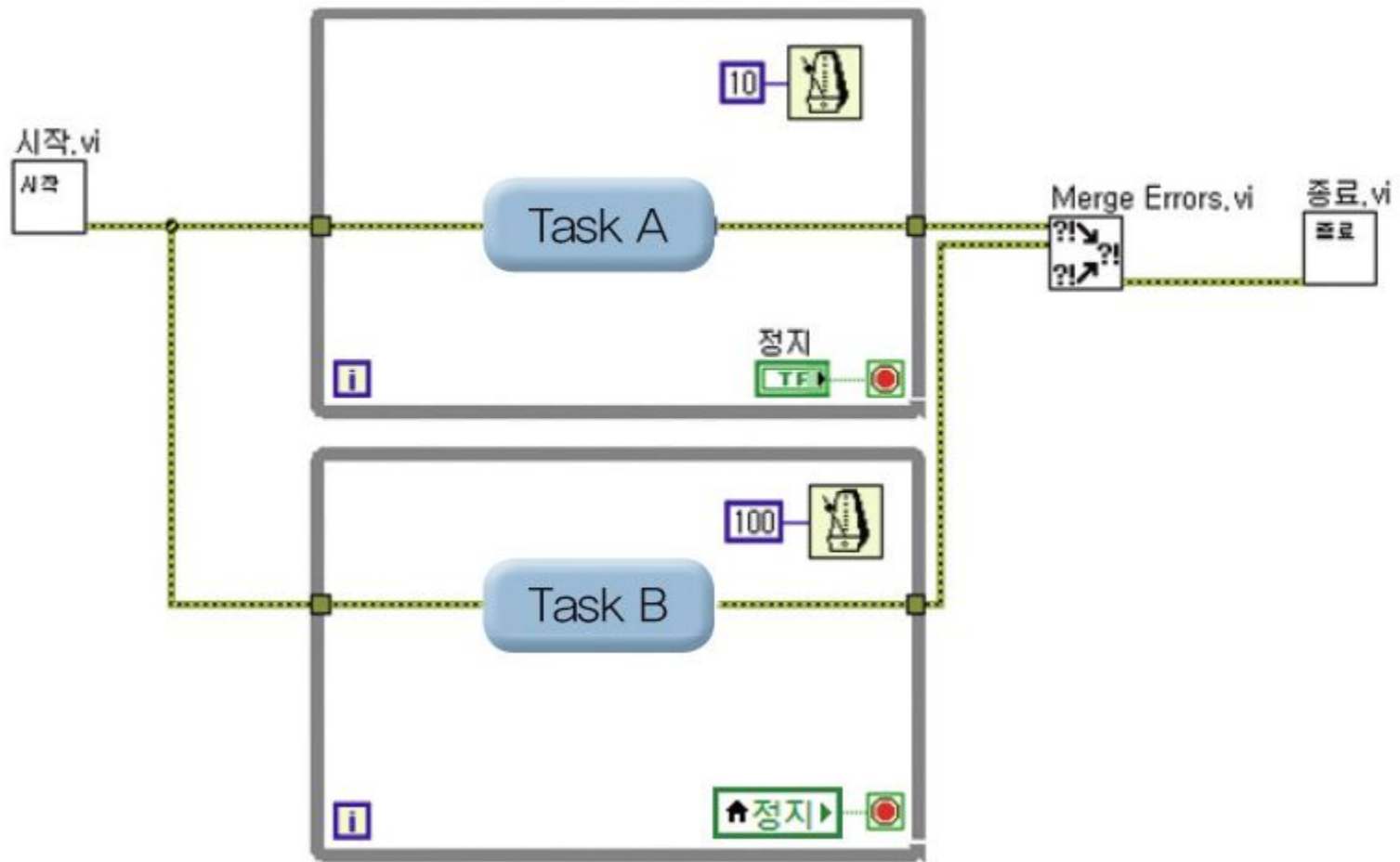
큐 메시지 핸들러 디자인 패턴



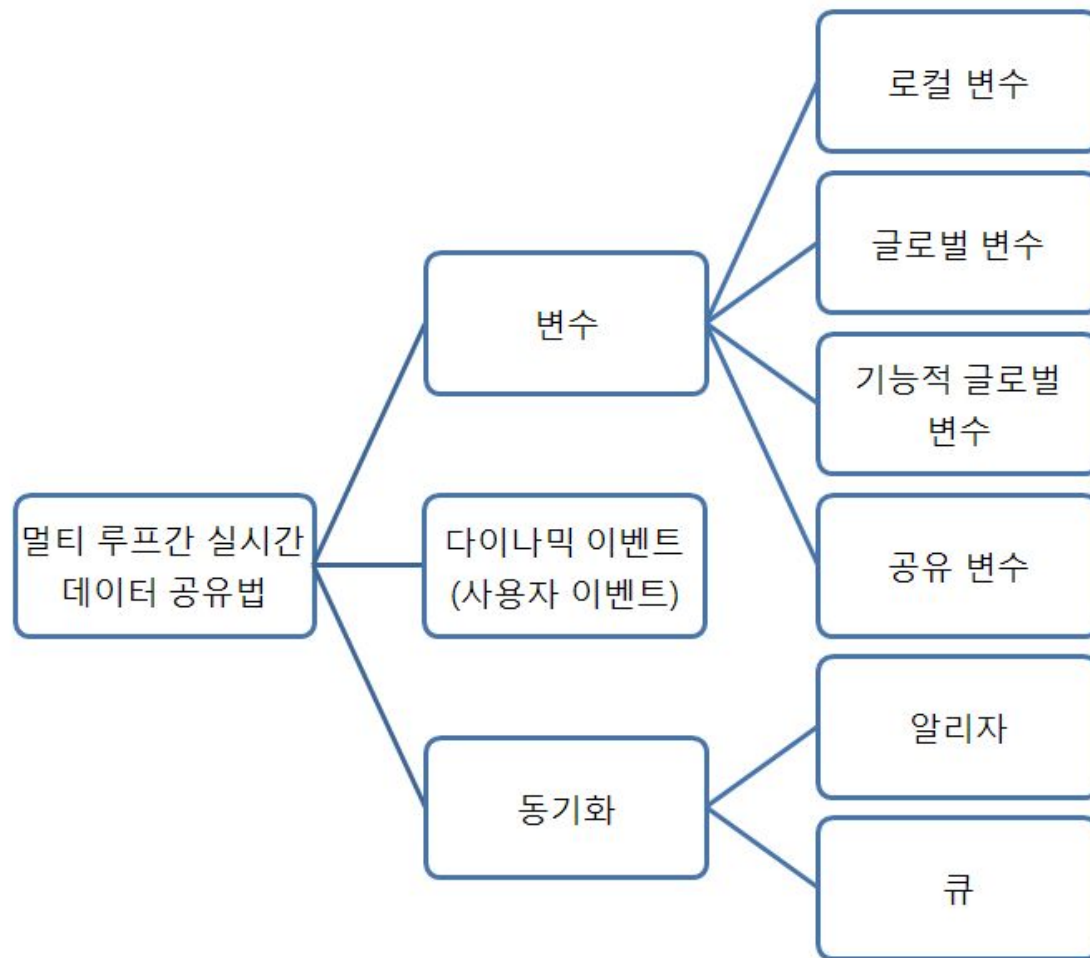
실습8-6-1)

큐 메시지 핸들러 디자인 패턴

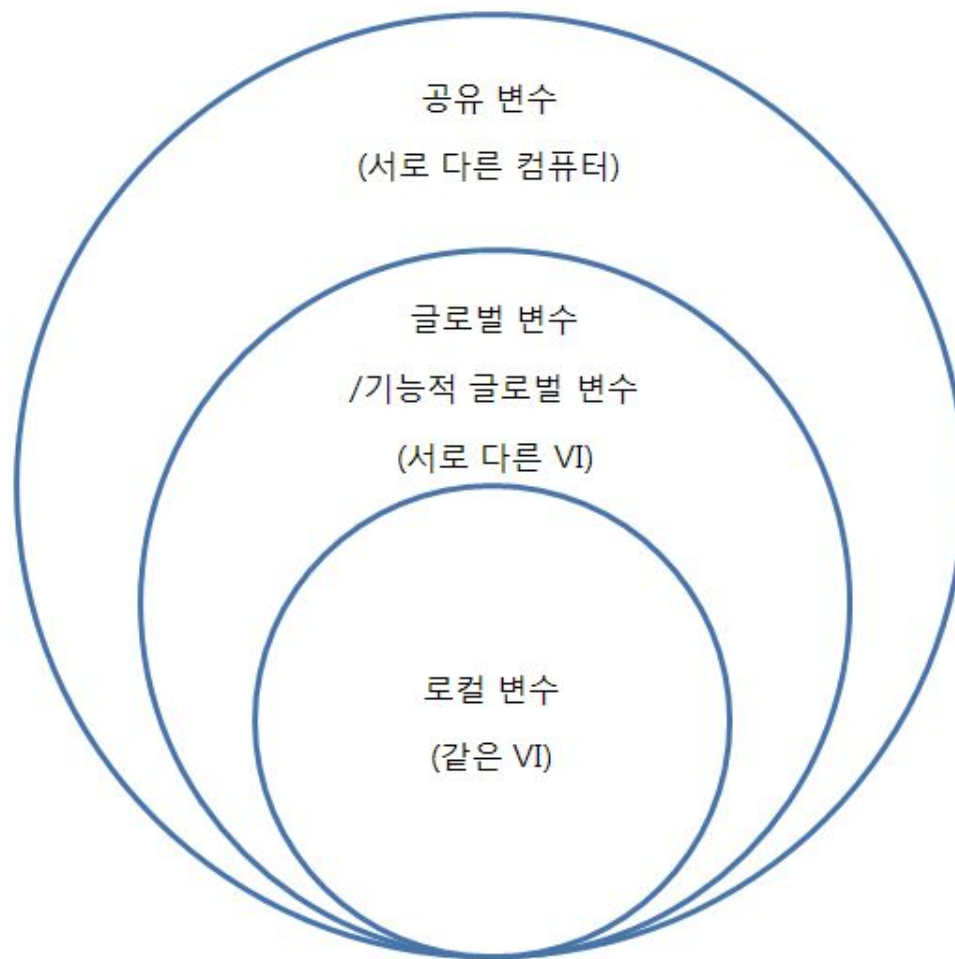
병렬 루프 디자인 패턴



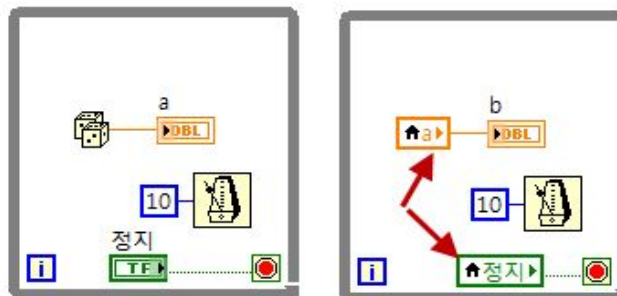
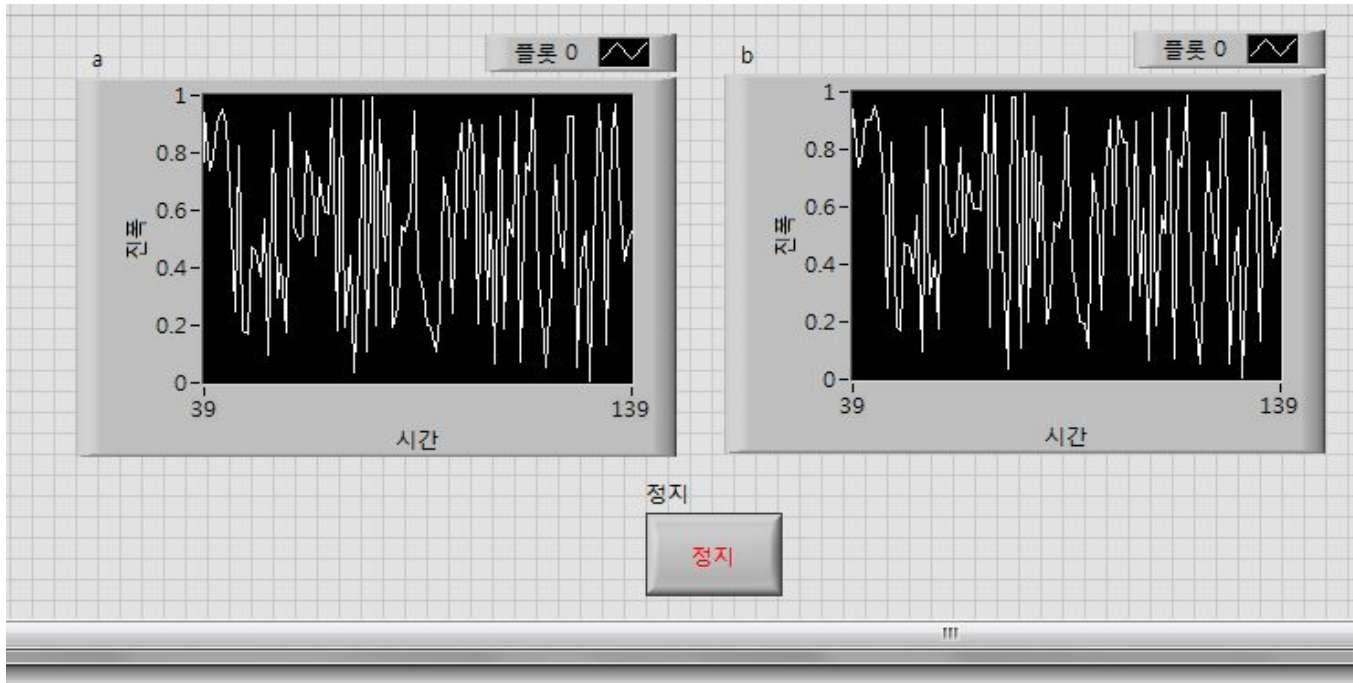
멀티 루프간의 실시간 데이터 공유



변수 분류



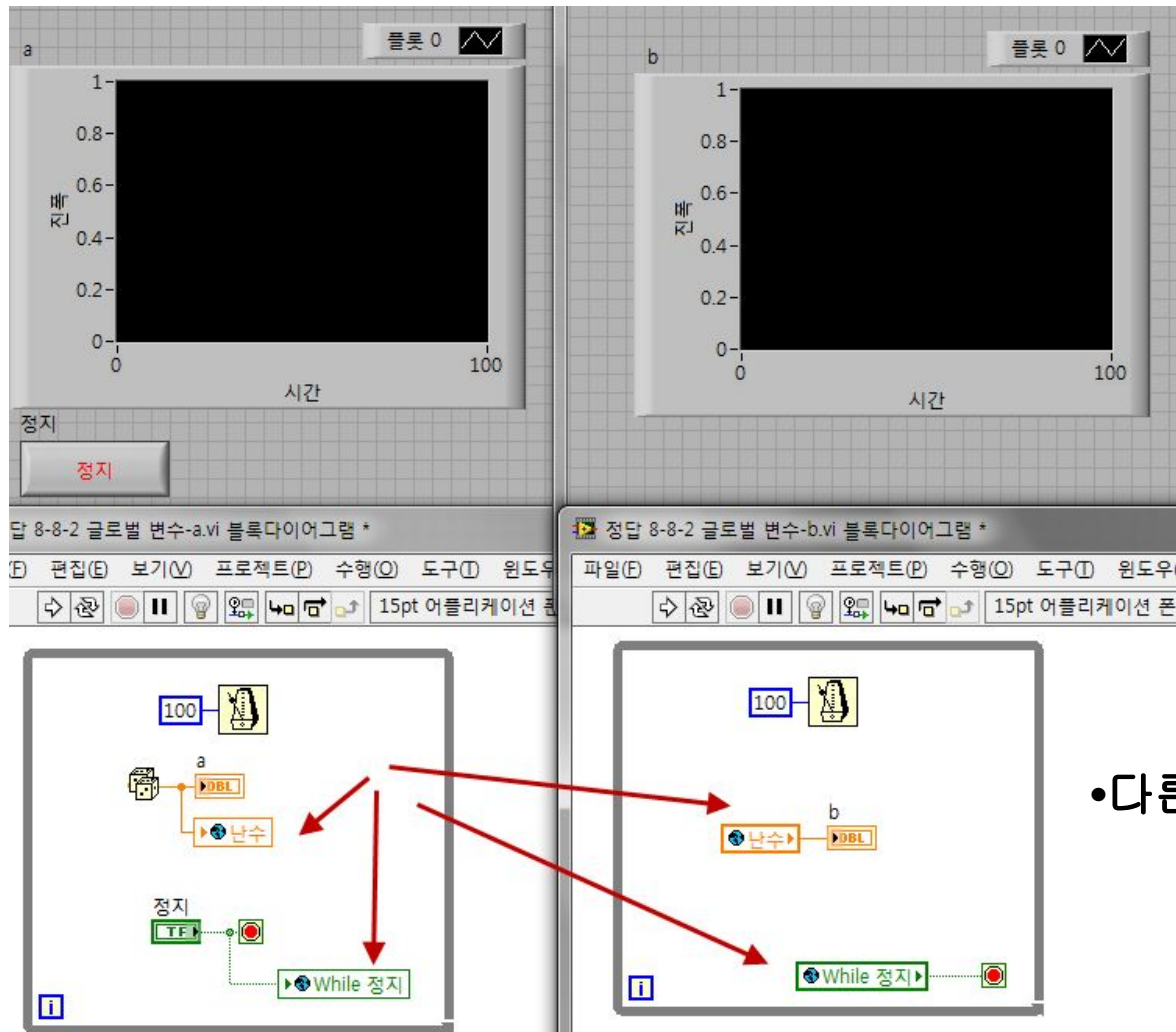
로컬 변수



•같은 VI 내에서 데이터 공유

실습8-8-1) 로컬 변수 사용법

글로벌 변수



• 다른 VI 간에 데이터 공유

실습8-8-2) 글로벌 변수 사용법

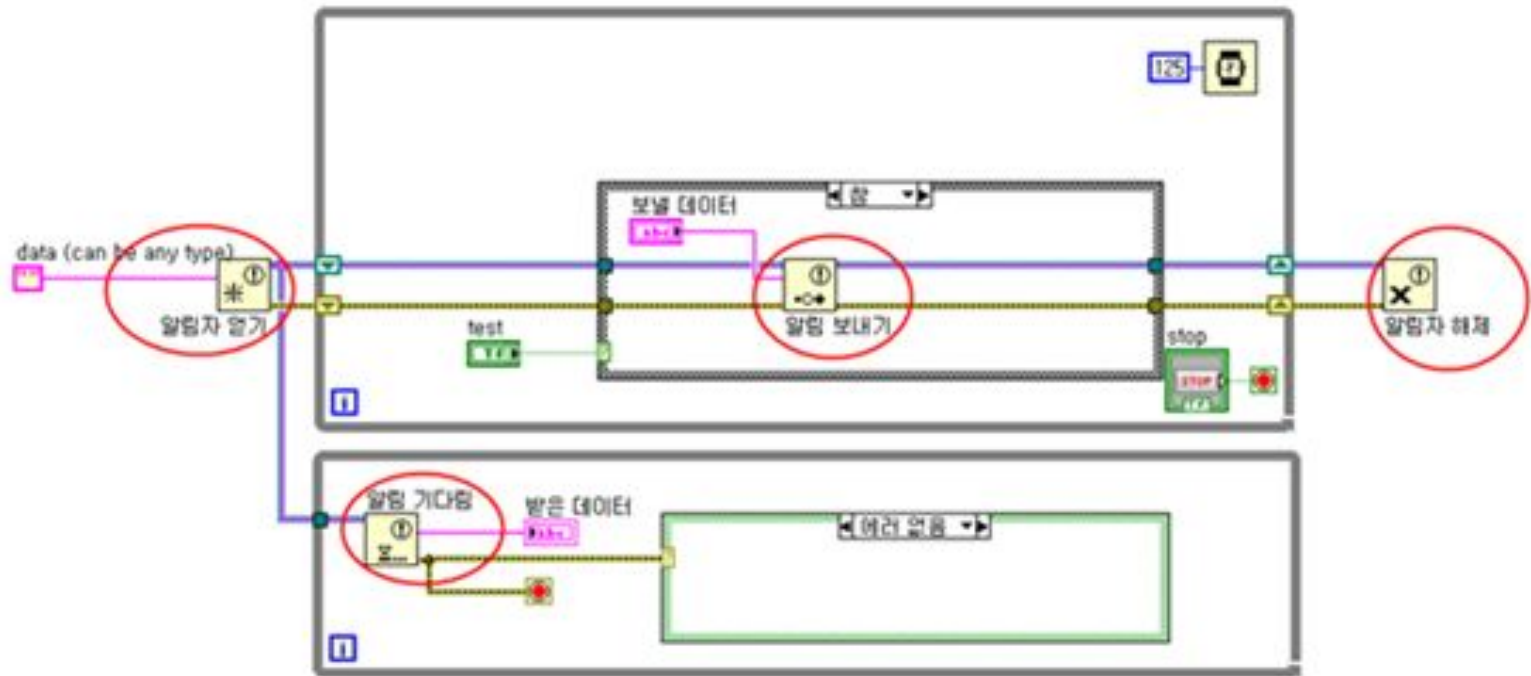
기능적 글로벌 변수



- 글로벌 변수의 기능을 하는 **VI**
- While**루프가 한 번만 실행되고,
초기화 되지 않는 시프트레지스터가 하나 이상 있어야 함.

실습8-8-3) 기능적 글로벌 변수 사용법

마스터/슬레이브 (알림자)

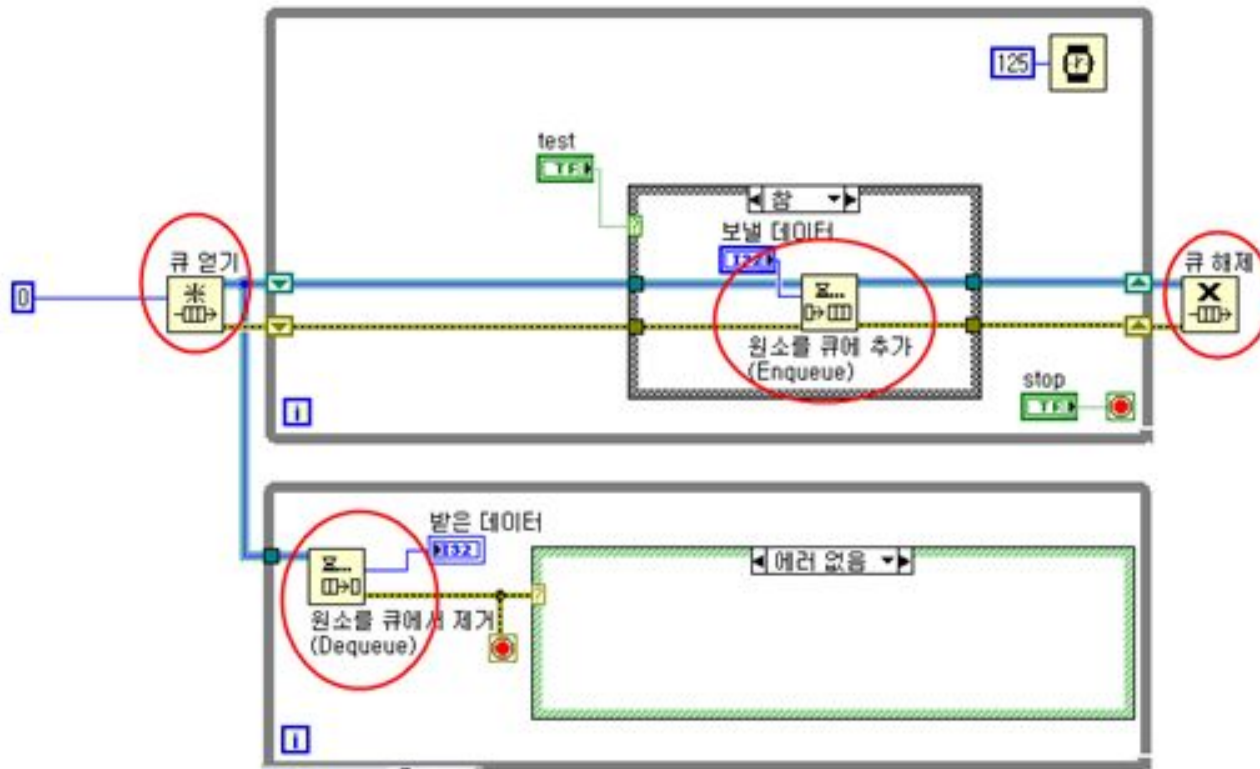


- '알림 보내기' 노드에서 '알림 기다림' 노드로 데이터 실시간 전달

실습8-9-1)

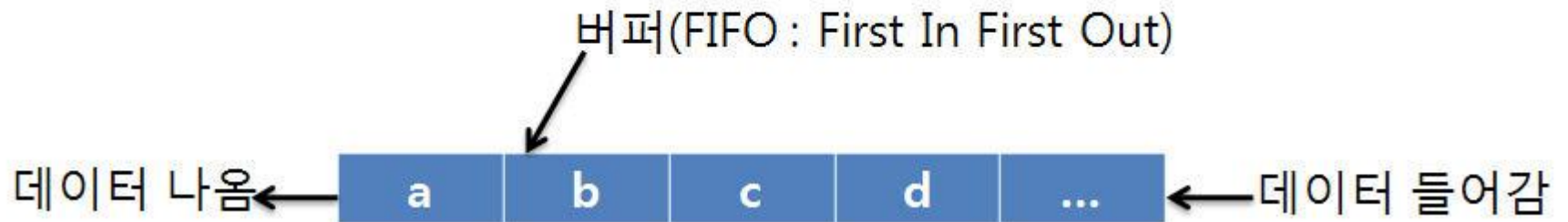
마스터/슬레이브 디자인 패턴 사용법

생산/소비 데이터(큐)



- '원소를 큐에 추가' 노드에서 '원소를 큐에서 제거' 노드로 데이터 실시간 전달

큐의 버퍼 구조

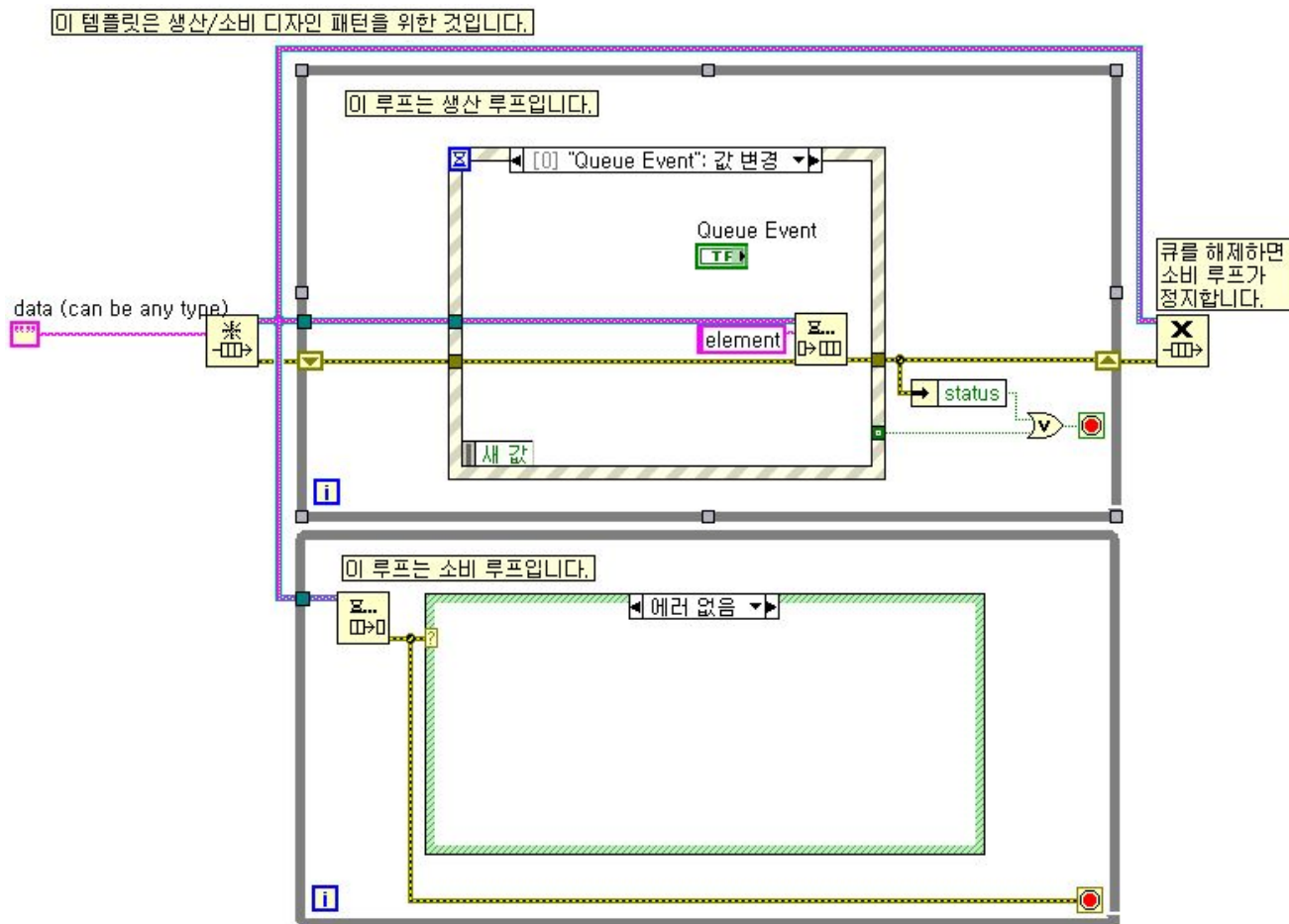


- 알림자와 달리 버퍼 구조를 가짐으로써 데이터 손실을 방지.

실습8-10-1)

생산/소비 데이터 디자인 패턴

생산/소비 이벤트(큐)

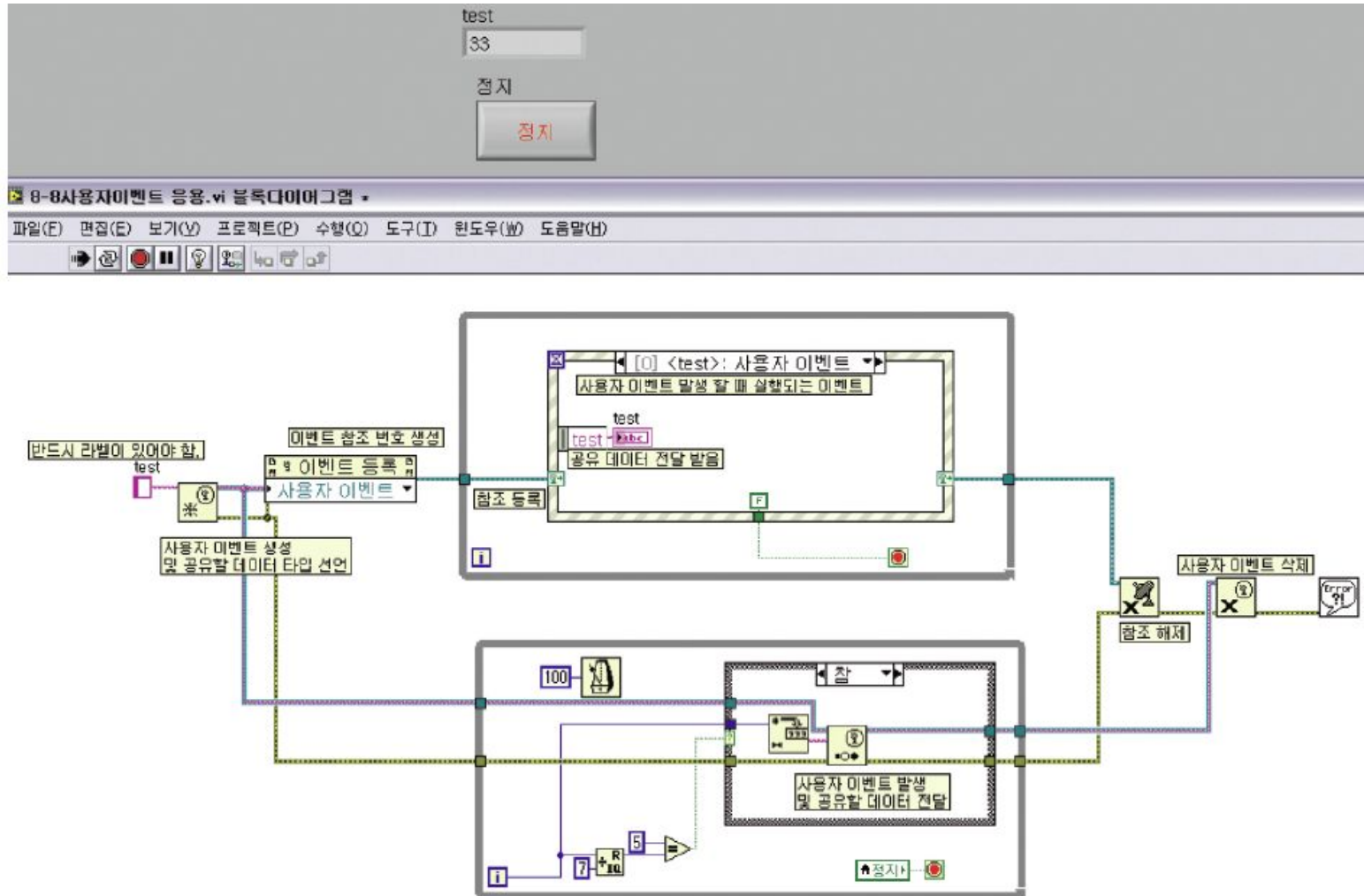


- 생산자 루프에 케이스 구조 대신 이벤트 구조가 들어감.

실습8-11-1)

생산/소비 이벤트 디자인 패턴

다이나믹 이벤트(=사용자 이벤트)



- 다이나믹 이벤트 : 사용자 정의 이벤트임.

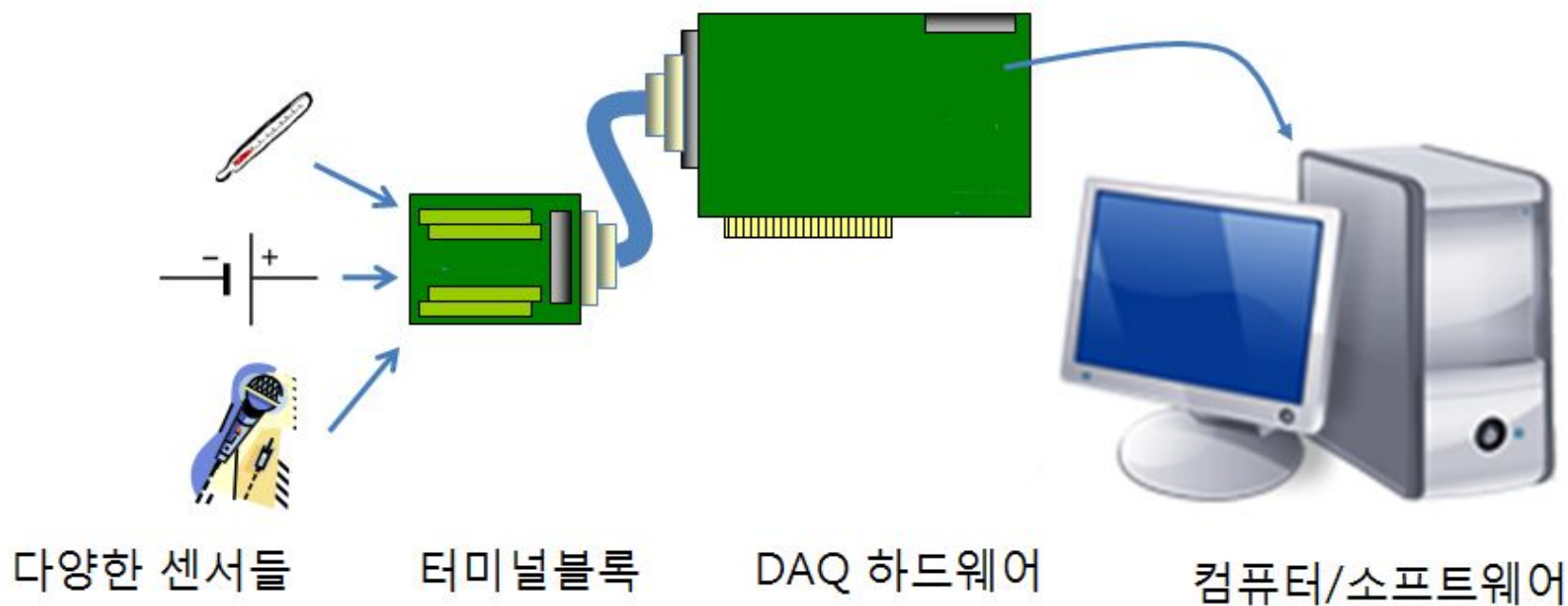
실습8-8-4) 다이나믹 이벤트 사용법

8장 요약

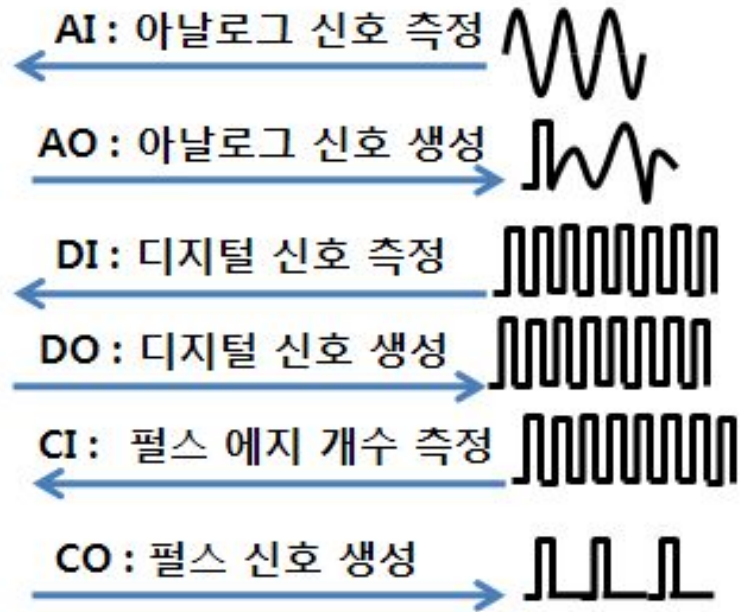
- 단일 루프 디자인 패턴 / 멀티 루프 디자인 패턴
- 멀티 루프간의 실시간 데이터 전송법
- 다양한 변수들 - 로컬, 글로벌, 기능적 글로벌, 공유
- 동기화 - 알림자, 큐

9. 데이터 수집

DAQ 시스템 구성

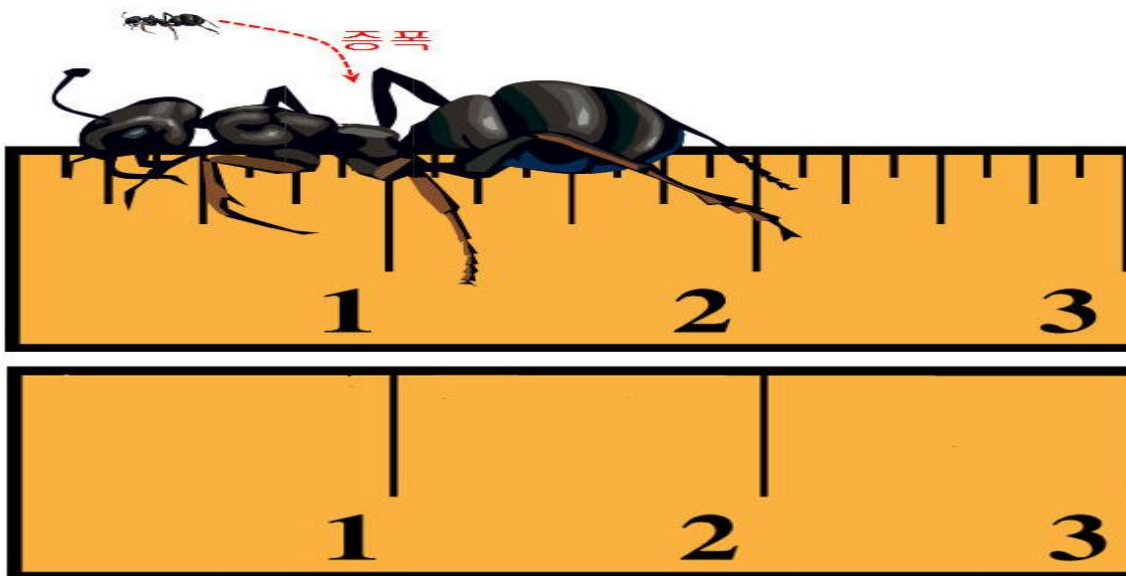


DAQ 기능



컴퓨터 기반의 측정 시스템
(DAQ 하드웨어가 장착된 상태)

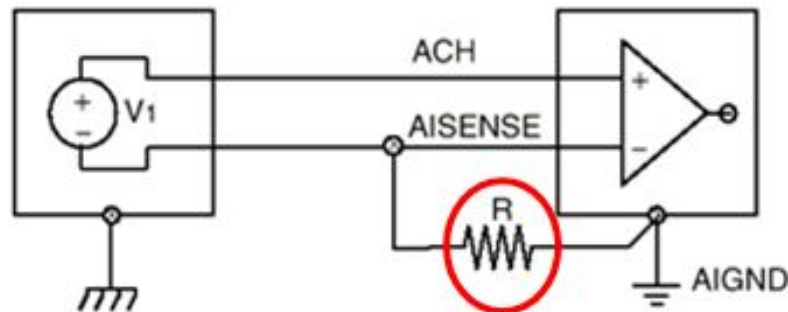
DAQ Spec



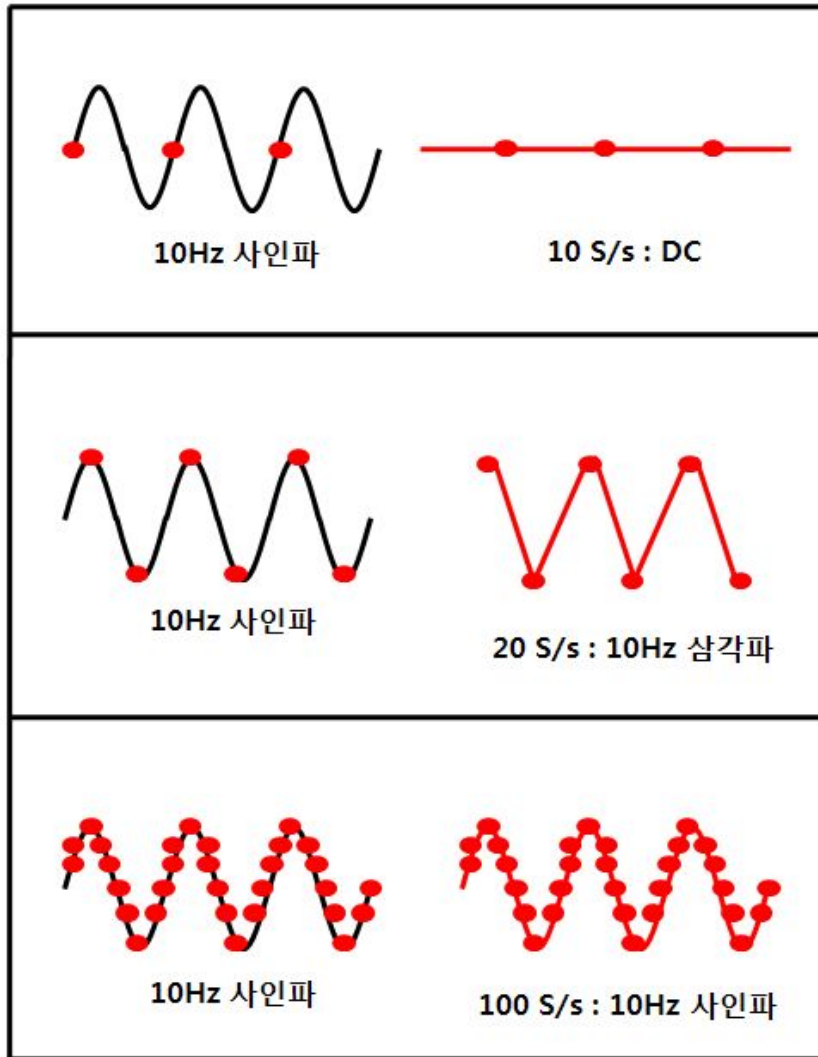
- 해상도 - '자'의 눈금 개수에 해당함.
- 입력 범위 - '자'가 잴 수 있는 길이에 해당함.
- 코드폭 - '자'의 눈금 최소 단위에 해당함.
- 증폭도 - 데이터를 증폭하는 정도.

DAQ Spec – 접지 모드

모드	접지 신호	유동 신호
<u>차동</u>	Best	Better(바이어스 저항 추가)
RSE	No Use	Best
NRSE	Good	Good(바이어스 저항 추가)

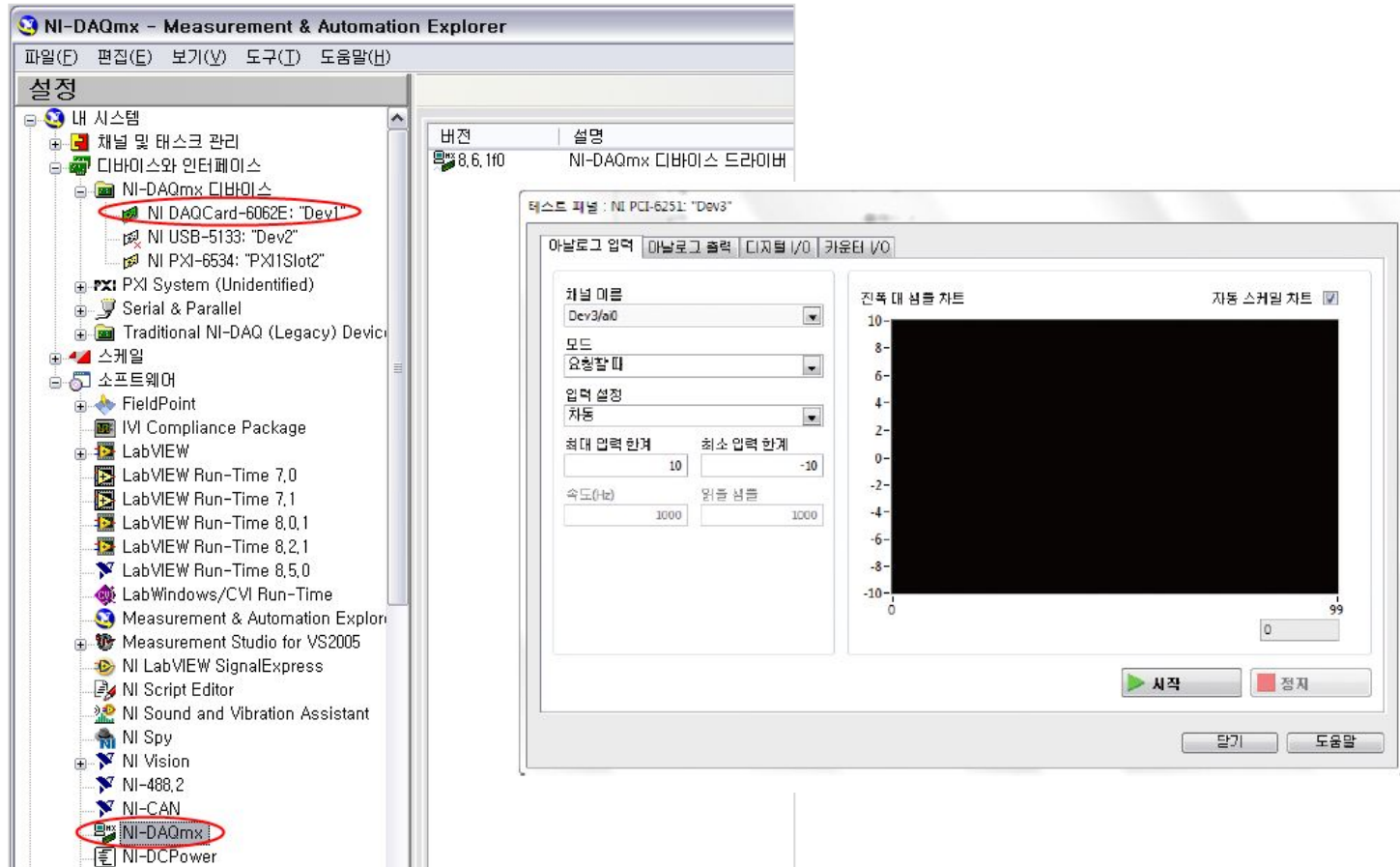


DAQ Spec – 샘플 속도



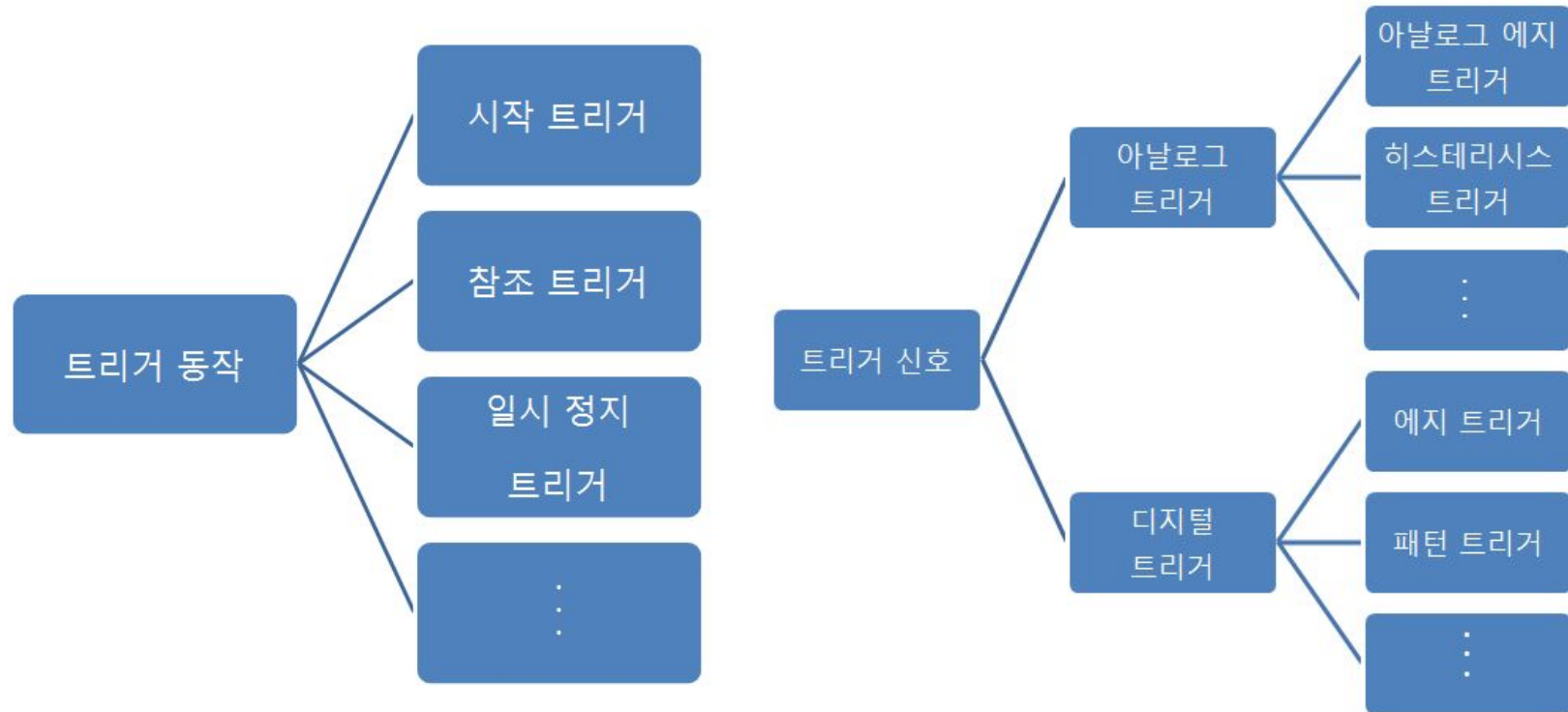
- 샘플 속도 - 아날로그를 디지털로 변환하는 속도를 의미함.
- 측정하고자 하는 신호 주파수의 **10~50배**가 적정.

MAX

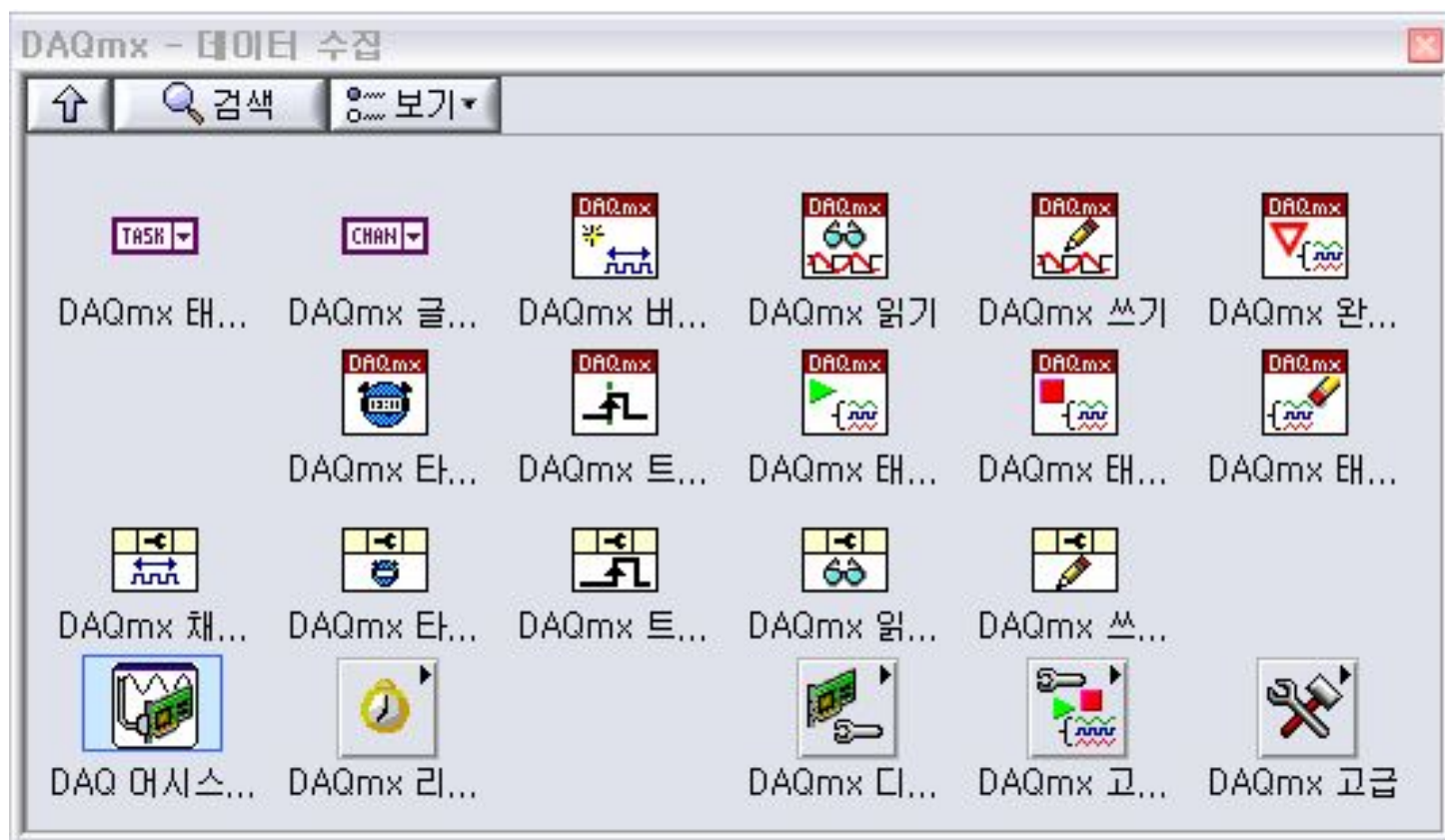


- NI 하드웨어를 설정하고 테스트 할 때 사용하는 유틸리티.

트리거



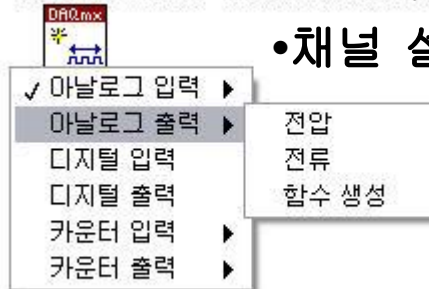
DAQmx 노드



DAQmx 노드 기능

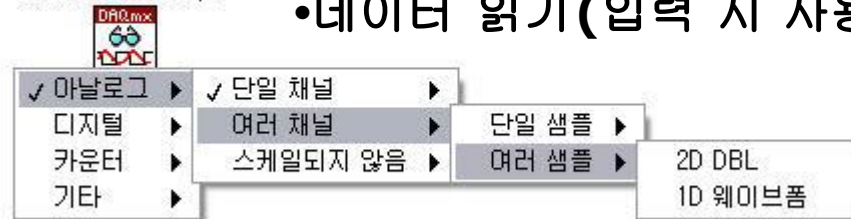
DAQmx Create Virtual Channel.vi

•채널 설정



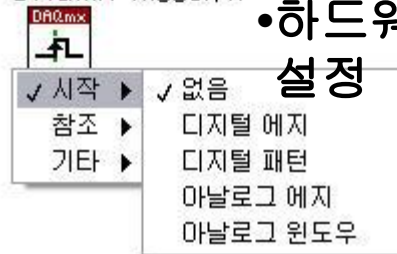
DAQmx Read.vi

•데이터 읽기(입력 시 사용)



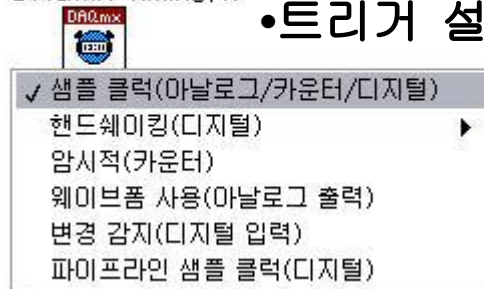
DAQmx Trigger.vi

•하드웨어 타이밍 설정



DAQmx Timing.vi

•트리거 설정



DAQmx Write.vi

•데이터 쓰기(출력 시 사용)



DAQmx 노드 기능

•작업 시작

DAQmx Start Task.vi



•대기 (유한 생성 /연속 생성)

DAQmx Wait Until Done.vi



DAQmx Is Task Done.vi



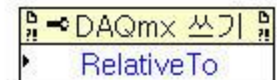
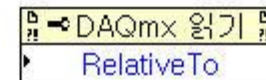
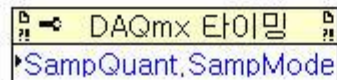
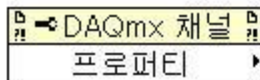
•작업 정지

DAQmx Stop Task.vi



•작업 삭제

DAQmx Clear Task.vi



•프로퍼티 노드들

아날로그 입력

물 떨어지는 속도 = 샘플 속도(S/s)
DAQmx Timing.vi의 '속도' 파라미터
에서 설정.

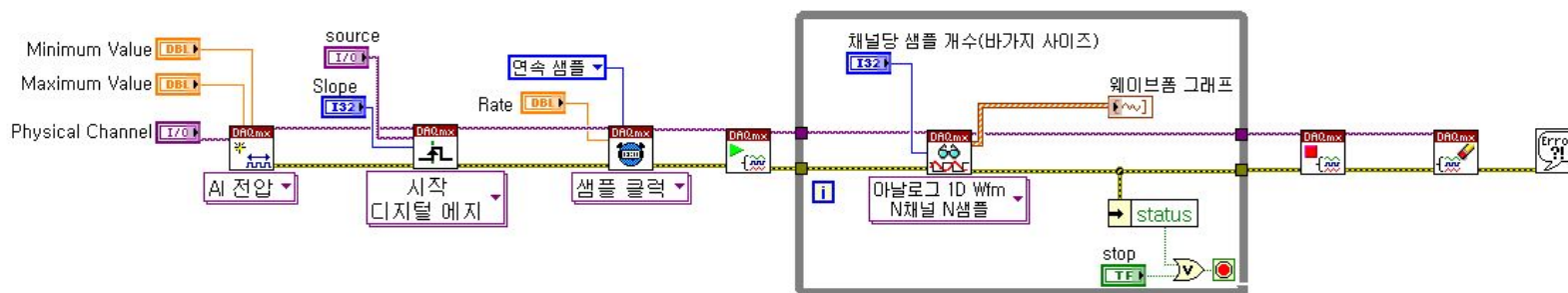
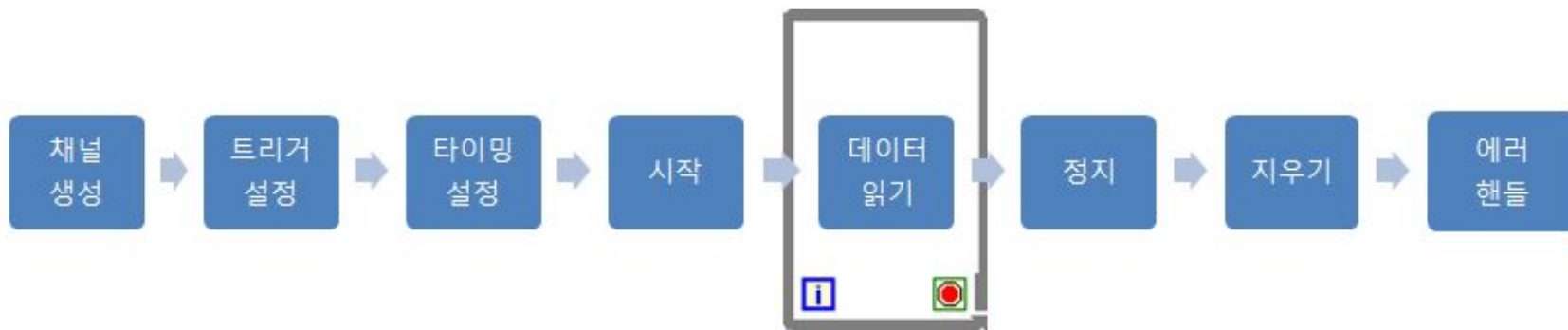
물 = 샘플



바가지 사이즈 = 어플리케이션 메모리
DAQmx Read.vi의 '채널당 샘플 개수'
파라미터에서 설정.

양동이 사이즈 = PC 버퍼
DAQmx Timing.vi의 '채널당 샘플' 파라미터
에서 설정.

시작 트리거 연속 아날로그 입력

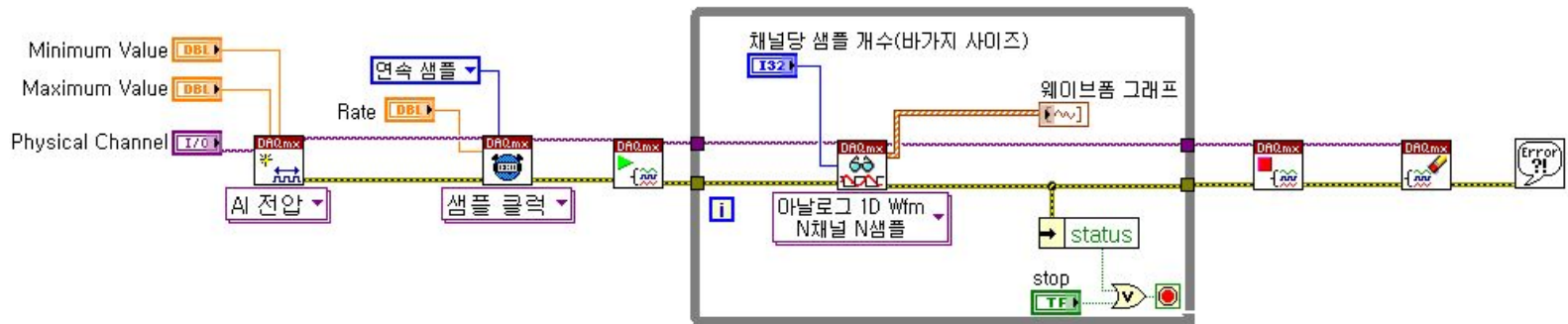
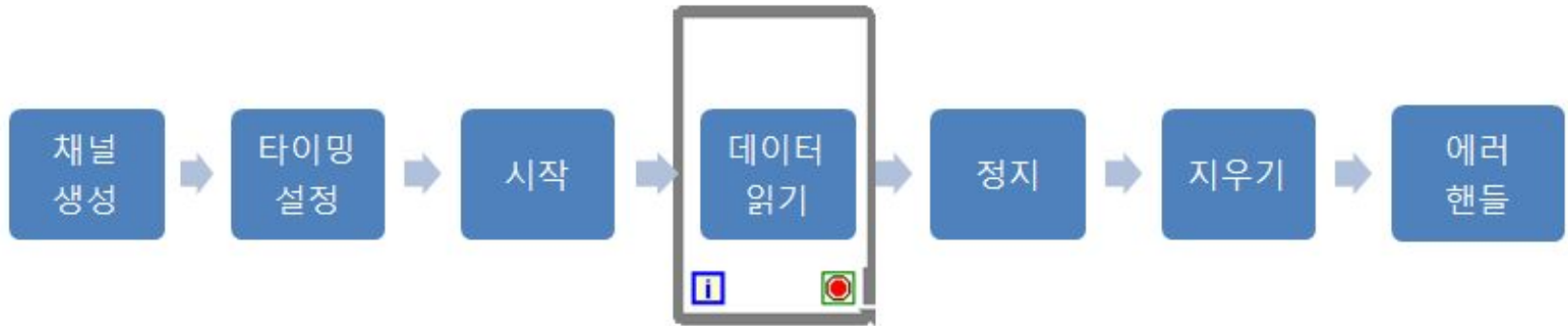


- 타이밍 노드를 사용하여 하드웨어 타이밍을 이용하고 시작을 알리는 디지털 신호가 감지되면 연속하여 데이터를 측정.

실습9-7-1)

시작 트리거 연속 아날로그 입력

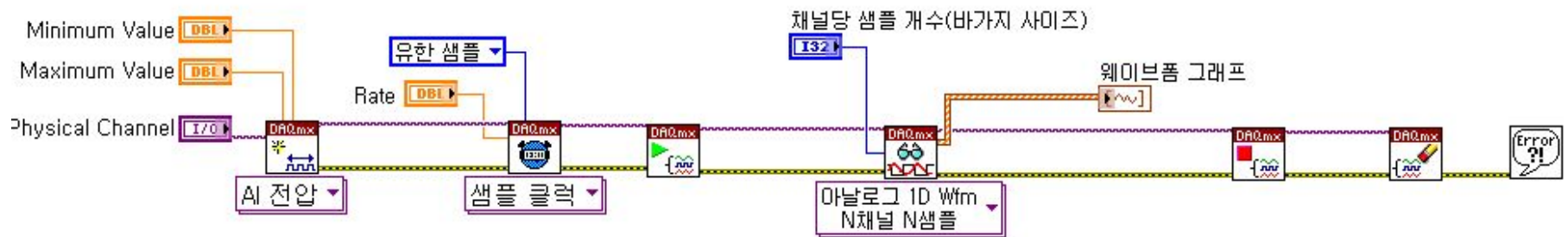
연속 아날로그 입력



- 타이밍 노드를 사용하여 하드웨어 타이밍을 이용하여 연속하여 데이터를 측정.

실습9-7-2) 연속 아날로그 입력

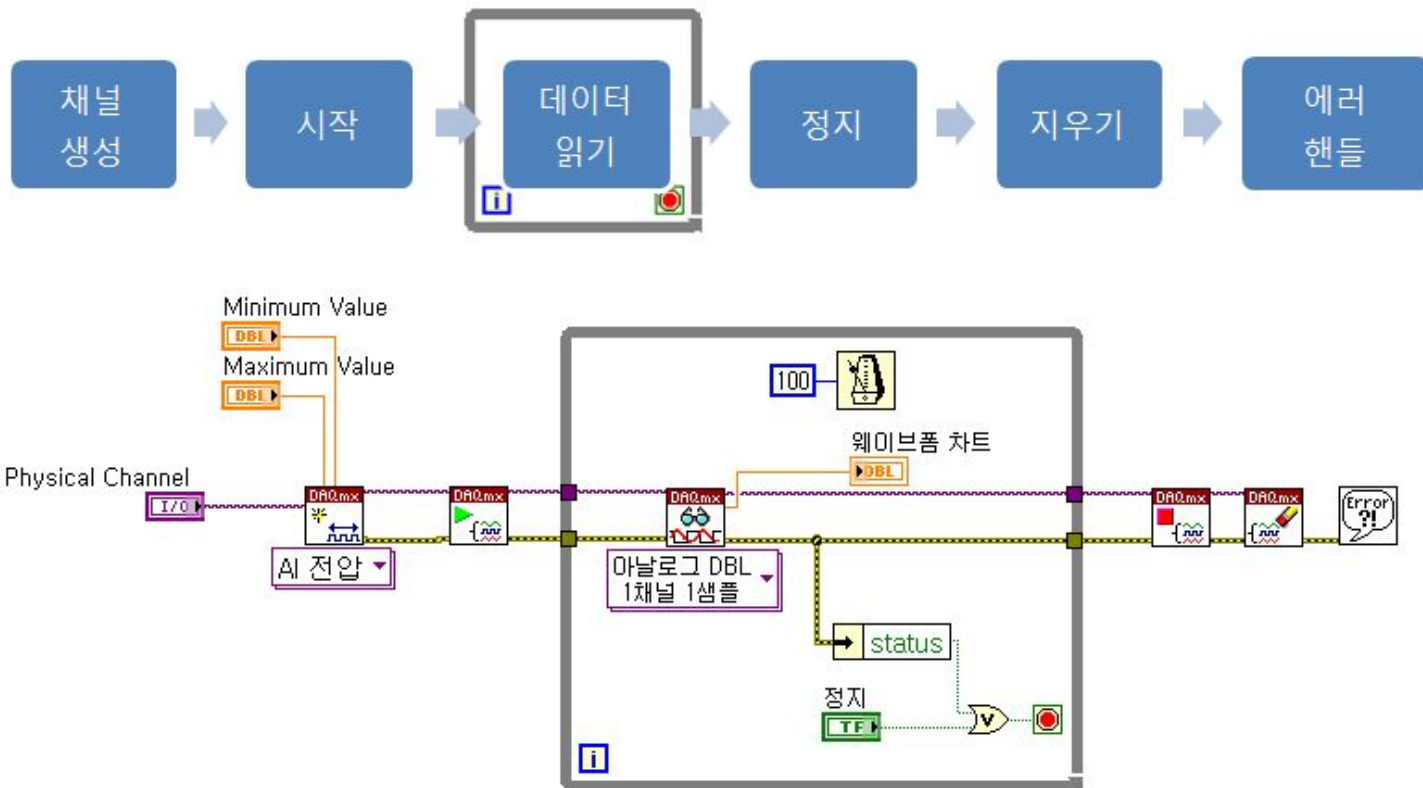
유한 아날로그 입력



- 타이밍 노드를 사용하여 하드웨어 타이밍을 이용하여 유한 개의 데이터를 측정.

실습9-7-3) 유한 아날로그 입력

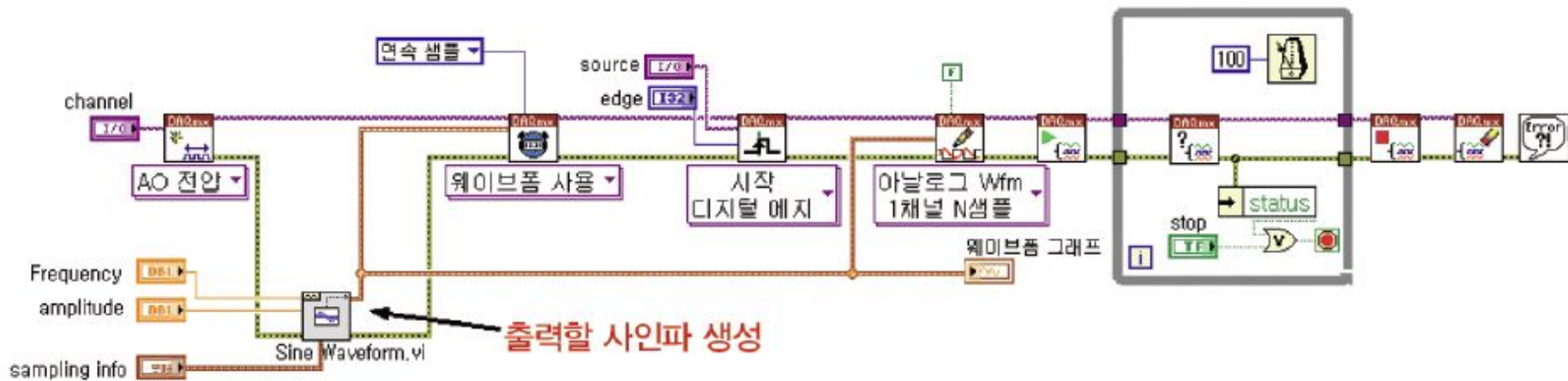
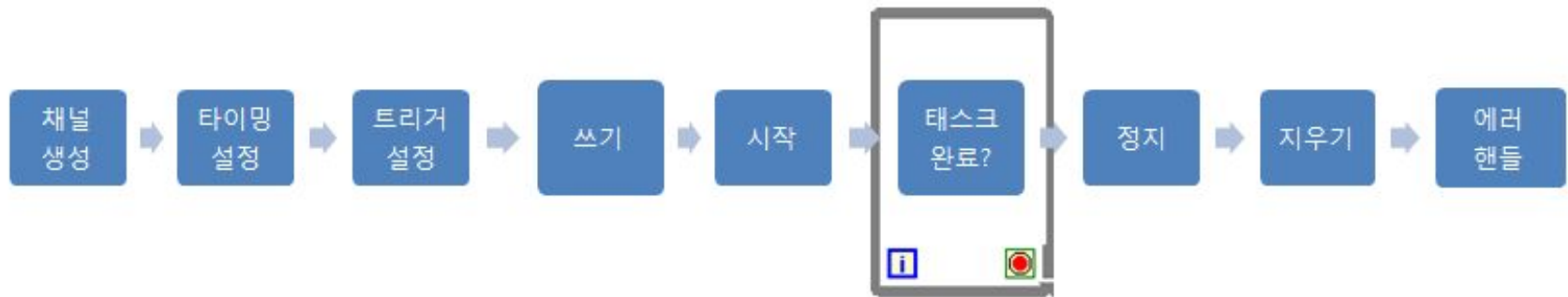
1샘플 아날로그 입력



- 소프트웨어 타이밍을 사용함으로써 **DC** 측정 시 주로 사용됨.

실습9-7-4) 1샘플 아날로그 입력

시작 트리거 연속 아날로그 출력

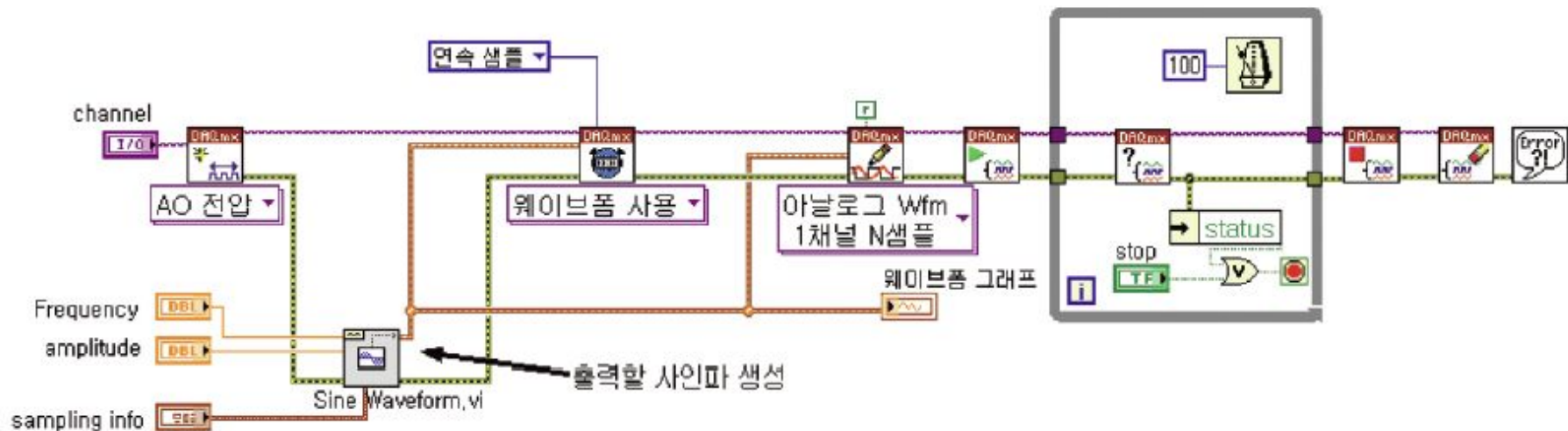
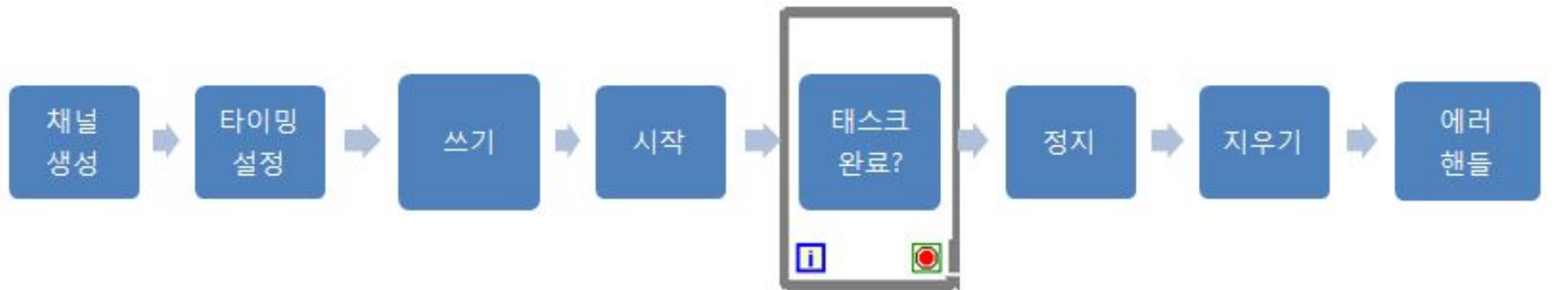


- 타이밍 노드를 사용하여 하드웨어 타이밍을 이용하고 시작을 알리는 디지털 신호가 감지되면 연속하여 데이터를 생성.

실습9-8-1)

시작 트리거 연속 아날로그 출력

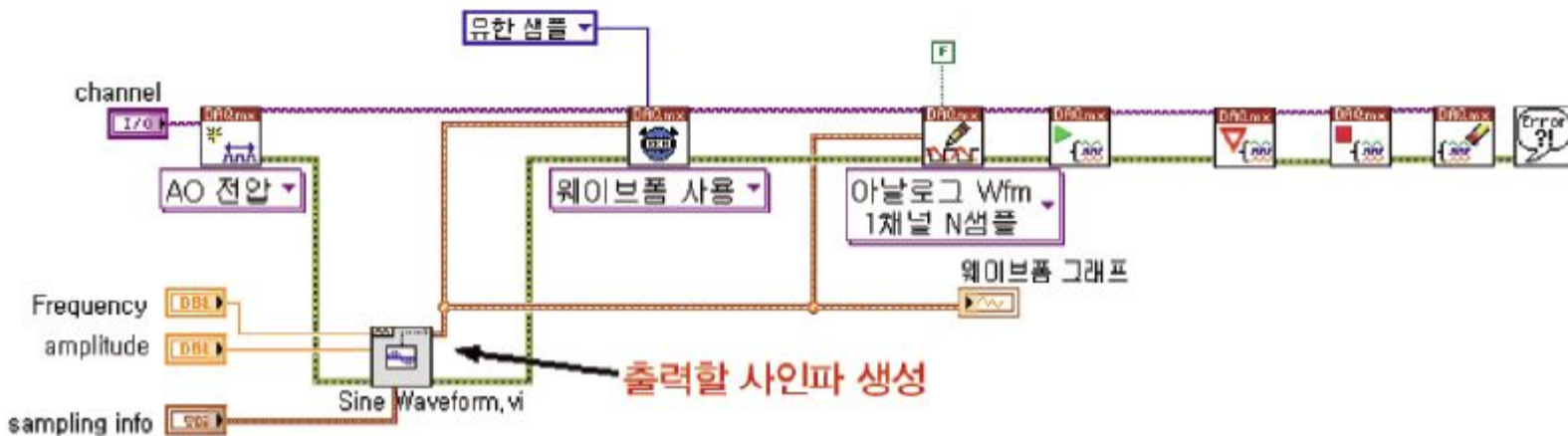
연속 아날로그 출력



- 타이밍 노드를 사용하여 하드웨어 타이밍을 이용하여 연속하여 데이터를 생성.

실습9-8-2) 연속 아날로그 출력

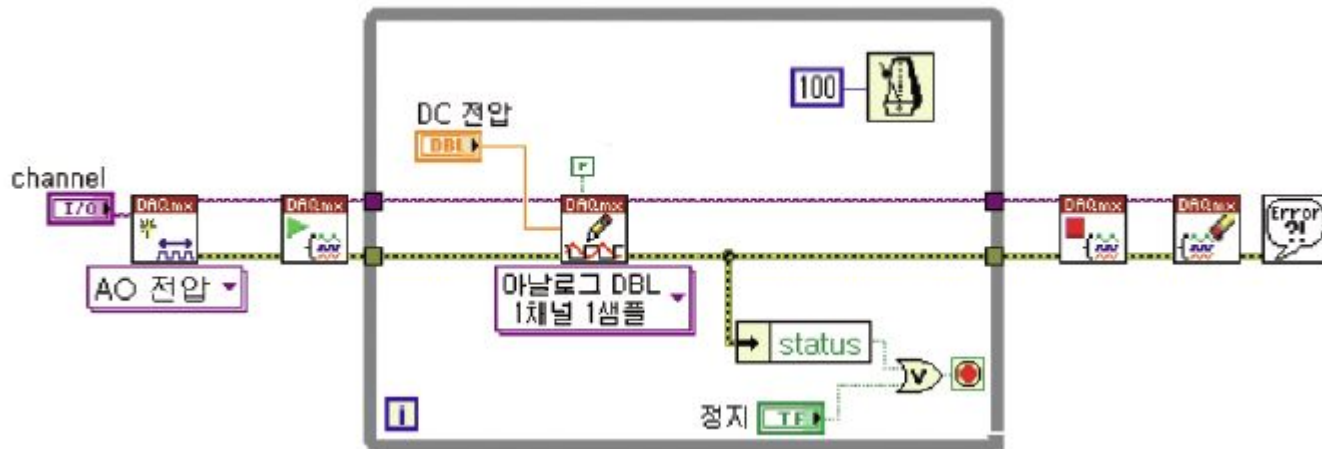
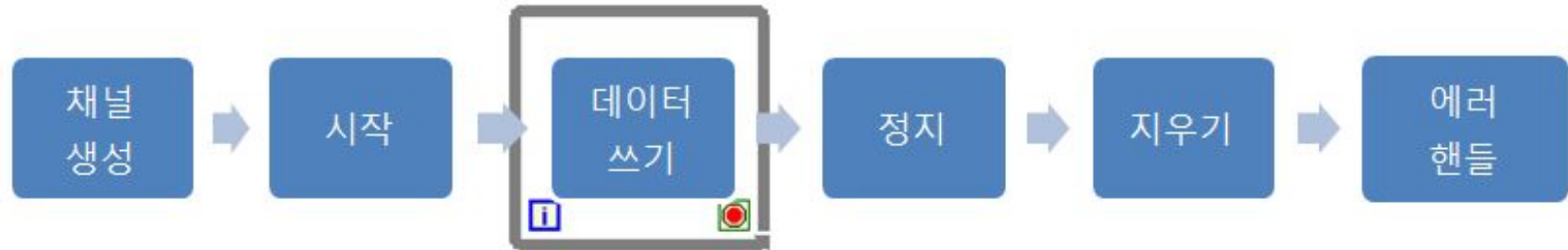
유한 아날로그 출력



- 타이밍 노드를 사용하여 하드웨어 타이밍을 이용하여 유한 개의 데이터를 생성.

실습9-8-3) 유한 아날로그 출력

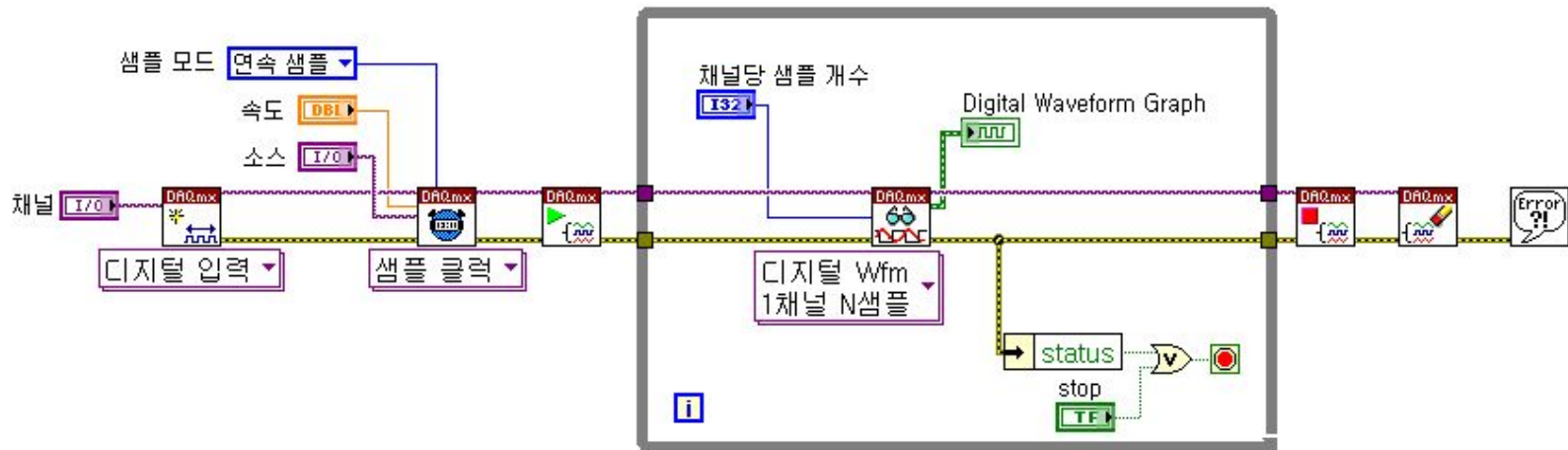
1샘플 아날로그 출력



- 소프트웨어 타이밍을 사용함으로써 **DC** 생성 시 주로 사용됨.

실습9-8-4) 1샘플 아날로그 출력

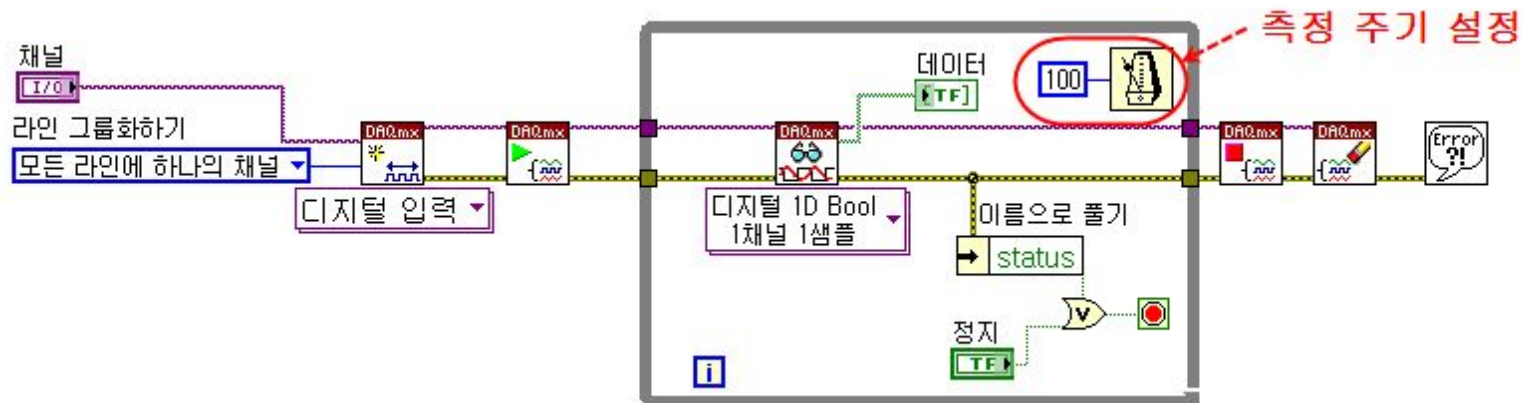
연속 디지털 입력



- 타이밍 노드를 사용하여 하드웨어 타이밍을 이용하여 연속하여 디지털 데이터를 측정. 주로 디지털 패턴 측정 할 때 사용.

실습9-9-1) 연속 디지털 입력

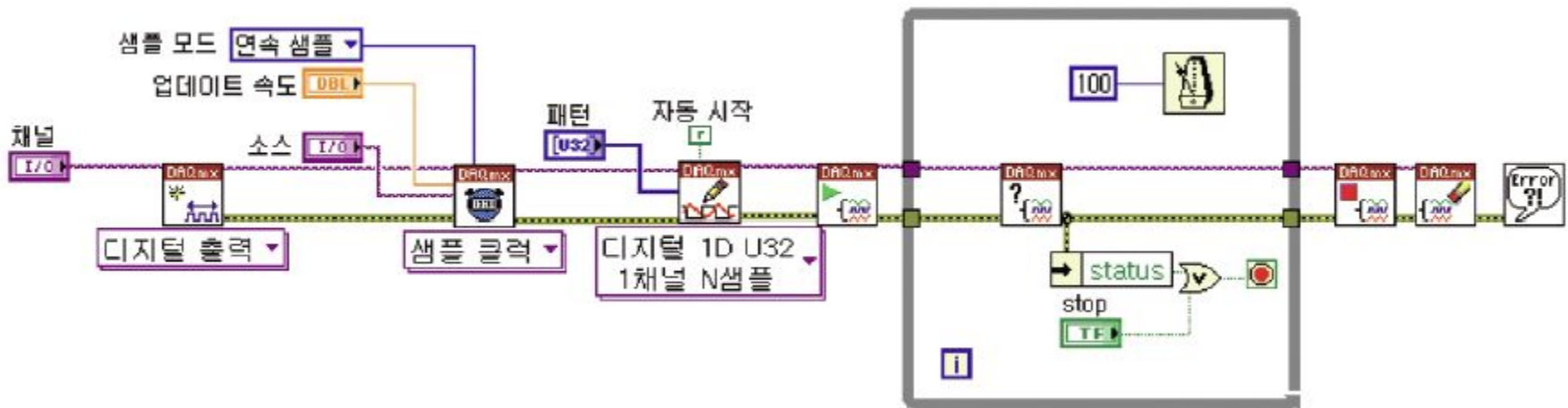
디지털 입력



- 소프트웨어 타이밍을 사용하여 디지털 신호를 측정함.

실습9-9-2) 디지털 입력

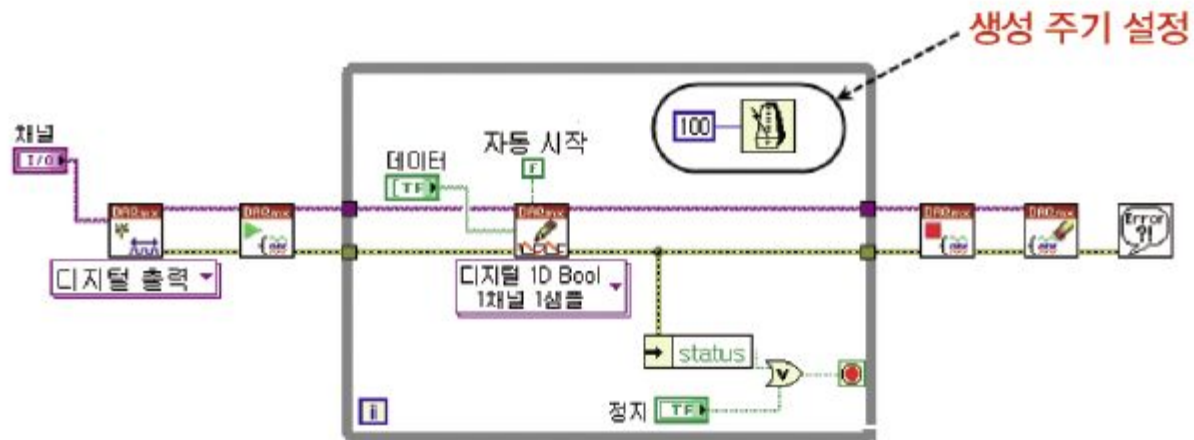
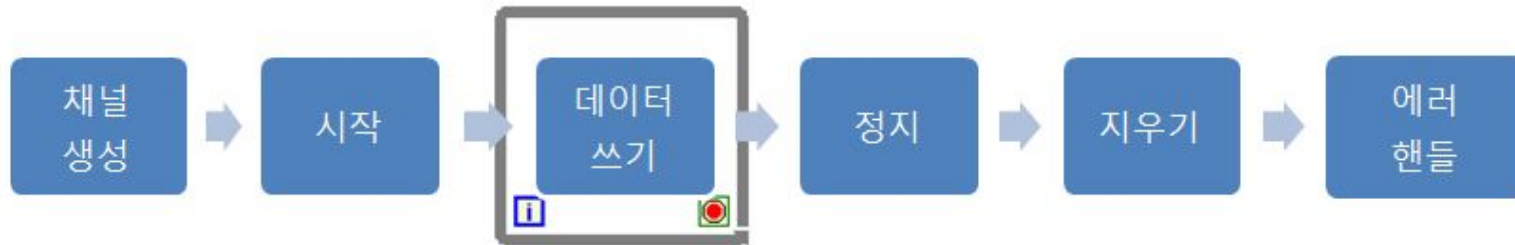
연속 디지털 출력



- 타이밍 노드를 사용하여 하드웨어 타이밍을 이용하여 연속하여 디지털 데이터를 생성. 주로 디지털 패턴 생성 할 때 사용.

실습9-9-3) 연속 디지털 출력

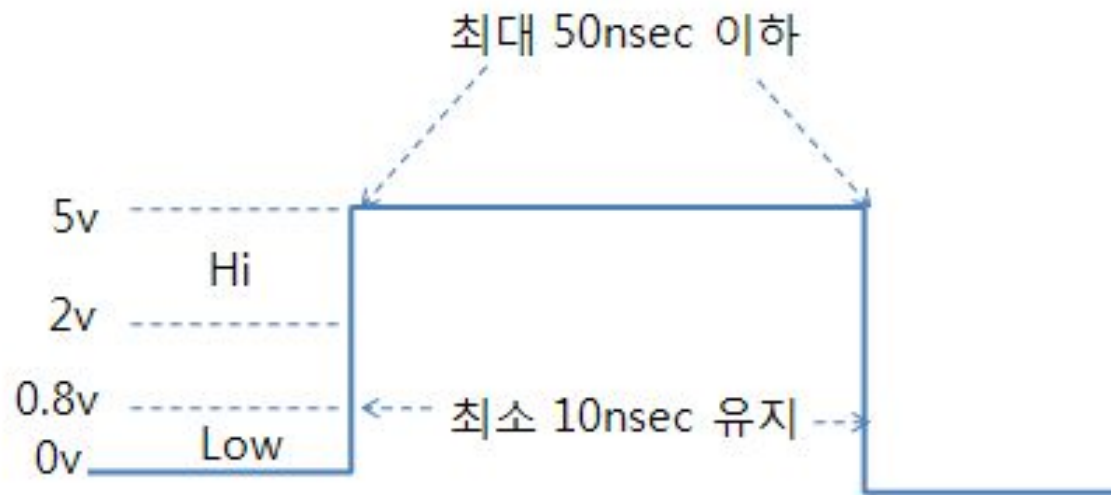
디지털 출력



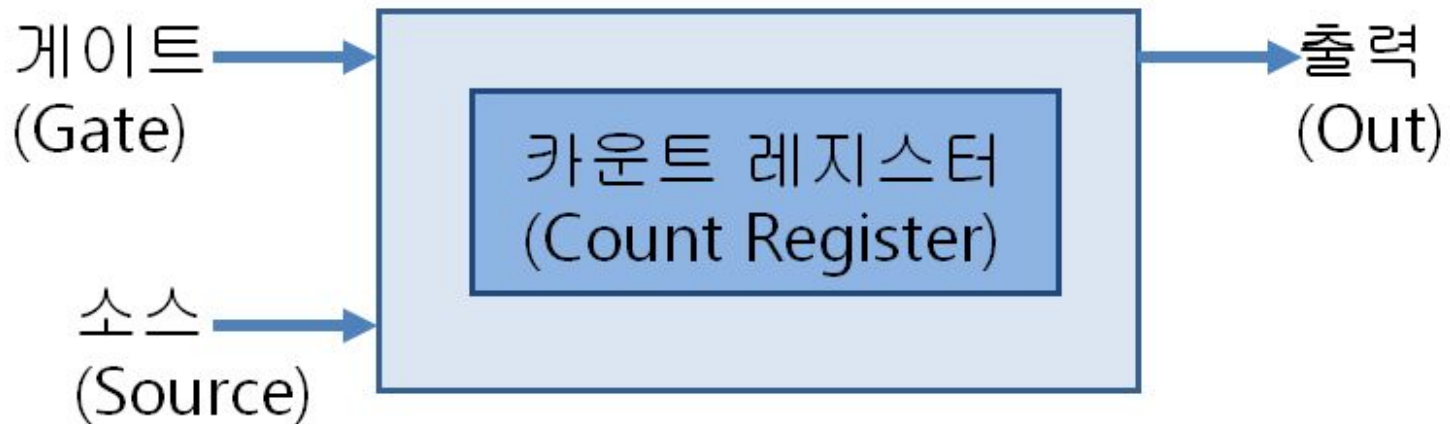
- 소프트웨어 타이밍을 사용하여 디지털 신호를 생성.

실습9-9-4) 디지털 출력

TTL 신호 정의

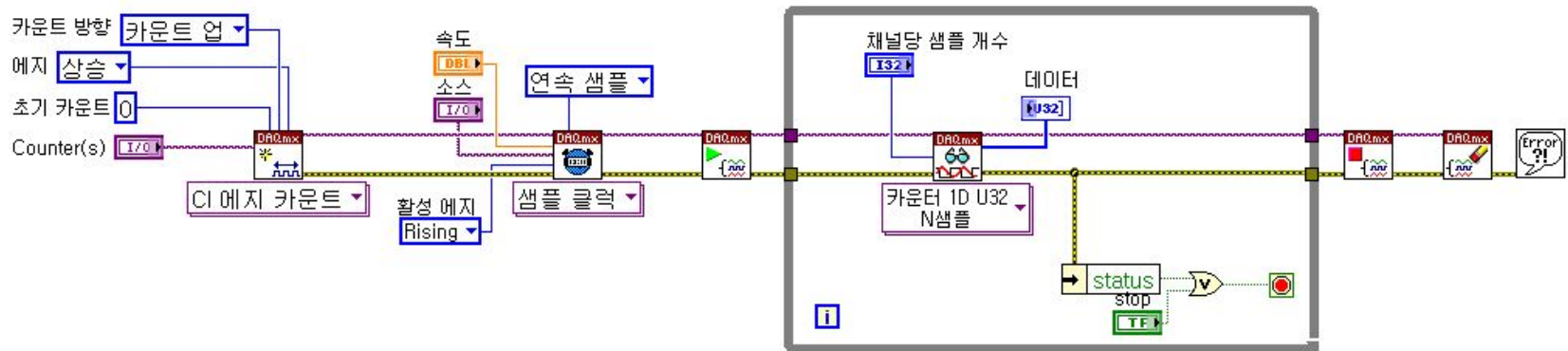


카운터 구조 및 기능



- 펄스 에지 카운팅
- 시간 측정
- 펄스 주기 / 주파수 / 펄스폭 측정
- 엔코더를 이용한 위치 측정
- 펄스 트레인 생성

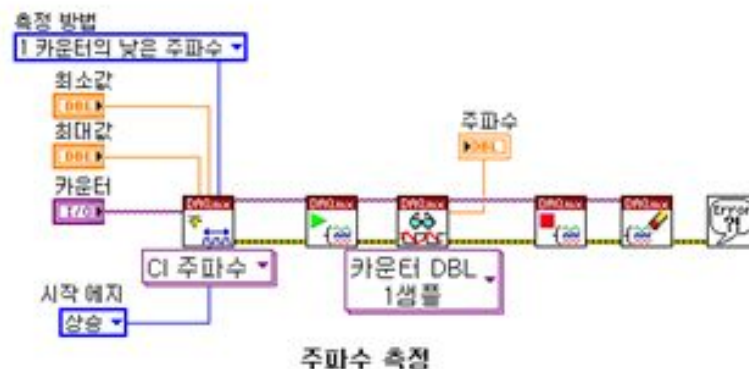
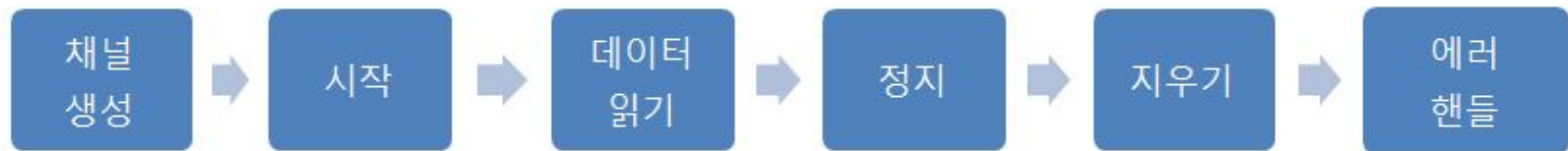
연속 에지 카운팅



- 타이밍 노드를 사용하여 하드웨어 타이밍을 이용하여 연속하여 디지털 에지 개수를 측정.

실습9-11-1) 연속 에지 카운팅

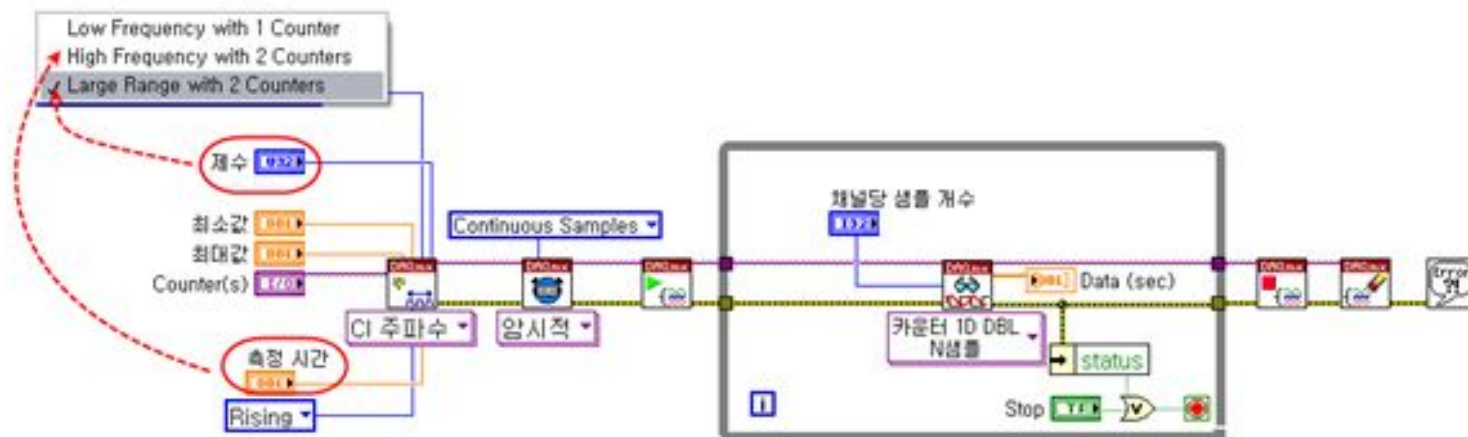
펄스폭/주기/주파수 단일 측정



실습9-11-3)

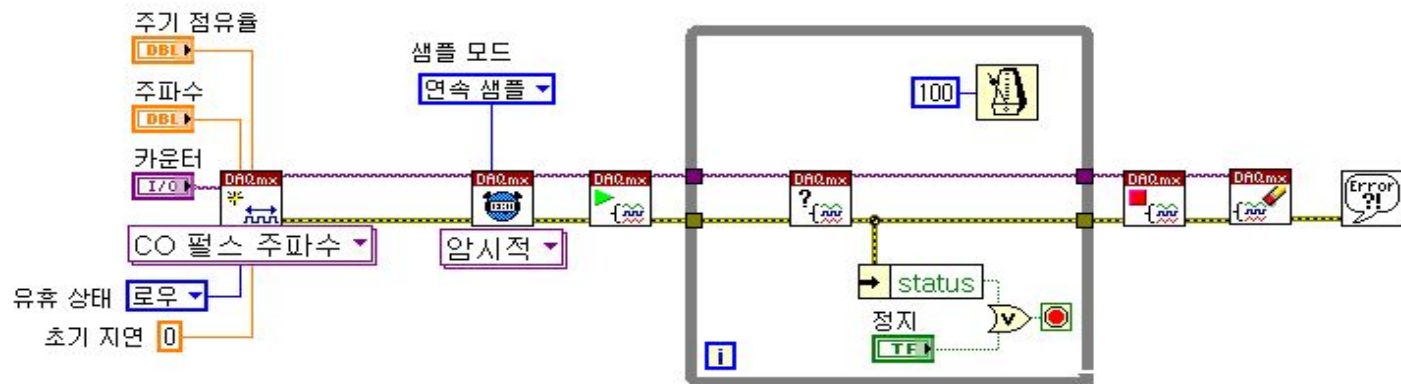
펄스폭/주기/주파수 단일 측정

주파수 연속 측정



실습9-11-4) 주파수 연속 측정

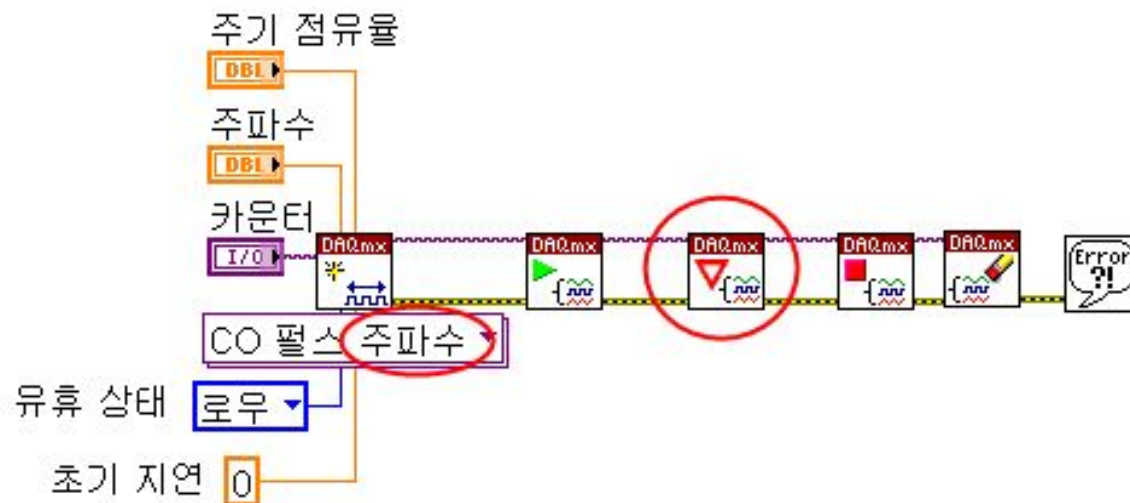
펄스 트레인 연속 생성



•펄스 트레인 연속 생성.

실습9-12-1) 펄스 트레인 연속 생성

단일 펄스 생성



•펄스 한 개 생성.

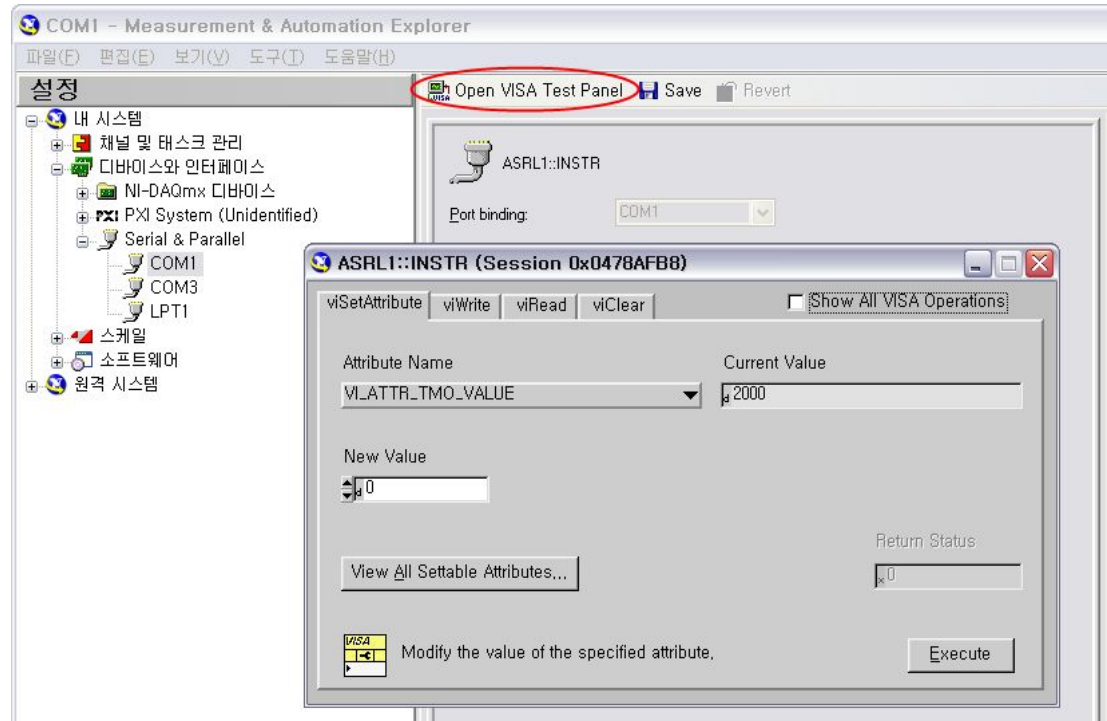
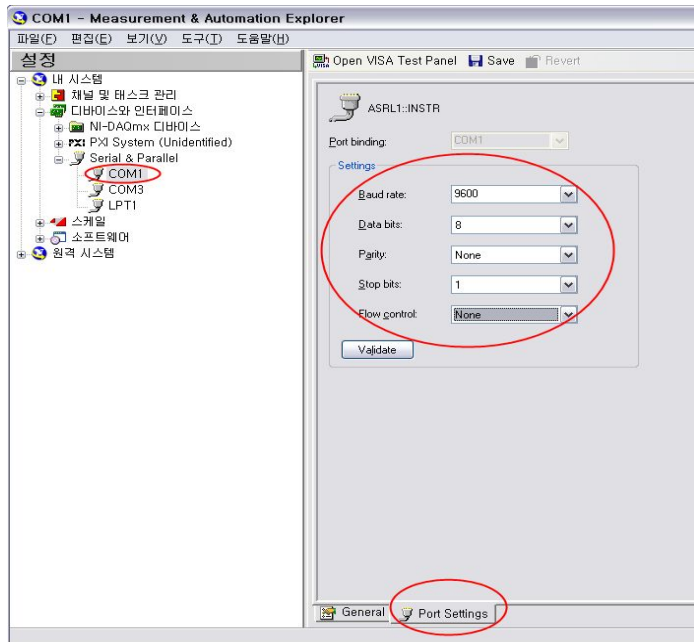
실습9-12-2) 단일 펄스 생성

9장 요약

- 아날로그 입력
- 아날로그 출력
- 디지털 입력
- 디지털 출력
- 카운터 입력
- 카운터 출력

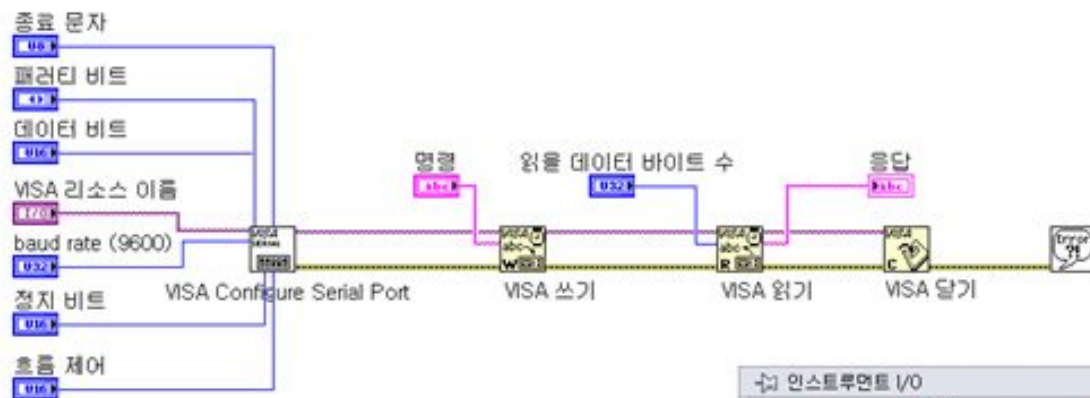
10. 통신

MAX 설정 및 테스트 - 시리얼 통신



- MAX를 사용하여 시리얼 통신 설정 및 테스트 작업 수행

프로그램 - 시리얼

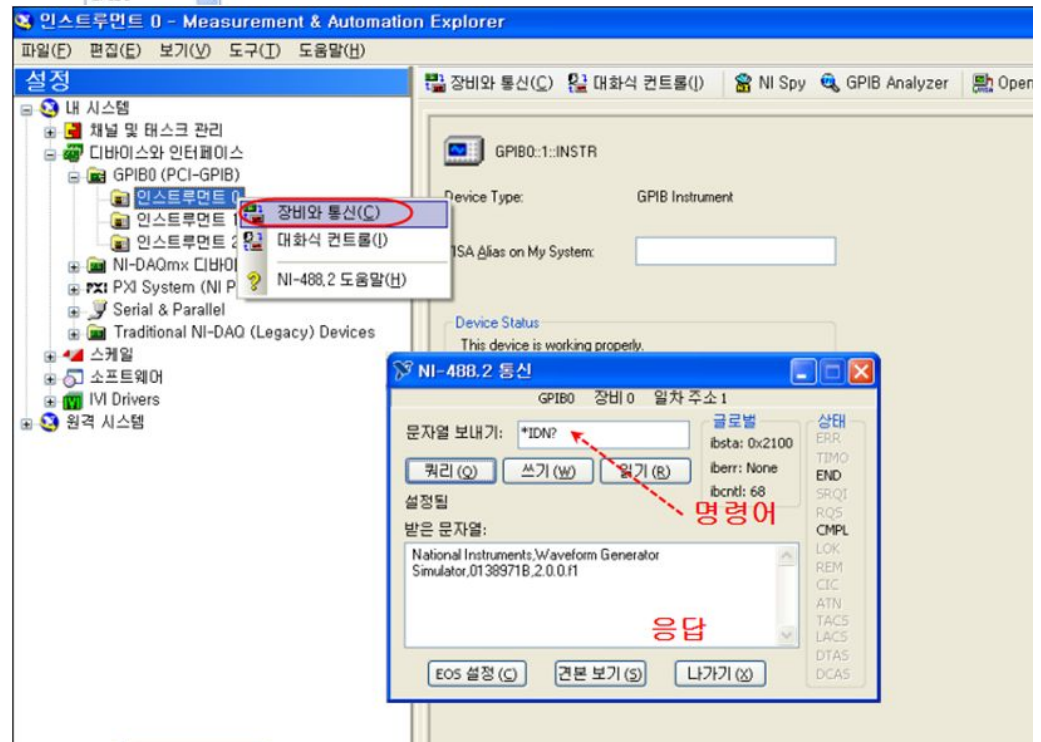
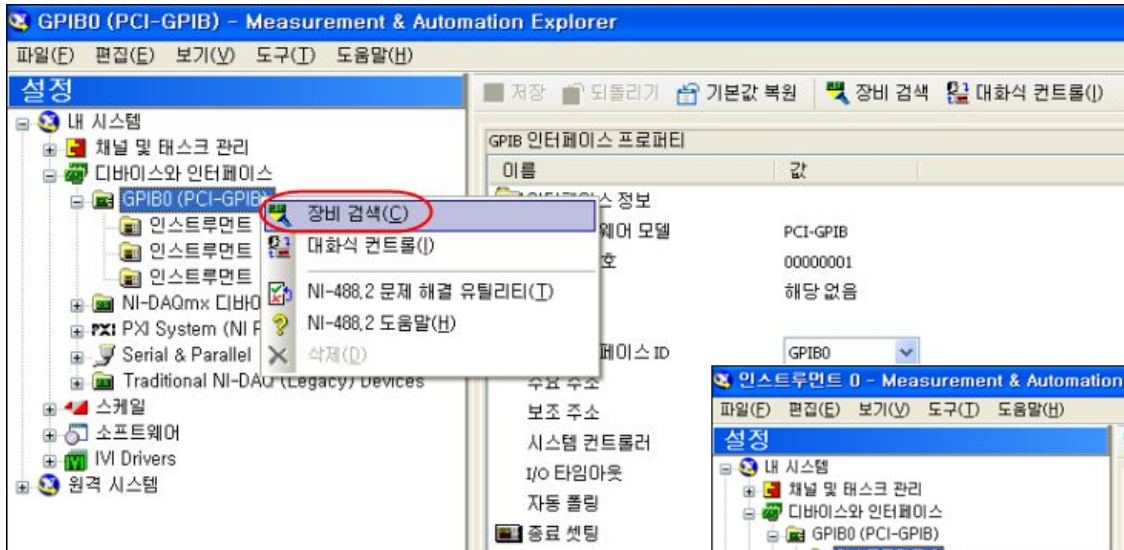


GPIB 구성도

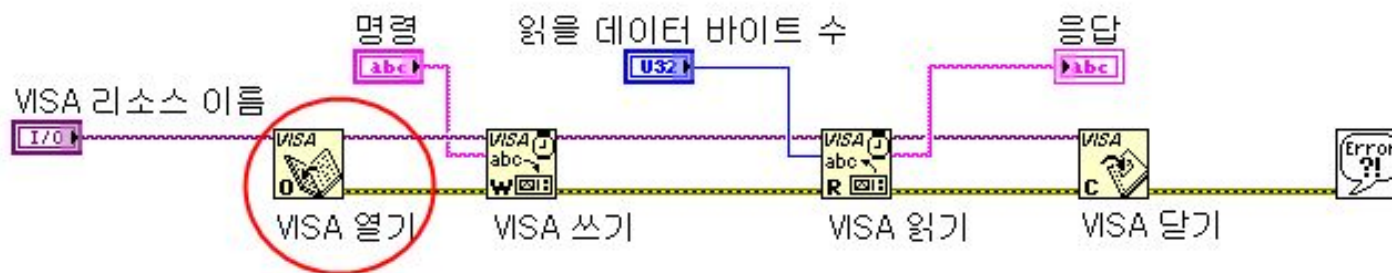


•**GPIB**는 계측기와 **PC** 간의 통신 중 가장 널리 쓰이는 통신 방법 중 하나임.

MAX 설정 및 테스트 -GPIB

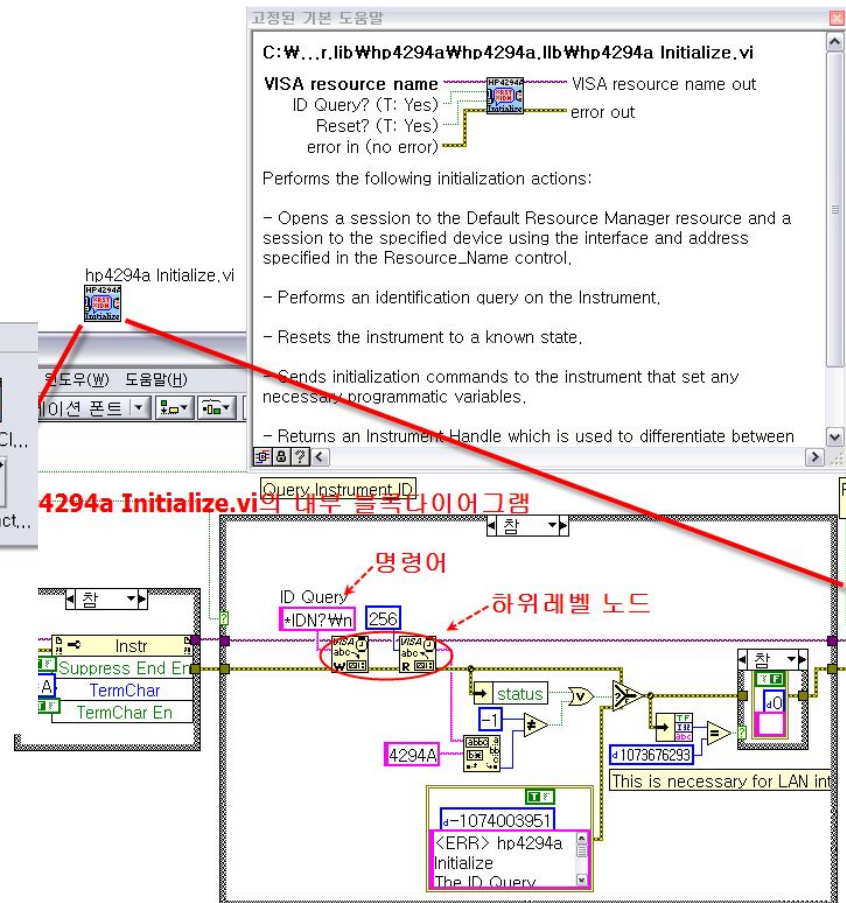
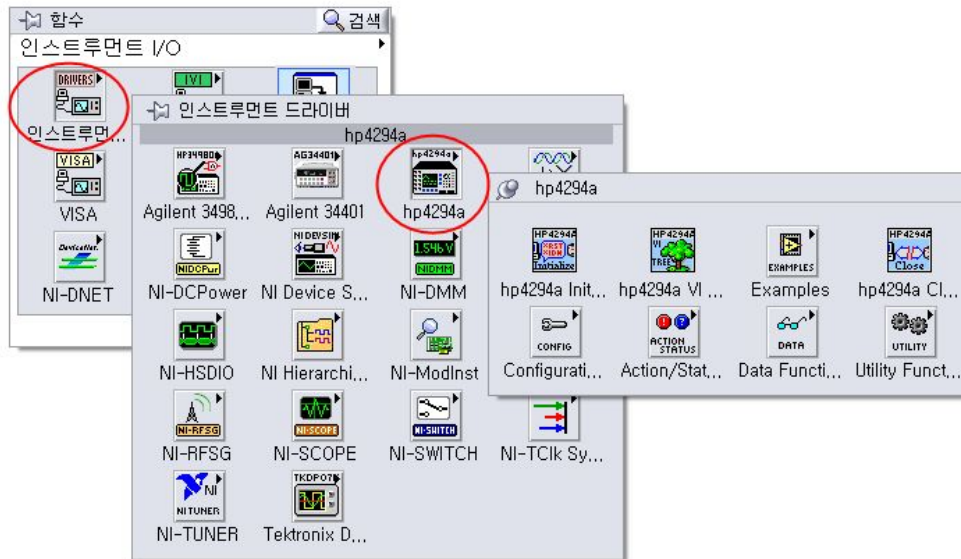


프로그램 - GPIB



인스트루먼트 드라이버

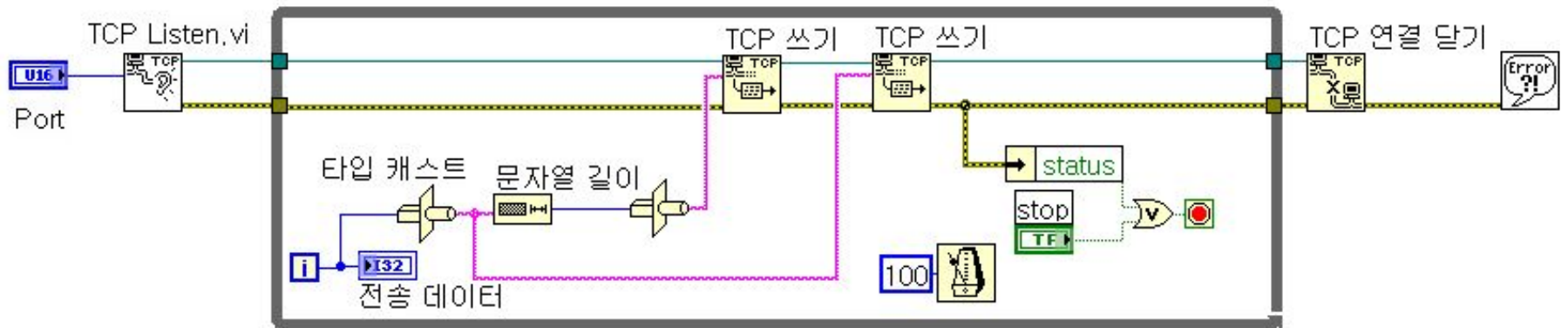
- 사용자가 명령어를 몰라도 손쉽게 사용할 수 있도록 장비 업체가 **VISA**노드와 명령어를 조합하여 만들어 놓음.



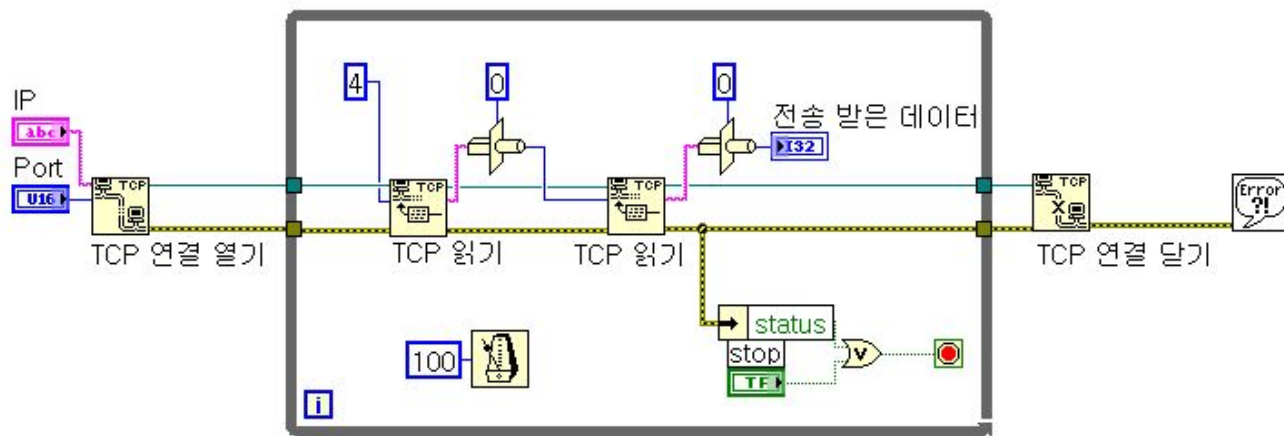
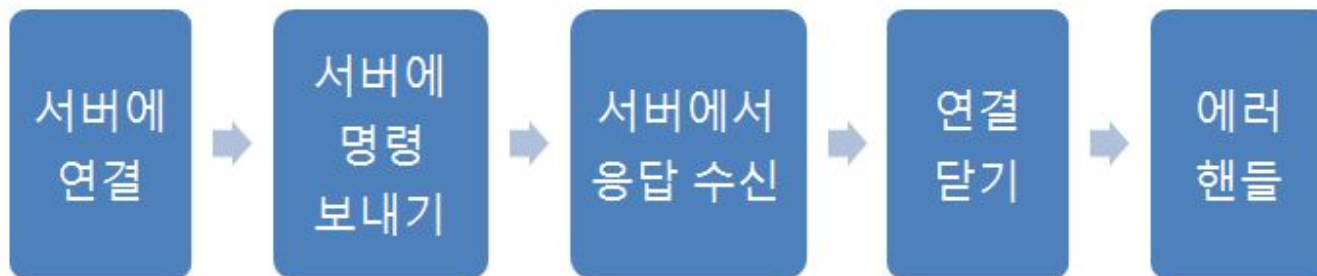
TCP/IP



서버 프로그램



클라이언트 프로그램

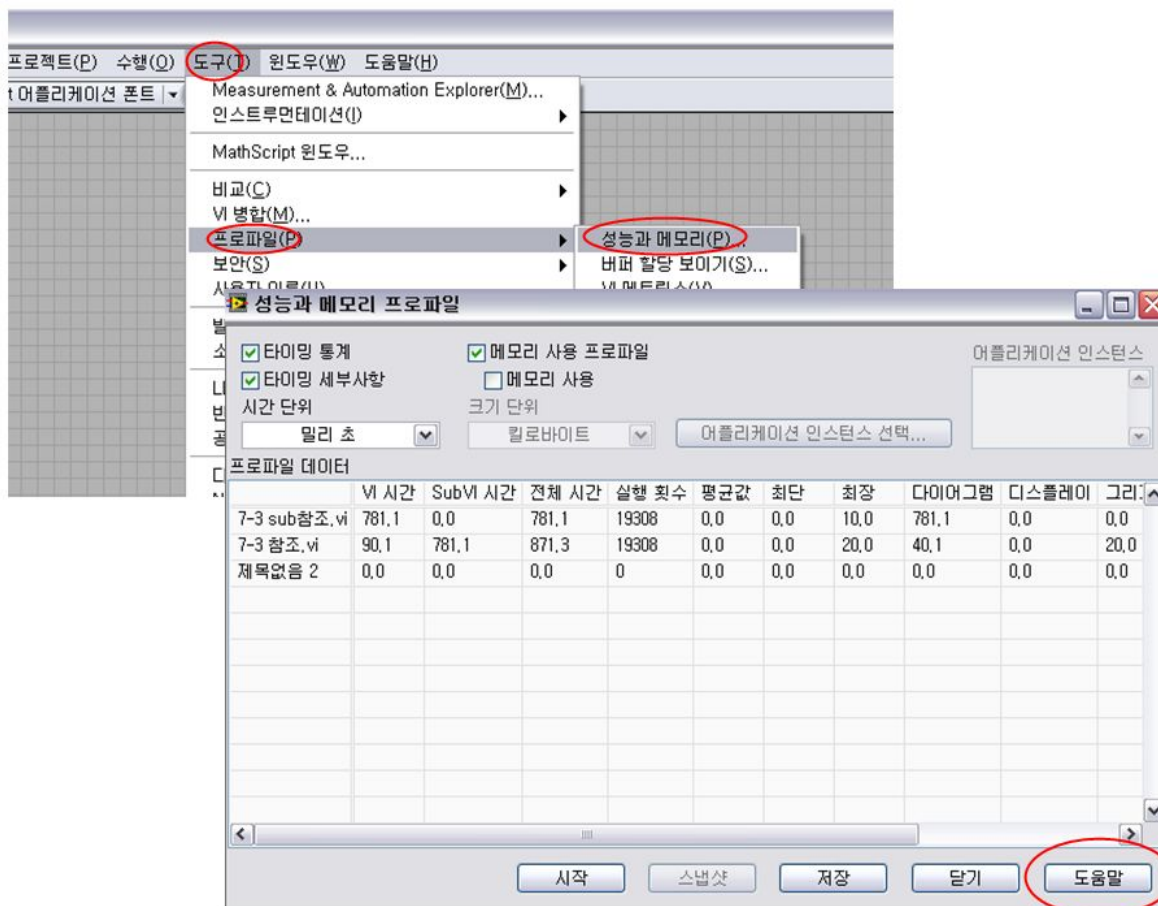


10장 요약

- 시리얼 통신
- **GPIB** 통신
- **TCP/IP** 통신

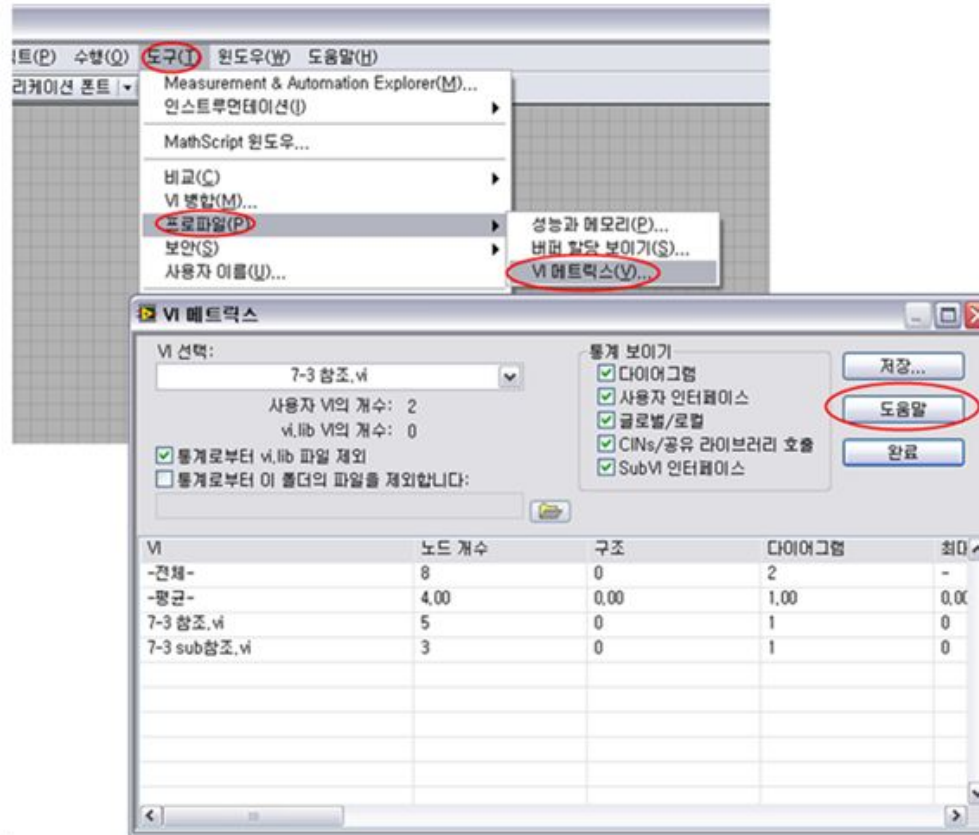
11. 어플리케이션 완성

성능과 메모리



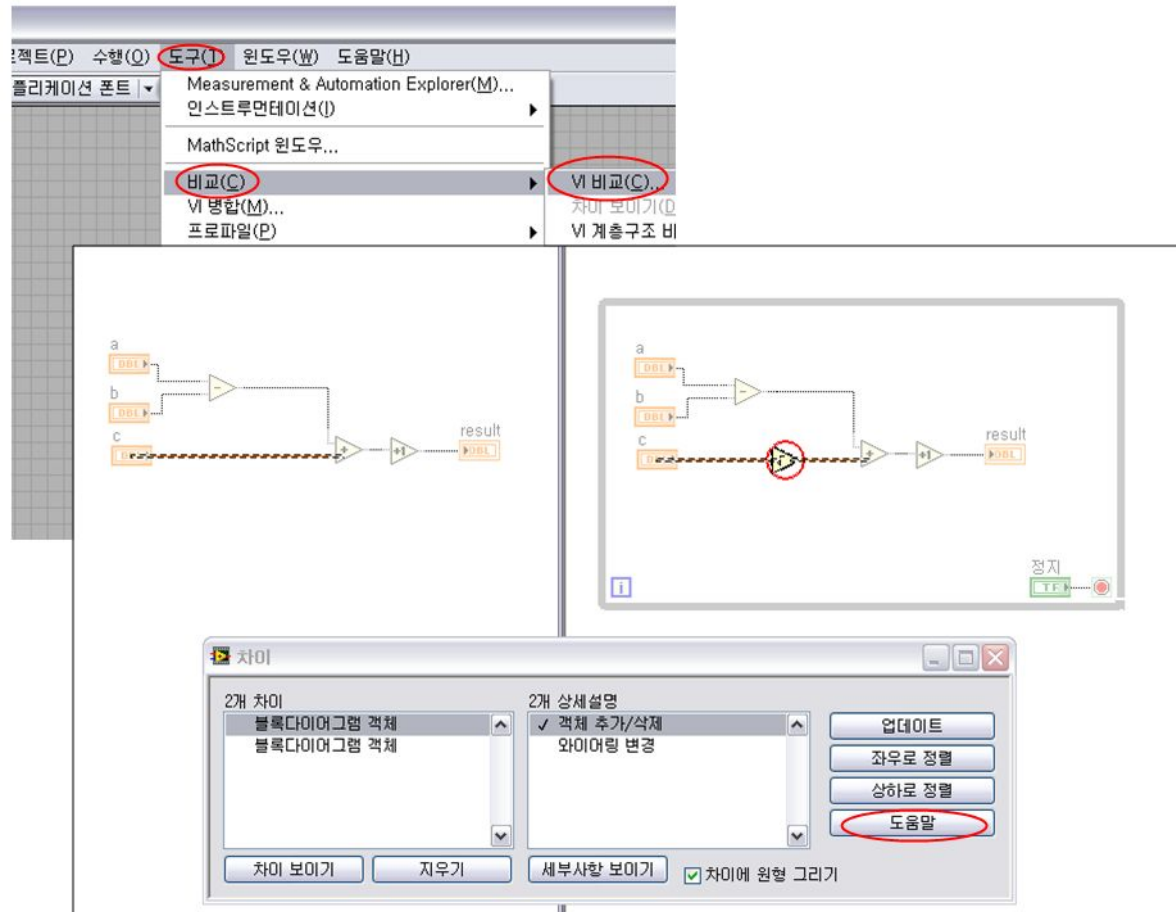
- VI 실행 할 때 소요되는 시간과 차지하는 메모리 모니터링

VI 메트릭스



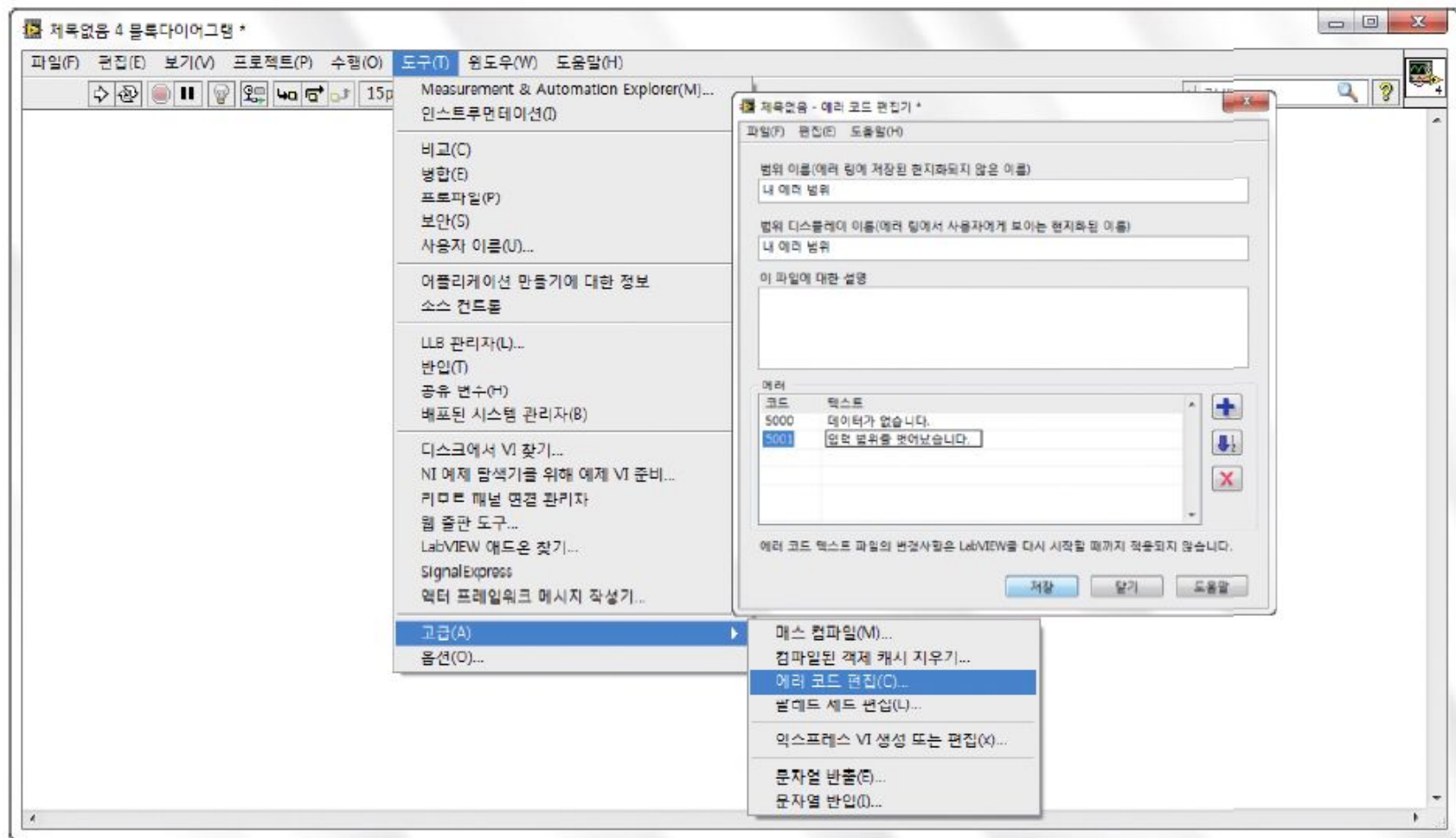
- 프로그램 시 사용한 노드 수 등을 모니터링

VI 비교



•두 VI의 차이점을 알려줌

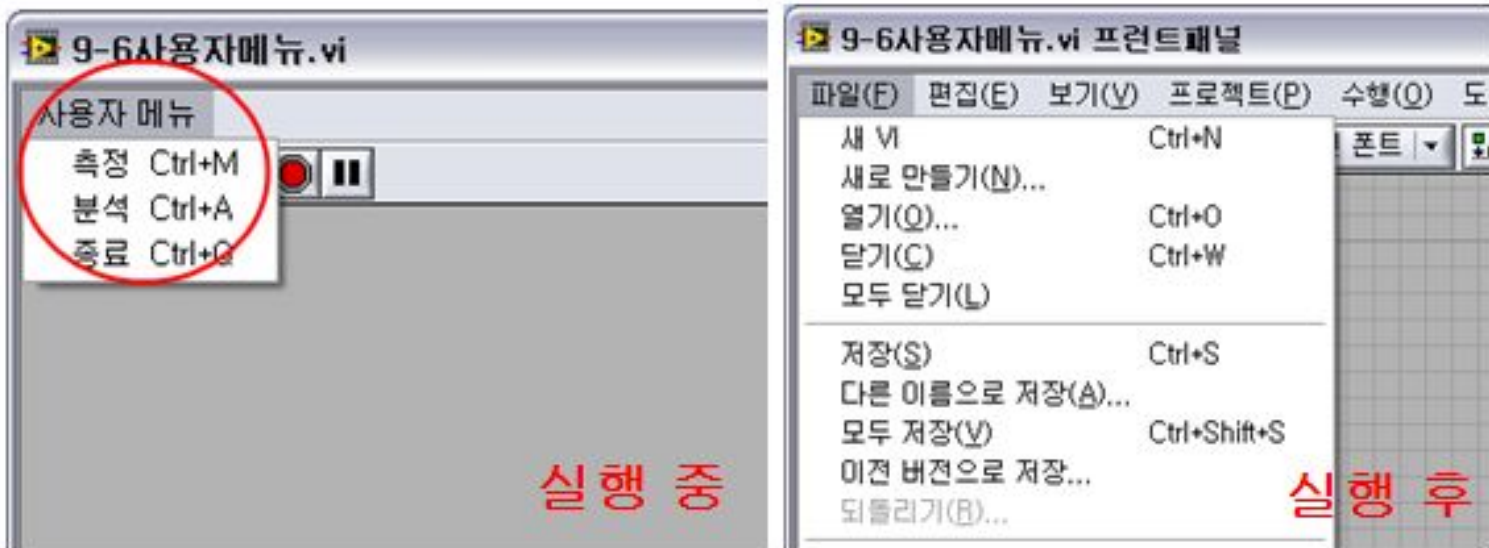
사용자 에러 정의



•사용자 에러 정의가 가능함.

실습11-4-1) 사용자 에러 정의 방법

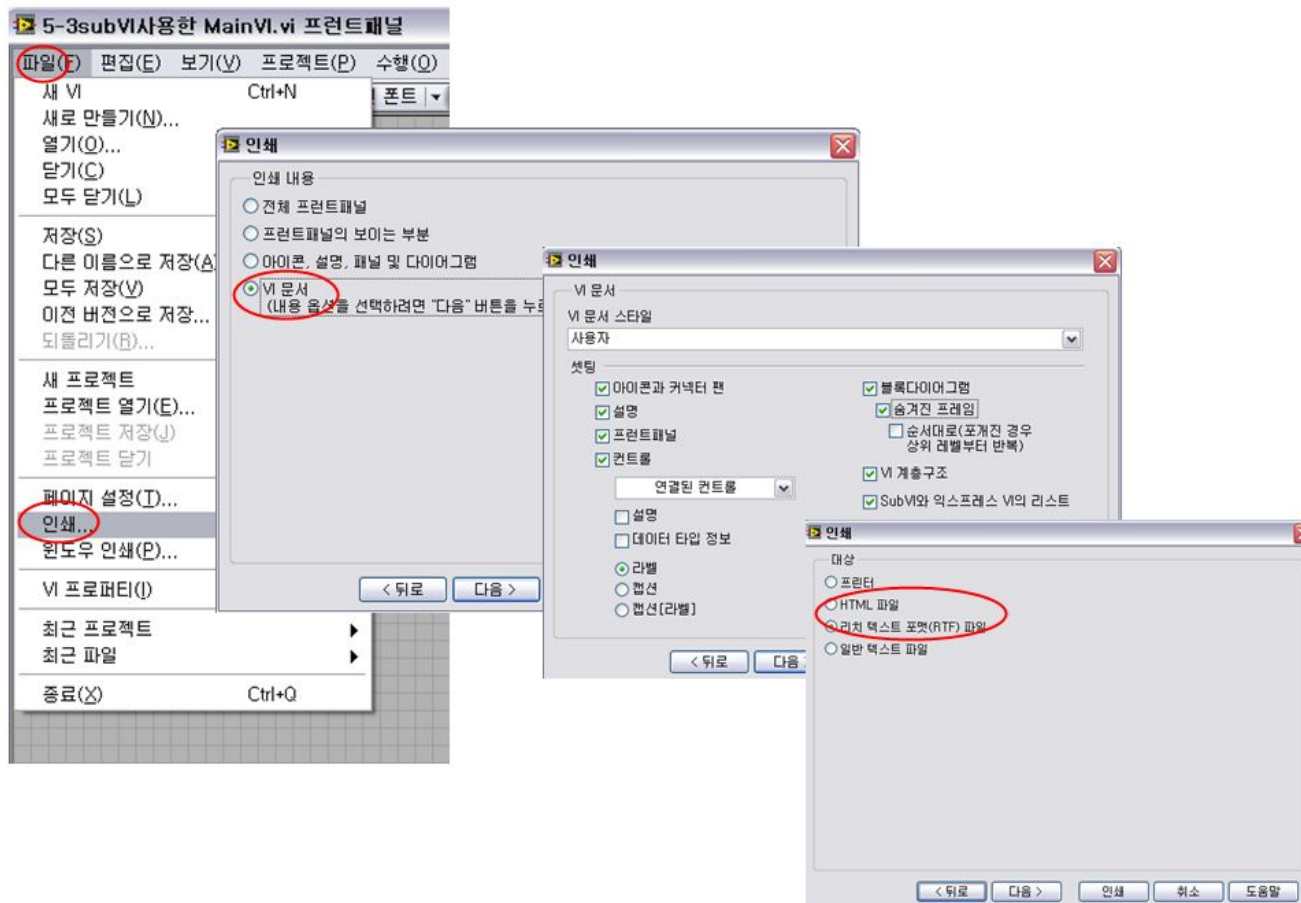
사용자 런타임 메뉴



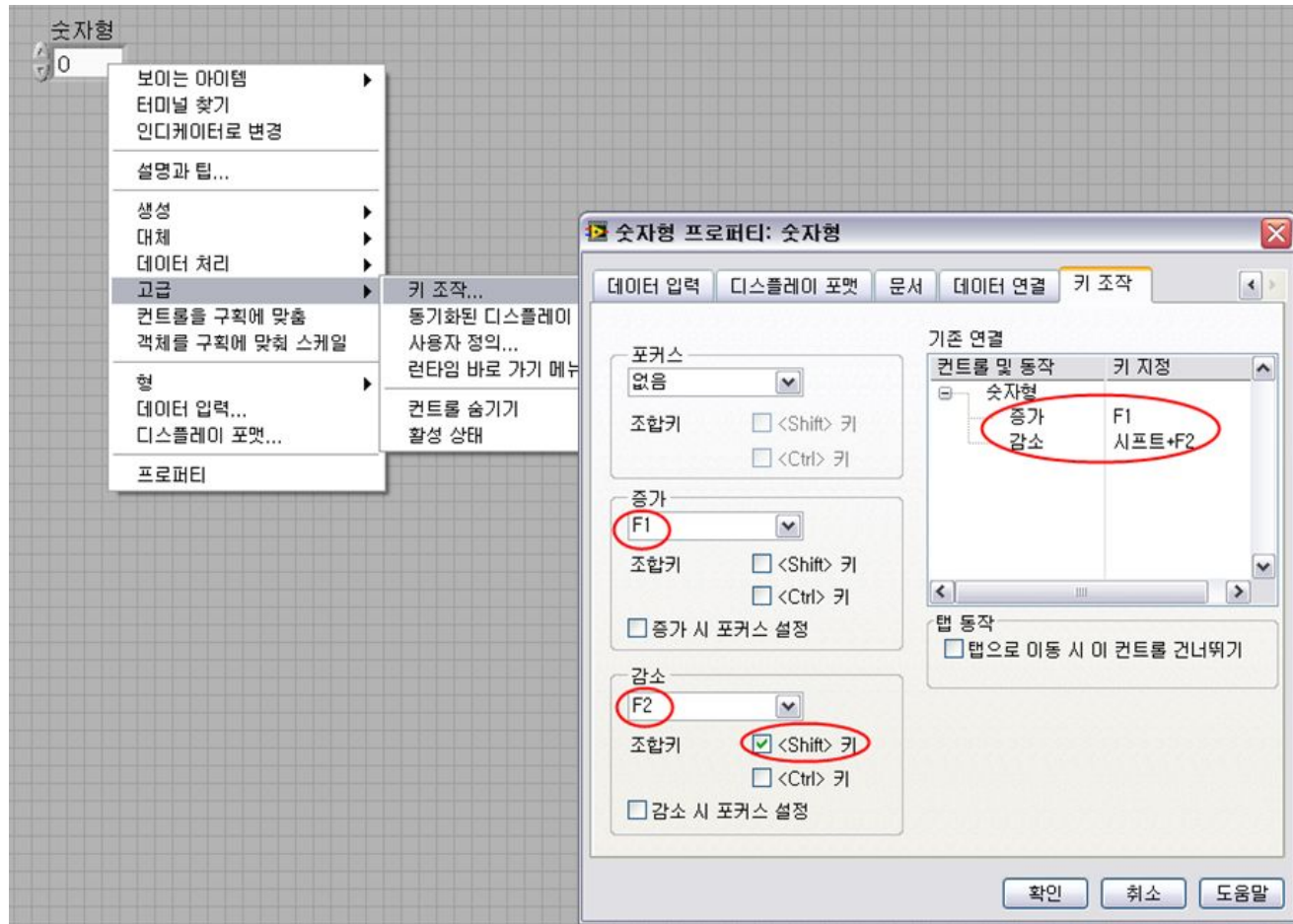
- 실행 중 파일폴다운메뉴를 사용자 정의 가능함.

실습11-5-1) 사용자 정의 런타임 메뉴

사용자 매뉴얼

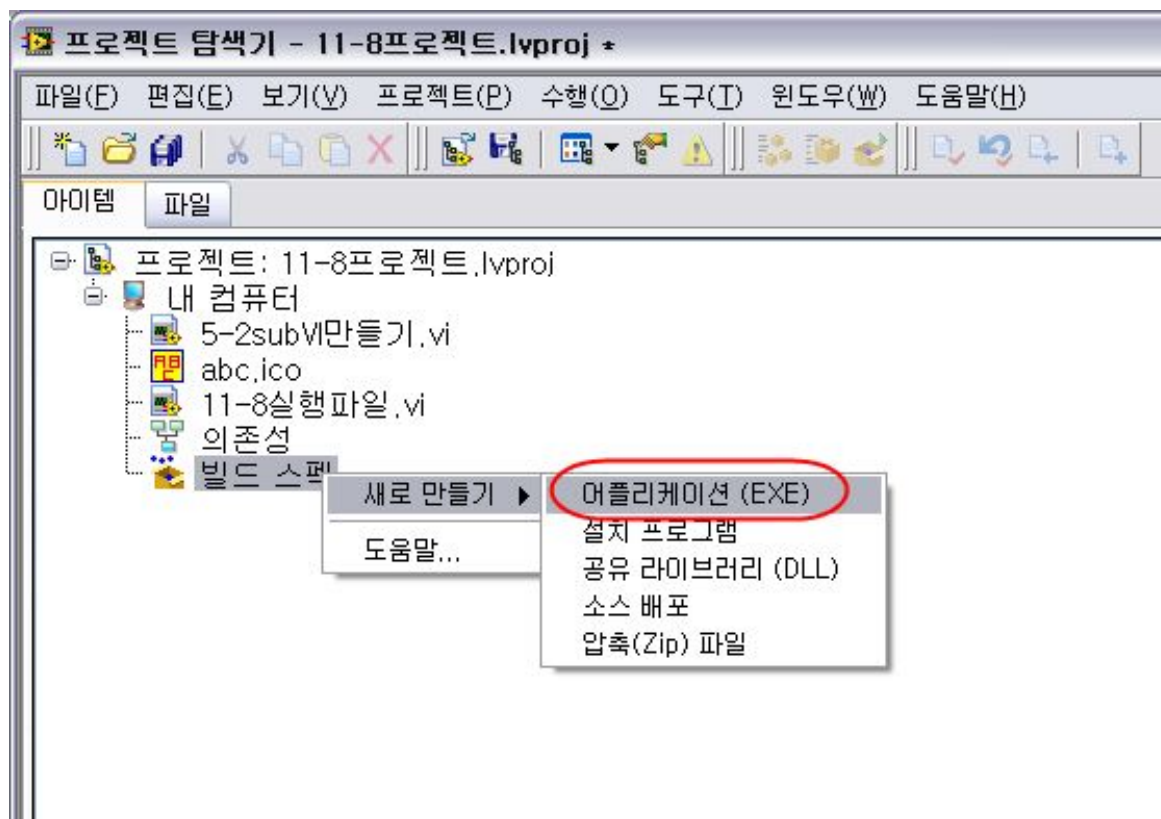


단축키 할당



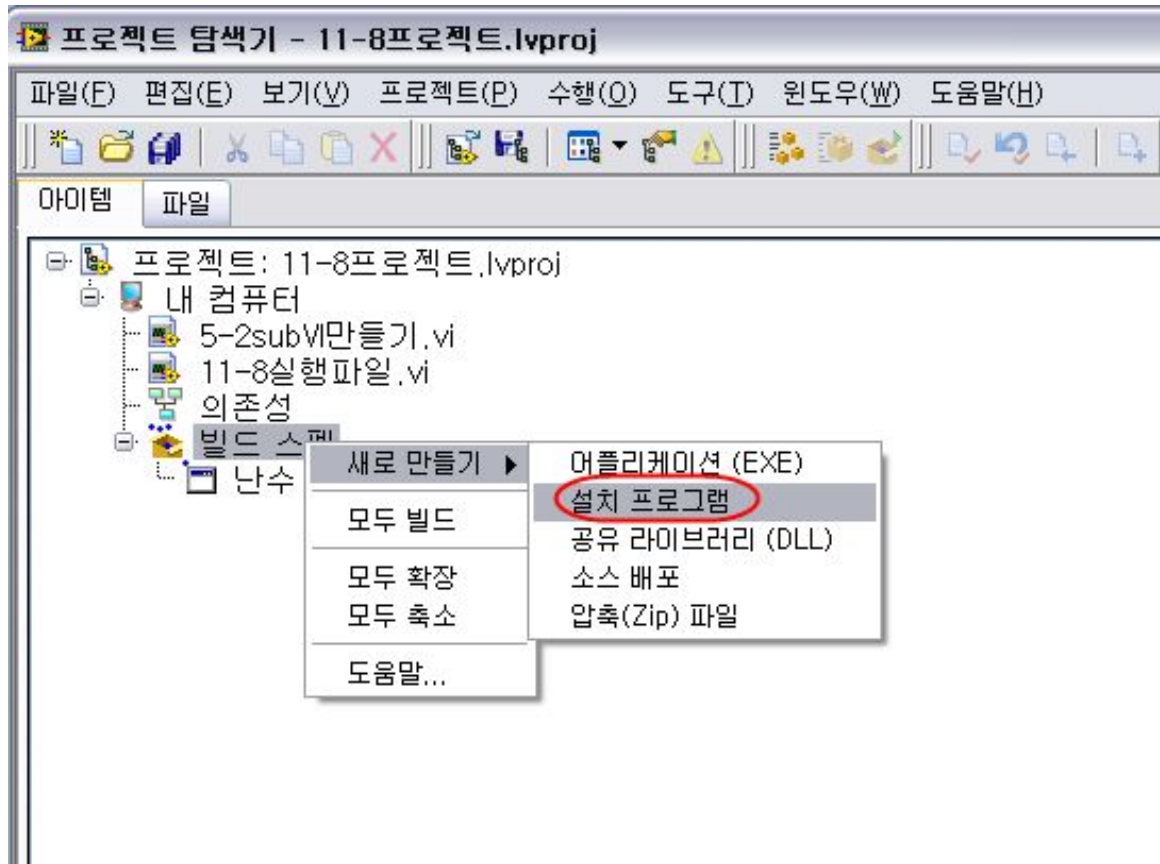
실습11-7-1) 단축키 할당

실행 파일



- 소스 코드 보안
- LabVIEW** 없이도 실행 가능.
(단 **LabVIEW** 런타임 엔진 있어야 함. 무료 다운로드 가능)

설치 파일



- 실행파일 (.exe)를 실행하는데 필요한 **LabVIEW** 런타임엔진, 하드웨어 드라이버 등을 포함.

11장 요약

- 성능과 메모리
- **VI** 메트릭스
- **VI** 비교
- 사용자 에러 정의
- 사용자 정의 런타임 메뉴
- 사용자 매뉴얼
- 단축키 할당
- 실행 파일
- 설치 파일

END

단축키

Ctrl+U : 와이어, 객체 정리

Ctrl + Mouse.L : 블록 다이어그램 늘리기

Ctrl + B : 깨진 와이어 지우기

Ctrl + Shift + N : 탐색

Ctrl + N : 새vi

Ctrl + SPACE : 빠른탐색

Ctrl + E : 전환